

面试题整理_含答案

综合整理自：面试题汇总 (个人学习总结)、面试题总结 (往期同学)、PDF 面试过程问题

优先级	分类
★★★★	Linux 基础、K8S、Docker
★★★	网络、Nginx、数据库、监控
★★	CI/CD、Shell、系统优化、安全
★	JVM、Ansible、信创、高可用、云服务

综合开放

自我介绍

答案：自我介绍需要包含个人基本信息、过往公司情况、熟悉的技术栈，例如：“我有 X 年运维 / SRE 经验，之前在 XX 公司负责 XX 业务的运维工作，熟悉 Linux、K8s、Docker、Nginx、MySQL 等技术，主导过 XX 项目的容器化迁移，实现了 XX 效果。”

自我介绍 (公司情况 + 自己熟悉的技术栈)

答案：先介绍原公司的业务方向，比如“原公司是做 XX 领域的互联网公司，主要服务 XX 用户，团队规模 XX”，然后介绍自己熟悉的技术栈，比如“我熟悉 Linux 系统运维、容器化技术（Docker/K8s）、CI/CD 流程搭建、监控告警体系（Prometheus/Grafana）、数据库运维（MySQL 主从、Redis）、网络排障等，能够独立完成从 0 到 1 的服务部署与运维工作。”

你们公司团队情况，你在里面的主要角色是什么？

答案：团队规模一般 X 人，分为运维开发、运维、监控等小组，我主要负责业务的日常运维、故障排查、服务部署与优化，同时参与容器化平台的搭建与维护，保障业务的高可用与稳定性。

你主要在哪些方面投入的精力比较多？

答案：主要精力投入在业务的稳定性保障、容器化平台的运维与优化、监控告警体系的完善，以及自动化运维工具的开发，减少重复性工作，提升运维效率。

你觉得你擅长哪一方面？

答案：我比较擅长故障排查与性能优化，能够快速定位系统、网络、数据库等层面的问题，同时在容器化与 K8s 集群的运维方面有比较丰富的经验，能够处理集群的各种异常问题。

你对运维工作的理解？从技术方面，从团队方面？

答案：

- **技术方面：**运维是保障业务稳定运行的核心，通过自动化、监控、容灾等手段，保障服务的高可用、高性能，同时支撑业务的快速迭代，提升研发效率。
- **团队方面：**运维需要和研发、测试、产品等团队协作，推动 DevOps 文化，打通研发到运维的流程，同时做好团队内部的知识沉淀，提升整体团队的运维能力。

你觉得一个好的运维工程师好的胜任力是哪几点？

答案：

1. **故障排查能力：**能够快速定位问题，有清晰的排查思路，处理突发故障。
2. **自动化能力：**能够将重复性工作自动化，提升效率，减少人为错误。
3. **学习能力：**运维技术更新快，需要持续学习新的技术，比如云原生、监控等。
4. **责任心：**对业务稳定性负责，能够主动发现问题，提前预防故障。
5. **沟通能力：**能够和不同团队协作，推动问题解决。

你对现在找工作有什么想法吗？

答案：希望能够找到一个更有挑战的平台，能够接触到更大规模的集群和业务，提升自己的技术能力，同时能够为公司的业务稳定性贡献自己的力量。

学习方面你自己有什么想法？

答案：计划深入学习云原生相关的技术，比如 K8s 的高级特性、服务网格等，同时学习监控与可观测性的相关技术，提升自己的架构能力，同时也会学习一些研发相关的知识，更好的和研发团队协作。

毕业之后怎么会想到往运维方向去发展的？

答案：一开始接触到服务器运维，发现运维能够接触到整个技术栈，从系统、网络到应用，能够解决很多实际的问题，保障业务的运行，很有成就感，同时运维的技术广度很大，能够持续学习，所以就选择了运维方向。

你的未来发展规划是什么？

答案：短期来说，希望能够深入学习云原生运维，成为资深的 SRE 工程师，能够处理大规模集群的运维问题；长期来说，希望能够往架构师方向发展，设计高可用、可扩展的运维架构，支撑业务的快速发展。

你有没有做过体系化线上的运维 (资源监控、容量管理、数据监控、巡检、事故处理、资源管理)?

答案：做过，搭建了完整的监控体系，包括系统资源监控、业务数据监控，实现了容量管理，根据业务负载提前扩容，同时做了自动化巡检，每天自动检查系统的各项指标，发现问题自动告警，事故处理方面，建立了完整的应急响应流程，分级处理事故，同时做了事后复盘，避免问题重复发生。

你对国产信创系统的理解是什么？跟其他操作系统最主要的区别是什么？

答案：信创系统是国产化的信息技术应用创新系统，主要是为了实现信息技术的自主可控，替代国外的产品。和普通操作系统的区别主要是：

1. 自主可控：内核或者整体是国产的，能够自主掌控，避免卡脖子问题。
2. 兼容性：需要适配国产的硬件、软件，比如鲲鹏芯片、国产数据库等。
3. 安全：有更强的安全特性，满足国内的安全合规要求。

遇到没有接触过没做过的东西如何做？

答案：首先会先查阅官方文档，了解这个东西的基本原理和使用方法，然后在测试环境进行测试，同时会去社区、论坛查找相关的资料，看看有没有其他人遇到过类似的问题，如果有问题的话，也会请教有经验的同事，快速上手，确保能够解决问题。

测试后没有报错后就可以直接部署到生产环境吗？

答案：不会，测试环境没有报错之后，还需要进行灰度发布，先部署到部分节点，观察业务的运行情况，确认没有问题之后，再逐步全量发布，同时需要做好回滚的准备，如果出现问题，可以快速回滚到之前的版本，保障生产环境的稳定。

你对加班和出差怎么看？能接受吗？

答案：如果是因为突发故障或者项目上线的需要，我可以接受加班，因为保障业务的稳定是运维的责任，出差的话，如果是工作需要，我也可以接受。

为什么考虑离职？到岗时间大概是多久？期望薪资多少？

答案：主要是希望能够有更大的平台，接触到更大规模的业务，提升自己的技术能力。到岗时间的话，我需要 X 周的交接时间，期望薪资是 XX。

有对象吗？结婚了吗？未来几年有结婚打算吗？

答案：（根据实际情况回答，一般可以说目前专注于工作，或者有相关的计划，不影响工作）

项目与业务

你们公司主要负责什么业务？

答案：我们公司主要做 XX 业务，比如电商、或者企业服务，服务 XX 量级的用户，每天的请求量大概 XX。

你们公司是自研还是外包？

答案：我们公司是自研业务，所有的产品都是自己研发的。

你们项目周期是多少？

答案：项目周期一般是 X 周，根据项目的大小不同，小项目可能 2 周，大项目可能 1-2 个月。

挑一个项目介绍一下，什么类型的项目，怎么做的？

答案：比如容器化迁移项目，之前我们的业务都是物理机部署，管理起来很麻烦，资源利用率也低，所以我们做了容器化迁移，把业务都迁移到 K8s 集群上，过程中我们先做了测试环境的迁移，验证了兼容性，然后逐步把生产环境的业务迁移过去，同时做了监控和告警的适配，最后实现了资源利用率提升了 X%，部署效率提升了 X 倍。

你们集群有多大？

答案：我们的 K8s 集群有 X 个节点，管理着 X 个 Pod，支撑着 XX 个业务。

你们公司服务器有多少台？

答案：物理机 + 云服务器大概 X 台，主要是运行业务和中间件。

你们小组多少人？组内的人是做什么的？

答案：我们小组有 X 人，其中 X 人负责业务运维，X 人负责平台运维，X 人负责监控和自动化，我主要负责业务运维和部分平台的工作。

你负责的部署的项目有多少？部署的方式有哪些？

答案：我负责大概 XX 个项目的部署，部署方式主要是 K8s 的滚动更新，还有部分是通过 Jenkins 做 CI/CD，自动部署。

你们的产品是部署到自己服务器上还是客户服务器上？

答案：我们的产品是 SaaS 模式，部署在我们自己的服务器上，客户通过公网访问。

你们用到多少容器？正常部署一个项目大概多少个容器？

答案：我们大概有 XX 个容器，正常部署一个项目，一般是 2 个副本，也就是 2 个容器，根据业务的负载不同，有的业务会更多。

你工作主要负责什么？

答案：我主要负责业务的日常运维，包括部署、故障排查、监控告警，同时参与 K8s 集群的维护，以及自动化工具的开发。

你们部署新业务的逻辑是什么，从 0 到 1？

答案：

1. 首先和研发沟通，了解业务的需求，比如资源需求、端口、存储等。
2. 然后在 K8s 集群上创建命名空间，配置资源配额。
3. 编写 Deployment、Service、Ingress 等资源清单，配置探针、资源限制。
4. 配置 CI/CD 流程，打通代码提交到自动部署的流程。
5. 配置监控和告警，监控业务的运行指标。
6. 测试环境部署验证，没问题之后部署到生产环境。

你平常工作对接开发吗？

答案：是的，平常会和开发对接，比如部署需求、故障排查、性能优化等，帮助开发解决部署和运行中的问题。

你们公司之前业务主要用什么开发的？

答案：主要是 Java、Go，还有部分前端是 Vue。

你对我们公司有什么了解吗？研究过我们公司的产品吗？

答案：我了解到贵公司是做 XX 业务的，在行业内很有影响力，你们的 XX 产品我也用过，体验很好，我很看好公司的发展。

容量管理怎么做的？

答案：

1. 监控业务的负载趋势，比如 CPU、内存、磁盘的使用情况，预测未来的容量需求。
2. 定期做容量评估，根据业务的增长，提前扩容，避免资源不足。
3. 做资源的优化，比如调整 Pod 的 Request 和 Limit，提升资源利用率。
4. 对闲置的资源进行回收，比如长时间没有流量的服务，进行缩容。

巡检怎么做的？是什么机制？

答案：做了自动化巡检，通过脚本每天定时检查系统的各项指标，比如磁盘使用率、CPU 负载、服务状态、数据库状态等，检查完之后生成巡检报告，发送到群里，如果有异常的话，会自动告警，同时也会有手动的巡检，每周对集群做一次全面的检查。

应急响应怎么做的？机制是啥？

答案：建立了应急响应机制，故障发生之后，首先拉应急群，相关的人员进入，然后按照故障等级进行处理，P0 故障需要所有人立刻响应，P1 的话工作时间内响应，同时有故障升级机制，如果一段时间没解决，就升级到更高层级的人员处理，故障处理完之后，做复盘，总结问题，避免重复发生。

事故管理是怎么做的？(生产故障、运维事故、管理分级、事故判断)

答案：

1. **事故分级：**分为 P0、P1、P2、P3，P0 是核心业务不可用，影响所有用户，P1 是核心业务部分不可用，影响部分用户，P2 是非核心业务不可用，P3 是小问题，不影响用户。
 2. **事故处理：**不同等级的事故有不同的响应时间，P0 需要 5 分钟内响应，30 分钟内恢复，P1 的话 30 分钟内响应，2 小时内恢复。
 3. **复盘：**事故处理完之后，1-3 天内做复盘，分析事故的原因，制定改进措施，跟进落实。
 4. **考核：**把事故纳入团队的考核，督促大家提升稳定性。
-

开放性问题

正在运行的系统业务突然 502 都有哪些层面的原因？

答案：

1. **应用层面：**后端服务挂了，或者进程崩溃了，导致 Nginx 无法转发请求。
2. **Nginx 层面：**Nginx 的配置错误，或者 upstream 的节点都不可用，或者 Nginx 进程挂了。
3. **网络层面：**后端服务器网络不通，或者端口没开放，防火墙拦截了。
4. **负载层面：**后端服务负载过高，无法处理请求，导致超时。
5. **K8s 层面：**如果是 K8s 的话，Pod 挂了，或者 Service 没有后端端点，Ingress 配置错误。

系统出现一个十分火急的故障你怎么办？

答案：

1. **先恢复业务：**首先优先恢复业务，比如先重启服务，或者切流量到备用节点，先让用户能用。
2. **拉应急群：**拉相关的人员进群，同步故障情况。
3. **排查原因：**同时排查故障的原因，看是哪里的的问题，比如系统、网络、应用。
4. **解决问题：**根据排查的原因，解决问题，比如修复配置，重启进程，扩容等。
5. **复盘：**故障恢复之后，做复盘，分析原因，制定改进措施，避免下次再发生。

无效告警有哪些原因，你该如何抑制无效告警？

答案：

无效告警的原因：

1. 告警阈值设置不合理，比如阈值太低，导致频繁告警。
2. 监控的指标有波动，比如短时间的峰值，导致误告警。
3. 告警没有做聚合，比如同一个故障，触发了很多个告警。
4. 监控的对象已经下线了，但是监控没删除，导致告警。

抑制无效告警的方法：

1. 调整告警阈值，根据业务的实际情况，设置合理的阈值，比如带波动的指标，设置持续时间，比如持续 5 分钟超过阈值才告警。
2. 做告警聚合，把同一个故障的告警聚合起来，只发一个告警。
3. 定期清理无效的监控，比如下线的服务，删除对应的监控。
4. 做告警抑制，比如某个父告警触发了，就抑制子告警，比如节点宕机了，就抑制这个节点上所有 Pod 的告警。
5. 做告警降噪，比如相同的告警，短时间内只发一次。

有很多重复性的工作你该怎么处理，会选择躺平吗？

答案：不会躺平，我会把这些重复性的工作自动化，比如写脚本，或者用工具，把这些工作做成自动化的，比如部署、巡检、备份这些，都做成自动化的，这样就不用手动做了，既提升了效率，也减少了人为错误，同时我也可以有更多的时间去做更有价值的事情，比如优化、架构改进。

印象比较深刻的故障有哪些？

答案：比如之前有一次，K8s 节点的磁盘满了，导致 Pod 无法写入日志，业务报错，当时排查了很久，最后发现是某个服务的日志没有轮转，导致日志文件把磁盘占满了，之后我们就做了日志轮转的配置，同时加了磁盘使用率的告警，提前发现问题，避免再发生。

遇到过一些容量问题吗？

答案：遇到过，比如有一次大促，业务的流量突然涨了很多，导致 Pod 的资源不够，请求超时，之后我们就做了容量预测，提前扩容，同时做了 HPA，根据负载自动扩缩容，应对突发的流量。

你每天做什么？给你一个服务器流程是什么？

答案：

每天的工作：

1. 早上看监控和告警，处理前一天的告警。
2. 看巡检报告，检查系统的状态。
3. 处理研发的部署需求，部署服务。
4. 排查故障，处理用户的问题。
5. 做一些优化和自动化的工作。

给我一个服务器的话，流程是：

1. 首先检查系统的基本信息，比如系统版本、内核版本、CPU、内存、磁盘。
2. 检查系统的负载，CPU、内存、磁盘、网络的使用情况。
3. 检查进程和服务，看有没有异常的进程。
4. 检查日志，看有没有错误日志。
5. 检查网络，看网络通不通，端口有没有开放。
6. 检查安全，比如有没有漏洞，有没有挖矿进程。

你们平时报警多吗？大的报警多吗？为什么大的报警不多？

答案：平时的告警不多，大的告警更少，因为我们做了很多提前的预防，比如监控、巡检，提前发现问题，在问题变大之前就解决了，同时我们做了高可用，比如集群、主从，即使某个节点出问题，也不会影响业务，所以大的故障很少。

如何保证服务器容灾问题？

答案：

1. **多可用区部署：**把服务部署在多个可用区，即使一个可用区出问题，另一个可用区的服务还能运行。
2. **集群化部署：**服务做集群，多个节点，一个节点挂了，其他节点还能处理请求。
3. **数据备份：**定期备份数据，比如数据库的备份，文件的备份，即使数据丢了，也能恢复。
4. **容灾演练：**定期做容灾演练，验证容灾方案的有效性。
5. **监报告警：**监控各个节点的状态，发现问题及时处理。

物理机没电怎么去部署服务？

答案：如果物理机没电的话，首先看有没有 UPS，先撑一段时间，然后启动备用的发电机，同时把服务切换到其他可用区的服务器上，保障业务不中断，等电恢复之后，再把服务切回来。

如何评估一个网络的质量？除了看带宽外还有哪些方式？

答案：

1. **延迟：**ping 的延迟，或者 mtr 的延迟，看网络的延迟是不是稳定。
2. **丢包率：**看网络的丢包情况，有没有丢包。
3. **抖动：**延迟的波动，也就是抖动，抖动大的话，对实时业务影响很大。
4. **吞吐量：**实际的传输速度，能不能达到带宽的上限。
5. **连通性：**各个节点之间的连通性，有没有网络分区。

带宽的单位是什么？是大 B 还是小 B？

答案：带宽的单位一般是 bps，也就是 bit per second，是小 b，而我们下载的速度是 Byte，大 B，1Byte=8bit，所以比如 100M 的带宽，下载速度大概是 12.5MB/s。

如果用户反馈系统服务无法通过公网访问，从哪些方面入手排查？

答案：

1. **客户端层面：**先看用户的网络是不是有问题，能不能 ping 通公网，能不能访问其他网站。
2. **网络层面：**
 - 检查服务器的公网 IP 是不是正常，能不能 ping 通服务器的公网 IP。
 - 检查服务器的端口是不是开放，telnet 或者 nc 测试端口通不通。
 - 检查防火墙，服务器的防火墙有没有放开端口，安全组有没有放开。
 - 检查路由，服务器的公网路由是不是正常。
3. **应用层面：**检查服务是不是正常运行，端口是不是监听了，有没有报错日志。
4. **负载均衡层面：**如果有 SLB 或者 Nginx 的话，检查负载均衡的配置是不是正常，后端节点是不是健康。
5. **DNS 层面：**检查域名解析是不是正常，有没有解析到正确的 IP。

如果客户给你打电话说系统上不去了，怎么办？

答案：

1. 先安抚客户，说我马上排查，很快给你回复。
2. 立刻查看监控，看系统的状态，是不是有告警。
3. 排查问题，比如网络、服务、数据库，看是哪里的的问题。
4. 优先恢复业务，比如重启服务，切流量，先让客户能用。
5. 解决问题之后，告诉客户问题解决了，然后做复盘，避免下次再发生。

客户提出需求后你的具体处理流程是什么？

答案：

1. 先了解客户的需求，确认需求的细节，比如要做什么，什么时候要。
2. 评估需求的可行性，需要什么资源，有没有风险。
3. 和客户沟通，确认需求的时间和方案。
4. 执行需求，比如部署、配置，在测试环境验证。
5. 交付给客户，让客户测试，确认没问题。
6. 后续跟进，看有没有问题。

Linux 基础

系统操作与常用命令

如何查看服务器 CPU 使用率？

答案：

- `top`：实时查看 CPU 使用率，还有进程的 CPU 占用。
- `htop`：top 的增强版，更友好的界面。
- `mpstat`：查看每个 CPU 的使用率，比如 `mpstat 1 5`，每秒查看一次，共 5 次。
- `cat /proc/stat`：查看 CPU 的统计信息。

如何查看磁盘占用量？

答案：

- `df -h`：查看磁盘分区的使用情况，h 是人类可读的格式，显示 GB、MB。
- `du -sh *`：查看当前目录下各个目录的大小，查看哪个目录占用了空间。

如何查看服务器空闲内存？

答案：

- `free -h`：查看内存的使用情况，包括总内存、已用、空闲、缓存。
- `cat /proc/meminfo`：查看详细的内存信息。

如何查看进程是否在线？

答案：

- `ps aux | grep 进程名`：查看进程是否存在。
- `pgrep 进程名`：查看进程的 PID，如果有输出说明进程存在。
- `systemctl status 服务名`：查看系统服务的状态，是否运行。

如何查询端口是否被占用？

答案：

- `netstat -tulnp | grep 端口号`：查看端口的占用情况。
- `ss -tulnp | grep 端口号`：比 netstat 更高效的命令。
- `lsof -i:端口号`：查看哪个进程占用了这个端口。

如何杀死进程？

答案：

- `kill PID`：正常终止进程，发送 SIGTERM 信号，让进程自己退出。
- `kill -9 PID`：强制终止进程，发送 SIGKILL 信号，直接杀死进程。
- `pkill 进程名`：根据进程名杀死进程。
- `killall 进程名`：杀死所有同名的进程。

列举五个常用的 Linux 指令及其用途

答案：

1. `ls`：列出目录下的文件和目录。
2. `cd`：切换当前目录。
3. `cp`：复制文件或目录。
4. `mv`：移动或重命名文件或目录。
5. `rm`：删除文件或目录。
6. 其他常用的：`grep`（查找文件中的内容）、`tar`（打包压缩）、`ssh`（远程连接）等。

请列出你最常使用的 Linux 命令及其用途

答案：

- `top/htop`：查看系统负载和进程状态。
- `df/du`：查看磁盘使用情况。
- `free`：查看内存使用情况。
- `ps`：查看进程。
- `netstat/ss`：查看网络连接和端口。
- `grep`：过滤文本。
- `tail`：查看日志，比如 `tail -f` 实时查看。
- `scp`：远程复制文件。
- `vim`：编辑文件。
- `systemctl`：管理系统服务。

查看系统版本用什么命令？内核版本怎么查？

答案：

- 查看系统版本：`cat /etc/os-release`，或者 `lsb_release -a`。
- 查看内核版本：`uname -r`，或者 `cat /proc/version`。

uptime 会显示哪些内容？

答案：

显示当前时间、系统已经运行了多长时间、当前登录的用户数、过去 1 分钟、5 分钟、15 分钟的系统平均负载。

查看 linux 网络流量以及带宽使用什么命令？

答案：

- `nload`：查看网卡的实时流量。
- `iftop`：查看各个连接的流量。
- `sar -n DEV 1 10`：查看网络流量，每秒一次，共 10 次。
- `iptraf`：交互式的网络流量监控。

如何查看系统负载？怎样定义它是高了还是低了？

答案：

- 查看系统负载：`uptime`，或者 `top`，看 1 分钟、5 分钟、15 分钟的平均负载。
- 负载的高低：一般来说，负载值不超过 CPU 的核心数，比如 8 核的 CPU，负载到 8 是正常的，超过 8 的话说明负载高了，CPU 处理不过来。

top 一般都看哪些参数？

答案：

- 首先看系统的负载：load average。
- 然后看 CPU 的使用率：%us（用户态）、%sy（内核态）、%id（空闲）。
- 内存的使用：Mem 的 total、used、free、buff/cache。
- 进程的信息：PID、USER、%CPU、%MEM、COMMAND，看哪个进程占用的 CPU 和内存高。

top 命令后面可以加什么参数？

答案：

- `top -p PID`：只查看指定的进程。
- `top -u 用户`：查看指定用户的进程。
- `top -n 次数`：刷新多少次之后退出。
- `top -d 秒数`：设置刷新的间隔时间。
- `top -H`：显示线程信息。

top 和 lscpu 的区别？

答案：

- `top`：是实时的系统监控工具，查看系统的负载、进程的状态、CPU 的使用率等动态的信息。
- `lscpu`：是查看 CPU 的硬件信息，比如 CPU 的核心数、架构、线程数、缓存大小等静态的信息，是硬件的属性。

ping 使用的是什么协议？工作在几层？

答案：

ping 使用的是 ICMP 协议，工作在网络层（OSI 七层模型的第三层）。

如何查看服务器网卡的上连信息 (包括接口、mac、名称等信息)？

答案：

- `ip addr`：查看网卡的信息，包括名称、MAC 地址、IP 地址。
- `ifconfig`：传统的命令，查看网卡信息。
- `ethtool 网卡名`：查看网卡的驱动、速率、端口等信息，比如 `ethtool eth0`。

File.txt 文件有 400 行，如何查看文件的第 200-300 行 (两种方法以上)？

答案：

1. `sed -n '200,300p' File.txt`：sed 命令，打印 200 到 300 行。
2. `awk 'NR>=200 && NR<=300' File.txt`：awk 命令，处理行号。
3. `head -n 300 File.txt | tail -n 101`：先取前 300 行，然后取最后 101 行，也就是 200-300 行。
4. `vim File.txt`：然后输入 `:200` 跳到 200 行，然后往下看。

用什么命令查看文件的详细信息 (stat)？

答案：

使用 `stat 文件名`，可以查看文件的大小、权限、创建时间、修改时间、访问时间等详细的元信息。

`du -h -m /* | sort -nr | head -10` 这条命令执行代表什么意思？

答案：

这条命令是查看根目录下各个目录的大小，然后排序，找出最大的 10 个目录：

- `du -h -m /*`：查看根目录下的各个目录的大小，-m 是以 MB 为单位，-h 是人类可读。
- `sort -nr`：排序，-n 是按数字排序，-r 是倒序，也就是从大到小。
- `head -10`：取前 10 个，也就是最大的 10 个。

如何查看系统启动的端口？

答案：

- `netstat -lntp`：查看所有监听的端口，l 是监听，n 是不解析域名，t 是 TCP，p 是进程。
- `ss -lntp`：和 netstat 类似，更高效。

统计目录文件数量用什么命令 (tree)?

答案：

- `tree 目录`：可以显示目录的树状结构，同时显示文件和目录的数量。
- 如果不用 tree 的话，可以用 `find 目录 -type f | wc -l`：统计目录下的文件数量。

如何实时查看日志文件？

答案：

`tail -f 日志文件`：实时查看日志的新增内容，-f 是 follow，跟踪文件的变化。
也可以用 `tail -F`，如果文件被轮转了，也能继续跟踪。

vi 编辑器内如何检索 error 字样？

答案：

在 vi 的命令模式下，输入 `/error`，然后回车，就可以向下搜索 error，按 n 可以找下一个，N 找上一个。

vi 编辑器内如何将 "北京" 批量替换为 "天津"？

答案：

在 vi 的命令模式下，输入 `:%s/北京/天津/g`，然后回车，就可以全局替换，% 是所有行，g 是一行内的所有匹配项。

如果要确认每个替换的话，可以加 c，`:%s/北京/天津/gc`，每个替换都会问你是否确认。

vim 编辑器如何保存退出？使用！呢？

答案：

- 保存退出：`:wq`，或者 `:x`。
- 如果是只读文件，或者要强制保存退出的话，`:wq!`。
- 如果不保存退出的话，`:q!`。

你上家公司都用过什么内核版本的操作系统？

答案：

主要用的是 CentOS 7，内核版本是 3.10，还有部分 CentOS 8，内核是 4.18，还有部分 Ubuntu 20.04，内核是 5.4。

DNS 的配置文件是哪个？

答案：

DNS 的配置文件是 `/etc/resolv.conf`，里面配置 nameserver，也就是 DNS 服务器的地址。

如何将一个进程放到后台持续运行 (screen/nohup)?

答案:

1. `nohup`: `nohup` 命令 & , 比如 `nohup python app.py &` , 这样进程就会放到后台运行, 即使退出终端, 进程也不会停, 输出会写到 `nohup.out` 里。
2. `screen`: 先输入 `screen` , 进入 `screen` 会话, 然后运行命令, 然后按 `Ctrl+A+D` , `detach` 会话, 进程就会在后台运行, 之后可以用 `screen -r` 恢复会话。
3. 还有 `tmux` , 和 `screen` 类似, 也是终端复用工具。

/etc/fstab 含义及每一列含义是什么?

答案:

`/etc/fstab` 是文件系统的挂载配置文件, 系统启动的时候会自动挂载里面的文件系统。

每一列的含义:

1. **FileSystem**: 要挂载的设备, 比如 `/dev/sda1`, 或者 UUID。
2. **Mount point**: 挂载点, 也就是挂载到哪个目录。
3. **Type**: 文件系统的类型, 比如 `ext4`、`xfs`、`swap`。
4. **Options**: 挂载选项, 比如 `defaults`、`rw`、`ro`、`noatime` 等。
5. **Dump**: 是否要 dump 备份, 0 是不备份, 1 是备份。
6. **Pass**: 开机自检的顺序, 0 是不自检, 1 是根分区, 2 是其他分区。

查看系统运行级别用什么命令?

答案:

- `runlevel`: 查看当前的运行级别。
- `systemctl get-default`: `systemd` 的系统, 查看默认的运行级别。

使用什么命令可以查看 Linux 的启动信息?

答案:

`dmesg`: 查看内核的启动信息, 也就是开机的时候的日志。

也可以查看 `/var/log/boot.log` , 里面是系统启动的日志。

如何查看当前目录位置 (pwd)?

答案:

`pwd`: Print Working Directory, 打印当前的工作目录, 也就是你现在所在的目录。

怎么看滚动日志?

答案:

用 `tail -f 日志文件`，实时跟踪日志的新增内容，也就是滚动日志，比如 `tail -f /var/log/nginx/access.log`。

系统的 7 个运行级别分别代表什么？

答案：

Linux 的运行级别：

- 0：关机，halt
 - 1：单用户模式，rescue 模式，用于维护
 - 2：多用户模式，没有 NFS
 - 3：完全的多用户模式，命令行模式，文本界面
 - 4：未使用，保留
 - 5：X11，图形化界面模式
 - 6：重启，reboot
-

文件与磁盘

磁盘空间快满了，你怎么做？

答案：

1. 首先排查是什么占用了空间，用 `df -h` 看哪个分区满了，然后用 `du -sh *` 看哪个目录大，然后找大文件。
2. 清理日志，比如过期的日志，或者没有用的日志文件。
3. 清理临时文件，比如 /tmp 下的过期文件。
4. 清理软件包的缓存，比如 `yum clean all`。
5. 如果是容器的话，清理无用的镜像和容器。
6. 如果清理完还是不够，就扩容磁盘，或者把数据迁移到其他磁盘。

删除大小很大的文件之后，磁盘空间没有释放，这是怎么回事？

答案：

因为这个文件被进程打开了，虽然删除了文件，但是进程还持有这个文件的句柄，所以磁盘空间不会释放，直到进程关闭这个文件。

当数据库程序一直写入日志文件，日志文件达到 2T,rm 删除后磁盘空间没有释放，在不重启程序的情况下如何处理？

答案：

可以用 `lsof | grep 文件名`，找到打开这个文件的进程，然后找到文件的描述符，比如 `/proc/PID/fd/文件描述符`，然后清空这个文件，比如 `cat /dev/null > /proc/PID/fd/FD`，这样就可以清空文件，释放空间，不用重启进程。

日志满了怎么处理？

答案：

1. 先清理过期的日志，释放空间。
2. 配置日志轮转，让日志自动切割和清理，避免日志太大。
3. 如果是业务日志，排查是不是有错误日志疯狂打印，导致日志暴涨，修复这个问题。
4. 把日志存储到其他的存储，比如 ELK，把日志存到 ES，不要存在本地。

日志轮转是什么？

答案：

日志轮转 (logrotate) 是一个日志管理工具，用来自动切割日志文件，当日志达到一定大小或者时间，就把旧的日志归档，然后创建新的日志文件，同时可以配置过期时间，自动删除旧的日志，避免日志文件把磁盘占满。

简单说一下挂盘的具体步骤

答案：

1. 查看磁盘，用 `fdisk -l`，找到新的磁盘，比如 `/dev/sdb`。
2. 分区，用 `fdisk /dev/sdb`，创建分区，比如 `/dev/sdb1`。
3. 格式化分区，`mkfs.ext4 /dev/sdb1`，或者 `xfs`。
4. 创建挂载点，`mkdir /data`。
5. 挂载，`mount /dev/sdb1 /data`。
6. 配置开机自动挂载，编辑 `/etc/fstab`，添加一行 `/dev/sdb1 /data ext4 defaults 0 0`，然后执行 `mount -a` 验证。

磁盘具体挂载流程是什么？

答案：

和上面的挂盘步骤一样，先分区，然后格式化，然后挂载，然后配置 `fstab` 开机自动挂载。

逻辑卷怎么扩容？怎么缩容？

答案：

扩容：

1. 如果有新的磁盘，先初始化物理卷：`pvccreate /dev/sdb`。
2. 把物理卷加入卷组：`vgextend 卷组名 /dev/sdb`。
3. 扩展逻辑卷的大小：`lvextend -L +100G /dev/卷组名/逻辑卷名`。
4. 调整文件系统的大小：`resize2fs /dev/卷组名/逻辑卷名` (ext4)，如果是 `xfs` 的话，用 `xfs_growfs`。

缩容：

缩容比较危险，需要先卸载文件系统，然后检查文件系统，然后缩小文件系统，然后缩小逻辑卷，然后重新挂载。

1. 卸载：`umount /dev/卷组名/逻辑卷名`。
2. 检查文件系统：`e2fsck -f /dev/卷组名/逻辑卷名`。
3. 缩小文件系统：`resize2fs /dev/卷组名/逻辑卷名 50G`。
4. 缩小逻辑卷：`lvreduce -L 50G /dev/卷组名/逻辑卷名`。
5. 重新挂载：`mount /dev/卷组名/逻辑卷名 挂载点`。

Linux 的逻辑卷有涉及吗？和普通逻辑卷有什么区别？

答案：

涉及过，逻辑卷 LVM 是 Linux 的逻辑卷管理，和普通的分区相比：

1. 灵活：可以动态的扩容缩容，不用像普通分区那样，要重新分区，风险大。
2. 可以跨磁盘：逻辑卷可以把多个磁盘组成一个卷组，然后逻辑卷可以跨磁盘，普通分区不行。
3. 快照：可以做快照，备份数据，普通分区没有。

df 和 du 有什么区别？

答案：

- `df`：是从文件系统的层面，查看磁盘的整体使用情况，是文件系统的统计，包括已经分配的空间，即使文件被删除了，但是进程还没释放，df 也会算进去。
- `du`：是从文件的层面，查看文件和目录占用的空间，是统计当前可见的文件的大小，如果文件被删除了，du 就看不到了，所以有时候 df 显示满了，du 却没找到大文件，就是因为有被删除的文件还被进程持有。

磁盘挂载后重启读取不了数据怎么解决？

答案：

一般是因为 `/etc/fstab` 里的配置错了，比如设备名写错了，或者文件系统类型写错了，导致开机挂载失败。

解决方法：

1. 开机进入紧急模式，或者单用户模式。
2. 编辑 `/etc/fstab`，把错误的那一行注释掉，然后重启。
3. 重启之后，重新检查磁盘的信息，比如 UUID，然后修改 `fstab` 的配置，正确配置，然后 `mount -a` 验证。

服务器运行一段时间后磁盘满了怎么排查？

答案：

1. 先用 `df -h` 看哪个分区满了。
2. 然后用 `du -sh /*` 看根目录下哪个目录大，一步步往下找。
3. 找大文件，`find / -size +100M`，找大于 100M 的文件。
4. 检查是不是有被删除的文件还被进程持有，用 `lsof | grep deleted`，看有没有 deleted 的文件，如果有的话，重启对应的进程，或者清空文件。
5. 检查日志文件，是不是日志没轮转，太大了。

ext2、ext3、ext4 会默认预留多少磁盘空间？用于什么？

答案：

默认预留 5% 的磁盘空间，留给 root 用户，这样即使普通用户把磁盘占满了，root 用户还能登录，清理文件，修复问题，避免系统完全无法使用。

inode 或者 block 满了会导致什么？

答案：

- block 满了：就是磁盘空间满了，无法写入新的文件。
- inode 满了：是索引节点满了，说明有太多的小文件，把 inode 用完了，这时候即使磁盘还有空间，也无法创建新的文件，因为没有 inode 了。

如何查找出系统中的大文件？

答案：

`find / -type f -size +100M -exec ls -lh {} \;`：查找根目录下大于 100M 的文件，并且显示文件的大小。

或者 `du -h / | sort -rh | head -20`，查找最大的目录。

查找系统中大于 30 天前的文件并且删除？

答案：

`find /path -type f -mtime +30 -delete`：查找 /path 下，修改时间大于 30 天的文件，然后删除。

-mtime 是修改时间，+30 就是 30 天前的。

服务器不关机的情况下挂载的目录卸载不掉该怎么办？如果进程也杀不死呢？

答案：

卸载不掉是因为有进程正在使用这个目录，用 `lsof 挂载点` 或者 `fuser -vm 挂载点`，找到正在使用这个目录的进程，然后杀掉进程，就可以卸载了。

如果进程杀不死的话，可以用强制卸载，`umount -f 挂载点`，强制卸载，或者如果是 NFS 的话，用 `umount -l`，懒卸载，等进程不用了再卸载。

往文件系统里面写数据发现无法写入可能是哪些原因？

答案：

1. 磁盘满了，block 满了。
2. inode 满了，无法创建新文件。
3. 文件系统是只读的，比如挂载的时候是 ro，或者文件系统损坏了，自动变成只读。
4. 权限不够，你没有写入的权限。
5. 磁盘满了，但是有预留空间，你是普通用户，用不了预留的。
6. 磁盘损坏了，无法写入。

磁盘经常空间不足一般是些什么文件引起的？

答案：

1. 日志文件：业务日志、系统日志，没有轮转，越来越大。
2. 临时文件：比如 /tmp 下的临时文件，没有清理。
3. 数据库文件：数据库的数据文件，随着业务增长越来越大。
4. 容器的镜像和日志：Docker 的镜像、容器的日志，没有清理。
5. 备份文件：备份的文件，没有清理旧的备份。

如何保留一个文件为 30 天？

答案：

可以用日志轮转，配置 logrotate，保留 30 天的日志。

或者写定时任务，每天删除 30 天前的文件，比如 `find /path -mtime +30 -delete`，放到 crontab 里，每天执行。

小于 2T 和大于 2T 的磁盘分别用什么工具分区 (fdisk/parted)?

答案：

- 小于 2T 的磁盘，用 fdisk 分区，因为 fdisk 是 MBR 分区表，最大支持 2T。
- 大于 2T 的磁盘，用 parted 分区，用 GPT 分区表，支持大于 2T 的磁盘。

RAID 常用级别以及区别是什么？

答案：

- RAID0：条带，把数据分到多个磁盘，读写速度快，没有冗余，一个磁盘坏了，所有数据都没了，利用率 100%。
- RAID1：镜像，两个磁盘，数据完全一样，冗余好，读快，写慢，利用率 50%，一个磁盘坏了，另一个还能用。
- RAID5：奇偶校验，需要至少 3 个磁盘，数据和奇偶校验分到不同磁盘，一个磁盘坏了，可以通过奇偶校验恢复，利用率是 $(n-1)/n$ ，读写都不错。

- RAID10: RAID1+RAID0, 先做镜像, 再做条带, 需要至少 4 个磁盘, 读写都快, 冗余好, 利用率 50%, 最多可以坏每个镜像组的一个磁盘。

你公司物理服务器磁盘配置是怎样的?

答案:

一般是系统盘用 SSD, 2 块, 做 RAID1, 然后数据盘用 4 块 SATA 盘, 做 RAID5, 或者有的是用 SSD 做数据盘, 做 RAID10, 性能更好。

取消挂载磁盘的命令是什么? 强制取消挂载的命令是什么?

答案:

- 取消挂载: `umount 挂载点`。
 - 强制取消挂载: `umount -f 挂载点`。
-

用户权限

chmod 修改文件权限为 755, 解释一下 755 的含义

答案:

Linux 的权限是三位八进制数, 分别代表所有者、所属组、其他用户的权限, 每个数字是 r (4)、w (2)、x (1) 的和。

755 的含义:

- 所有者: $7=4+2+1$, 有读、写、执行权限。
- 所属组: $5=4+1$, 有读、执行权限, 没有写。
- 其他用户: $5=4+1$, 有读、执行权限, 没有写。
也就是文件的所有者可以修改, 其他人只能读和执行。

修改文件权限的命令是什么?

答案:

`chmod`, 比如 `chmod 755 file`, 或者 `chmod u+x file`, 给所有者加执行权限。

修改属主属组用什么命令?

答案:

`chown`, 比如 `chown user:group file`, 把文件的所有者改成 user, 所属组改成 group。

chown 命令是用来做什么的?

答案:

用来修改文件或目录的所有者和所属组, 比如把文件的所有者从 root 改成普通用户。

如何添加用户, 设置密码?

答案：

- 添加用户： `useradd 用户名` ，然后会创建同名的组，还有家目录。
- 设置密码： `passwd 用户名` ，然后输入密码。

如何查看系统中有多少用户？

答案：

`cat /etc/passwd | wc -l` ，因为 `/etc/passwd` 里每个用户一行，所以统计行数就是用户数。
或者 `getent passwd | wc -l` ，包括 LDAP 的用户。

命令限制是怎么实现的？

答案：

可以用 `sudoers`，配置用户可以执行哪些命令，比如让用户只能执行特定的命令，不能执行其他的。
或者用 `ulimit`，限制用户的资源，比如进程数、文件描述符。
或者用权限，把命令的权限改成只有 `root` 能执行。

进程管理

查找一个正在运行的进程

答案：

`ps aux | grep 进程名`，或者 `pgrep 进程名`，或者 `systemctl status 服务名`。

怎么根据文件名找到进程 ID？

答案：

`lsof 文件名`，可以找到打开这个文件的进程，里面有 PID。
或者 `fuser 文件名`，直接显示进程 ID。

kill -9 和 kill -15 的区别是什么？

答案：

- `kill -15`：是 `SIGTERM` 信号，正常终止进程，给进程一个机会，让它自己清理资源，然后退出，是优雅的关闭。
- `kill -9`：是 `SIGKILL` 信号，强制杀死进程，进程无法捕获这个信号，直接被内核杀死，不会做清理，可能会导致数据丢失，或者文件损坏。
一般优先用 `kill -15`，等进程自己退出，如果不行，再用 `kill -9`。

ps 有什么参数？

答案：

- `ps aux`：查看所有进程，`a` 是所有用户的进程，`u` 是显示用户，`x` 是显示没有终端的进程。

- `ps -ef`：查看所有进程，全格式。
- `ps -l`：长格式，显示更多信息。
- `ps -p PID`：查看指定的进程。

系统负载高怎么排查？

答案：

1. 先用 `uptime` 看负载，然后看是 CPU、内存还是 IO 导致的。
2. `top` 看 CPU 的使用率，%us 是不是很高，如果是的话，看哪个进程占用 CPU 高。
3. 如果 CPU 不高，看内存，`free -h`，看是不是内存不够，用了 swap，`vmstat` 看 si/so，如果有，说明内存不够。
4. 如果内存也没问题，看 IO，`iostat`，看磁盘的 IO 是不是很高，%util 是不是 100%，如果是的话，看哪个进程的 IO 高，用 `iotop`。
5. 然后根据情况处理，比如 CPU 高的话，优化进程，IO 高的话，优化磁盘的读写。

磁盘 IO 过高怎么排查是哪个进程导致的？

答案：

用 `iotop`，这个命令可以查看各个进程的 IO 使用情况，看哪个进程的读写速度高。
或者用 `iostat -x 1`，看磁盘的使用率，然后用 `lsdf` 看哪个进程在读写这个磁盘。

如果 CPU 负载过高怎么处理？

答案：

1. 先用 `top` 找到占用 CPU 高的进程。
2. 如果是业务进程，看是不是有死循环，或者是不是流量太高，需要扩容。
3. 如果是系统进程，看是不是有什么异常的进程，比如挖矿进程。
4. 优化进程，比如优化代码，减少 CPU 的使用，或者调整进程的优先级，用 `nice`。
5. 如果是流量太高，就扩容，加节点，分担负载。

系统负载高排查流程是什么？(uptime → top → 区分 CPU / 内存 / IO → 定位进程)

答案：

就是上面的流程：

1. 先 `uptime` 看负载值，确认负载高。
2. 然后 `top`，看 CPU 的空闲率，如果 %id 很低，说明 CPU 瓶颈。
3. 如果 CPU 没问题，看内存，`free`，看是不是内存不够，用了 swap，`vmstat` 看 swap 的读写。
4. 如果内存没问题，看 IO，`iostat` 看磁盘的 IO 使用率，是不是 100%。

5. 定位到瓶颈之后，再找对应的进程，比如 CPU 高的话，top 里的 % CPU，IO 高的话，iotop。

大量僵尸进程怎么处理？

答案：

僵尸进程是子进程退出了，父进程没有回收它的状态，导致它变成僵尸。

处理方法：

1. 找到父进程，`ps -ef | grep 僵尸进程的PID`，看父进程是谁。
2. 如果父进程还在，给父进程发 SIGCHLD 信号，让它回收僵尸进程，`kill -CHLD 父PID`。
3. 如果父进程不处理，那就重启父进程，这样父进程退出，僵尸进程就会被 init 进程收养，init 会回收它们。
4. 不能直接 kill 僵尸进程，因为它已经退出了。

查看文件被哪个进程所调用 (lsof)

答案：

`lsof 文件名`，就可以看到哪个进程打开了这个文件，里面有 PID、USER 等信息。

系统启动与服务

Linux 的开机启动流程

答案：

1. 开机 BIOS 自检，检查硬件。
2. 读取 MBR，加载 BootLoader，比如 GRUB。
3. BootLoader 加载内核，启动内核。
4. 内核启动 systemd（或者 initd），这是第一个进程，PID 是 1。
5. systemd 读取默认的运行级别，然后启动对应的服务。
6. 启动各个系统服务，比如网络、sshd 等。
7. 启动终端，登录界面。

部署过开机自启服务吗？Service 文件写过吗？

答案：

部署过，写过 Service 文件，比如自定义的服务，写一个.service 文件，放到 /etc/systemd/system/，然后配置 ExecStart、User、WorkingDirectory 等，然后执行 `systemctl daemon-reload`，然后 `systemctl enable 服务名`，就可以开机自启了。

物理机没有正常启动，进入紧急模式，排查思路是什么？

答案：

紧急模式一般是因为挂载失败，比如 `fstab` 里的配置错了，或者磁盘坏了。

排查：

1. 输入 root 密码，进入紧急模式。
2. 看挂载的情况，`mount`，看哪个分区没挂载上。
3. 检查 `/etc/fstab`，看是不是有错误的配置，比如设备名错了，注释掉错误的行。
4. 检查磁盘，`fdisk -l`，看磁盘是不是识别到了，是不是磁盘坏了。
5. 检查文件系统，`fsck`，修复文件系统，如果文件系统损坏了。
6. 修复完之后，重启。

一台服务器异常关闭，开机后异常现象 (报错) 怎么查看？用什么命令？

答案：

- `dmesg`：查看内核的日志，看关机的时候有没有硬件错误。
- `journalctl -xb`：查看 `systemd` 的日志，看启动的时候的报错。
- `/var/log/messages`：系统的日志，查看有没有报错。
- `/var/log/boot.log`：启动的日志，看启动的时候的错误。

开机自动执行命令的文件有哪些 (/etc/rc.local、/etc/profile、systemctl enable)？

答案：

1. `/etc/rc.local`：开机最后执行的脚本，把命令写到这里，开机就会执行。
2. `/etc/profile`：用户登录的时候执行，不是开机，是登录的时候。
3. `systemctl enable`：把服务设置为开机自启，`systemd` 的方式。
4. 还有 `crontab` 的 `@reboot`，定时任务，开机的时候执行。
5. `/etc/profile.d/`：里面的脚本，用户登录的时候执行。
6. `/etc/bashrc`：bash 启动的时候执行。

系统优化

系统优化做过什么？

答案：

做过系统的内核参数优化，比如 TCP 的参数，调整连接队列，`time_wait` 的复用，还有文件描述符的调整，加大最大文件描述符，还有磁盘的挂载优化，用 `noatime`，还有 swap 的优化，调整 `swappiness`，还有 SSH 的优化，禁止 root 登录，修改默认端口，还有服务的优化，关闭不用的服务，减少资源占用。

新系统部署服务需要做什么优化？

答案：

1. 关闭不用的服务，减少资源占用。
2. 调整文件描述符，加大最大文件描述符，因为服务需要打开很多文件。
3. 内核参数优化，比如 TCP 的参数，优化网络性能。
4. 磁盘挂载优化，用 noatime，提升磁盘性能。
5. 调整 swap 的参数，减少 swap 的使用，提升性能。
6. SSH 优化，禁止 root 登录，修改默认端口，提升安全。
7. 配置时间同步，保证时间准确。
8. 配置监控，监控系统的指标。

一个新系统用过需要干什么优化？

和上面的一样，新系统部署完之后，做这些优化。

SSH 优化、禁止 root 登录如何实现？

答案：

编辑 /etc/ssh/sshd_config：

- `PermitRootLogin no`：禁止 root 登录。
- `PasswordAuthentication no`：如果用密钥的话，关闭密码登录。
- 修改默认端口，`Port 2222`，改成其他端口，减少暴力破解。
- 限制登录的用户，`AllowUsers user`，只允许指定的用户登录。
然后重启 sshd 服务，`systemctl restart sshd`。

加大文件描述符怎么配置？

答案：

编辑 /etc/security/limits.conf，添加：

代码块

```
1 * soft nfile 655360
2 * hard nfile 655360
3 root soft nfile 655360
4 root hard nfile 655360
```

然后编辑 /etc/sysctl.conf，添加 `fs.file-max = 655360`，然后执行 `sysctl -p` 生效。
然后重启服务，或者重新登录，就生效了。

内核参数优化一般优化哪些？

答案：

主要优化网络相关的，比如 TCP 的参数：

- `net.core.somaxconn`：TCP 连接队列的大小。
- `net.ipv4.tcp_fin_timeout`：TIME_WAIT 的超时时间。
- `net.ipv4.tcp_tw_reuse`：复用 TIME_WAIT 的连接。
- `net.ipv4.tcp_rmem`、`net.ipv4.tcp_wmem`：TCP 的读写缓冲区。

还有内存相关的：

- `vm.swappiness`：减少 swap 的使用。
- `vm.dirty_ratio`、`vm.dirty_background_ratio`：脏页的比例。

还有文件系统的：

- `fs.file-max`：最大文件描述符。

TCP 连接队列大小怎么调整？

答案：

调整 `net.core.somaxconn`，这个参数是 TCP 的半连接队列的大小，默认是 128，高并发的话，改成 1024 或者更大，比如 32768，编辑 `/etc/sysctl.conf`，添加 `net.core.somaxconn = 32768`，然后 `sysctl -p` 生效。

还有 `net.ipv4.tcp_max_syn_backlog`，这个是半连接队列的最大长度，也可以调大。

TCP 快速打开怎么启用？

答案：

TCP Fast Open，启用的话，编辑 `/etc/sysctl.conf`，添加 `net.ipv4.tcp_fastopen = 3`，然后 `sysctl -p` 生效，3 是同时支持客户端和服务端。

swap 分区的作用是什么？用或不用的好处坏处是什么？

答案：

swap 是交换分区，当内存不够的时候，把内存里的不常用的数据写到 swap 里，腾出内存。

- **用 swap 的好处**：当内存不够的时候，可以临时用 swap，避免 OOM 杀死进程，比如内存突然暴涨的时候，有个缓冲。
- **用 swap 的坏处**：swap 是磁盘的，速度比内存慢很多，用了 swap 之后，系统会变卡，性能下降。
- **不用 swap 的好处**：系统不会用到慢的磁盘，性能更好，内存够的话，没问题。
- **不用 swap 的坏处**：如果内存突然不够了，就会直接 OOM，杀死进程。

部署 K8S 时为什么要关闭 swap 分区？

答案：

因为 K8s 的调度是基于内存的，如果开了 swap，节点的内存使用情况就不准了，kubelet 会认为内存还有很多，其实已经用了 swap，导致调度错误，而且 swap 的速度慢，会影响 Pod 的性能，所以 K8s 要求关闭 swap，保证性能和调度的准确。

系统优化一般是在什么场景下优化？

答案：

一般是在新系统部署的时候，先做优化，还有就是系统性能不够的时候，比如高并发的场景，网络延迟高，或者连接数不够的时候，做优化，还有就是业务增长，原来的参数不够用了，需要调大参数。

vm.swappiness 参数的作用是什么？

答案：

这个参数是控制内核使用 swap 的倾向，值是 0-100，越小，内核越不倾向于用 swap，越大，越倾向于用 swap。

默认是 60，一般我们会改成 10，或者更小，让内核尽量用物理内存，减少 swap 的使用，提升性能。

文件描述符限制怎么调整？

就是之前的那个，limits.conf 和 sysctl.conf 的配置。

SSD 使用 noatime、nodiratime 挂载选项有什么好处？

答案：

默认情况下，每次读取文件的时候，都会更新文件的访问时间（atime），这会有很多的写操作，SSD 的话，虽然快，但是也会有额外的 IO，用 noatime 的话，就不更新访问时间了，nodiratime 是不更新目录的访问时间，这样可以减少磁盘的写操作，提升 SSD 的性能，同时减少 SSD 的磨损，延长寿命。

net.core.somaxconn 的作用是什么？

答案：

这个参数是 TCP 的监听队列的最大长度，也就是已经完成三次握手，还没被应用层 accept 的连接的队列大小，默认是 128，高并发的时候，这个队列太小的话，会导致新的连接被丢弃，所以调大这个参数，可以支持更多的并发连接。

net.ipv4.tcp_fin_timeout 的作用是什么？

答案：

这个参数是 TCP 连接在 TIME_WAIT 状态的超时时间，默认是 60 秒，也就是连接关闭之后，要等 60 秒才会释放，调小这个参数，可以更快的释放 TIME_WAIT 的连接，释放资源，避免太多的 TIME_WAIT 占用端口。

net.ipv4.tcp_tw_reuse 的作用是什么？

答案：

这个参数是开启 TIME_WAIT 连接的复用，也就是当新的连接要建立的时候，可以复用已经处于 TIME_WAIT 状态的连接，这样可以减少 TIME_WAIT 的数量，提升端口的利用率，高并发的时候很有用。

vm.overcommit_memory 参数的作用是什么？

答案：

这个参数是控制内存的超额分配，0 是检查，有没有足够的内存，1 是允许超额分配，2 是不允许，总分配不能超过 swap + 物理内存的比例。

比如 Redis，需要设置这个为 1，因为 Redis 会用很多的内存，超额分配，不然可能无法启动。

vm.dirty_ratio 和 vm.dirty_background_ratio 的作用是什么？

答案：

这两个参数是控制脏页的比例，脏页就是内存里修改了，还没写到磁盘的页。

- `vm.dirty_background_ratio`：当脏页的比例达到这个值的时候，内核会后台线程把脏页写到磁盘，默认是 10。
 - `vm.dirty_ratio`：当脏页的比例达到这个值的时候，所有的写操作都会阻塞，直接把脏页写到磁盘，默认是 20。
- 调大这两个值，可以提升写的性能，但是如果掉电的话，可能会丢数据，调小的话，更安全，但是性能差点。

vm.max_map_count 参数的作用是什么？

答案：

这个参数是最大的内存映射区域的数量，默认是 65530，比如 Elasticsearch，需要很多的内存映射，所以需要调大这个参数，比如改成 262144，不然 ES 启动会报错。

worker_rlimit_nofile 的作用是什么？

答案：

这个是 Nginx 的参数，设置 Nginx 进程的最大文件描述符，因为 Nginx 需要打开很多的连接，所以要把这个调大，和系统的 nofile 对应，比如 `worker_rlimit_nofile 655360`；，这样 Nginx 就可以打开更多的文件和连接。

时间同步写入定时任务怎么配置？

答案：

可以写一个定时任务，比如每天同步一次时间，编辑 crontab，添加 `0 0 * * *`

`/usr/sbin/ntpdate ntp.aliyun.com >> /var/log/ntpdate.log 2>&1`，这样每天凌晨同步时间，或者用 chrony，自动同步，不用定时任务。

SSH 连接不上 Linux 怎么排查？

答案：

1. 先看网络通不通，ping 服务器的 IP，能不能通。
2. 看端口通不通，telnet IP 22，看端口能不能通，是不是防火墙拦截了。
3. 检查 sshd 服务是不是运行了，`systemctl status sshd`，是不是启动了。
4. 检查 sshd 的配置，是不是禁止了 root 登录，是不是改了端口。
5. 检查用户的权限，是不是用户被锁了。
6. 看日志，/var/log/secure，看 ssh 的登录日志，有没有报错。

SSH 配置文件中 PermitRootLogin 和 PasswordAuthentication 的作用是什么？

答案：

- `PermitRootLogin`：是否允许 root 用户登录，设置为 no 的话，就禁止 root 登录，提升安
- `PasswordAuthentication`：是否允许密码登录，设置为 no 的话，就只能用密钥登录，避免密码暴力破解，提升安全。

麒麟系统忘记密码怎么办？

答案：

和 CentOS 类似，重启系统，进入 GRUB，编辑启动项，加 `init=/bin/bash`，然后进入单用户模式，然后用 `passwd` 命令修改密码，然后重启。

CentOS、Ubuntu、Kylin 系统的不同点是什么？

答案：

- CentOS：是 RedHat 的社区版，稳定，适合服务器，用 yum 包管理器。
- Ubuntu：是 Debian 的衍生版，更新快，桌面和服务器都用，用 apt 包管理器，对新硬件支持好。
- Kylin：是国产的操作系统，基于 Linux，自主可控，适配国产硬件和软件，有桌面和服务器版，符合信创的要求。

Ubuntu 的包管理器是什么？apt 更新到最新版本怎么操作？

答案：

Ubuntu 的包管理器是 apt，更新的话：

1. `apt update`：更新软件源的缓存。
2. `apt upgrade`：升级所有的包到最新版本。

如果对端服务器设置了禁 ping, 如何测试连通性？

答案：

可以用 telnet 测试端口，比如 `telnet IP 端口`，看端口通不通，或者用 nc，`nc -zv IP 端口`，或者用 tcping，测试 TCP 的连通性，或者用 traceroute，看能不能到对端。

怎么判断服务器是 x86 还是 ARM 架构？

答案：

`uname -m`，如果是 x86 的话，会显示 x86_64，如果是 ARM 的话，会显示 aarch64 或者 armv7。

软硬连接

软硬链接的区别是什么？

答案：

- **硬链接**：是文件的多个入口，多个文件名指向同一个 inode，删除一个硬链接，不会影响其他的，只有所有的硬链接都删除了，文件才会被删除，不能跨文件系统，不能给目录做硬链接。
 - **软链接**：是一个快捷方式，是一个新的文件，里面存的是原文件的路径，删除原文件的话，软链接就失效了，变成死链接，可以跨文件系统，可以给目录做软链接。
-

定时任务

定时任务每分钟执行一次怎么配置？

答案：

crontab 里配置 `* * * * *` 命令，五个星，分别是分、时、日、月、周，所以每分钟就是所有的都星。

crontab 的语法格式是什么？

答案：

crontab 的格式是：`分 时 日 月 周 命令`

- 分：0-59
- 时：0-23
- 日：1-31
- 月：1-12
- 周：0-7（0 和 7 都是周日）
可以用代表所有，/ 代表每隔，比如/5 就是每 5 分钟，- 代表范围，比如 1-5，, 代表枚举，比如 1,3,5。

每天晚上 12 点执行一次定时任务怎么配置？

答案：

`0 0 * * *` 命令，也就是分 0，时 0，日、月、周都是所有，每天的 0 点 0 分执行。

硬件与 RAID

问硬件配置？你接触过物理机吗？

答案：

接触过，物理机的配置一般是 2 路 CPU，每个 CPU16 核，内存 128G 或者 256G，磁盘的话，系统盘 2 块 1T SSD，数据盘 4 块 4T SATA，或者 SSD。

用过交换机吗？用过啥型号的？

答案：

用过，比如华为的 S5735，还有 H3C 的，主要是配置 VLAN、端口聚合这些。

服务器配置是怎样的？

就是上面的，CPU、内存、磁盘。

你接触过嵌入式的板卡吗？

答案：

接触过一点，比如一些硬件的管理板卡，比如 IPMI 的板卡，用来管理服务器的。

你会组网吗？只有服务器你会怎么组网？

答案：

如果只有服务器的话，用交换机把服务器连起来，然后配置 VLAN，把不同的业务分开，然后配置网关，让服务器可以上网，然后做 bond，把多个网卡绑定，提升带宽和冗余。

磁盘空间不够时如何解决 (新磁盘 + 软链接方案)？

答案：

比如某个目录满了，比如 /var/lib/docker，然后加了新的磁盘，挂载到 /data/docker，然后把原来的 /var/lib/docker 的文件复制过去，然后做软链接，`ln -s /data/docker /var/lib/docker`，这样就可以了，不用改配置，就把数据移到新磁盘了。

网络相关

TCP/IP

TCP 三次握手四次挥手详细说一下

答案：

三次握手（建立连接）：

1. 第一次：客户端向服务端发送 SYN 报文，seq=x，客户端进入 SYN_SENT 状态，请求建立连接。

- 第二次：服务端收到 SYN，回复 SYN+ACK 报文，seq=y，ack=x+1，服务端进入 SYN_RCVD 状态，同意连接，同时也发送自己的 SYN。
- 第三次：客户端收到 SYN+ACK，回复 ACK 报文，seq=x+1，ack=y+1，客户端和服务端都进入 ESTABLISHED 状态，连接建立完成。

四次挥手（关闭连接）：

因为 TCP 是全双工的，所以关闭需要双方各自关闭。

- 第一次：客户端发送 FIN 报文，seq=u，客户端进入 FIN_WAIT_1 状态，告诉服务端，我没有数据要发了。
- 第二次：服务端收到 FIN，回复 ACK 报文，ack=u+1，服务端进入 CLOSE_WAIT 状态，客户端收到 ACK，进入 FIN_WAIT_2 状态，这时候服务端还可以给客户端发数据。
- 第三次：服务端发完数据了，发送 FIN 报文，seq=v，服务端进入 LAST_ACK 状态，告诉客户端，我也没数据要发了。
- 第四次：客户端收到 FIN，回复 ACK 报文，ack=v+1，客户端进入 TIME_WAIT 状态，等 2MSL，确保服务端收到 ACK，服务端收到 ACK，进入 CLOSED 状态，客户端等时间到了，也进入 CLOSED 状态，连接关闭。

为什么要三次握手？

答案：

- 防止失效的连接请求：**如果是两次握手的话，客户端之前的失效的 SYN 请求，迟到了到了服务端，服务端就会建立连接，然后等客户端发数据，客户端其实没有这个连接，就会浪费服务端的资源，三次握手的话，客户端会回复 ACK，服务端收到才会建立连接，失效的请求就会被拒绝。
- 同步双方的序列号：**三次握手可以让双方都确认对方的发送和接收能力，同步初始序列号，确保双方都能收发数据。

完成通信之后建立什么状态？

答案：

完成通信之后，关闭连接，客户端会进入 TIME_WAIT 状态，等待 2MSL，然后才会关闭，服务端会直接进入 CLOSED。

TCP 连接数过多类似的问题，IP 通端口不通你怎么解决？

答案：

IP 通端口不通的排查：

- 检查服务是不是监听了这个端口，`ss -lntp`，看有没有监听这个端口，是不是监听的 [0.0.0.0] (0.0.0.0)，还是 127.0.0.1。
- 检查防火墙，服务器的防火墙是不是放开了这个端口，`iptables -L`，或者 `firewall-cmd --list-port`。
- 检查安全组，云服务器器的话，安全组是不是放开了端口。

4. 检查是不是有其他的设备，比如中间的防火墙，拦截了端口。

5. 测试端口，`telnet IP 端口`，`nc -zv IP 端口`。

如果服务器长期处于 `established` 状态是什么原因？

答案：

说明连接一直是建立的，没有关闭，可能是：

1. 长连接，比如 HTTP 的长连接，或者数据库的连接池，连接一直保持，复用。
2. 客户端没有正常关闭连接，比如客户端突然断网了，没有发 FIN，导致连接一直保持。
3. 服务端的连接超时设置的太长，没有回收空闲的连接。

高并发下 TCP 连接延迟升高，如何从操作系统层面进行调优？

答案：

1. 调大连接队列：`net.core.somaxconn`，`net.ipv4.tcp_max_syn_backlog`，支持更多的并发连接。
2. 复用 TIME_WAIT 连接：`net.ipv4.tcp_tw_reuse=1`，复用 TIME_WAIT 的端口。
3. 调小 TIME_WAIT 的超时：`net.ipv4.tcp_fin_timeout=30`，更快的释放连接。
4. 调大 TCP 的缓冲区：`net.ipv4.tcp_rmem`、`net.ipv4.tcp_wmem`，提升网络的读写性能。
5. 开启 TCP 快速打开：`net.ipv4.tcp_fastopen=3`，减少握手的延迟。
6. 调大端口范围：`net.ipv4.ip_local_port_range`，扩大可用的端口范围，支持更多的连接。

我想查看 TCP 的各种连接状态并统计，怎么统计？

答案：

`ss -s`：可以直接统计各个状态的连接数。

或者 `netstat -an | awk '/tcp/ {print $6}' | sort | uniq -c`，统计每个状态的数量。

常见的网络协议有哪些？

答案：

- 应用层：HTTP、HTTPS、FTP、SMTP、DNS、SSH。
- 传输层：TCP、UDP。
- 网络层：IP、ICMP、ARP、BGP。
- 数据链路层：PPP、以太网。

DNS

DNS 解析流程说一下

答案：

DNS 递归解析的流程：

1. 浏览器先查自己的缓存，有没有这个域名的 IP。
2. 然后查操作系统的缓存，比如 Windows 的 DNS 缓存，Linux 的 /etc/hosts。
3. 然后向本地 DNS 服务器（比如运营商的 DNS，或者 8.8.8.8）发送请求。
4. 本地 DNS 查自己的缓存，如果有，就返回。
5. 如果没有，本地 DNS 就向根域名服务器发送请求，根服务器告诉它顶级域名服务器的地址。
6. 然后本地 DNS 向顶级域名服务器发送请求，顶级服务器告诉它权威域名服务器的地址。
7. 然后本地 DNS 向权威域名服务器发送请求，权威服务器返回域名对应的 IP。
8. 本地 DNS 把结果返回给客户端，同时缓存下来。

DNS 解析失败排查思路是什么？

答案：

1. 先 ping 域名，看能不能解析，是不是失败。
2. 用 nslookup 或者 dig，测试 DNS 解析，看是不是 DNS 服务器的问题。
3. 检查 /etc/resolv.conf，看 DNS 服务器的配置是不是正确，能不能 ping 通 DNS 服务器。
4. 检查是不是域名本身错了，或者域名过期了。
5. 检查是不是防火墙拦截了 53 端口，DNS 的端口是 53，UDP 和 TCP。
6. 换个 DNS 服务器试试，比如 8.8.8.8，看能不能解析。

ping 域名、指定 yum 仓库、FTP 下载解析失败排查思路，如果 nslookup 解析失败怎么办？

答案：

nslookup 解析失败的话，说明 DNS 解析有问题：

1. 检查 DNS 服务器的配置，/etc/resolv.conf，是不是配置错了。
2. 测试能不能 ping 通 DNS 服务器，网络通不通。
3. 检查防火墙是不是拦截了 53 端口，导致无法访问 DNS。
4. 换个 DNS 服务器，比如 8.8.8.8，用 nslookup 域名 8.8.8.8，看能不能解析，如果能，说明原来的 DNS 有问题。
5. 检查域名是不是正确，是不是过期了，或者被墙了。

DNS 在 Linux 里面的解析域名失败排查思路是什么？

和上面的一样。

通过 IP 可以访问，但通过域名访问不到，怎么排查？

答案：

说明 IP 是通的，域名解析有问题：

1. 用 nslookup 或者 dig，看域名解析到的 IP 是不是正确的，是不是解析错了。
2. 检查 /etc/hosts，是不是有错误的域名解析。
3. 检查 DNS 服务器，是不是解析错了，或者 DNS 缓存有问题。
4. 检查端口，是不是域名访问的端口和 IP 的不一样，比如域名加了端口。
5. 检查是不是有反向代理，域名的话，反向代理配置错了。

HTTP

浏览器输入 URL 到页面加载完成，涉及到的网络协议有哪些？

答案：

1. DNS：解析域名到 IP。
2. TCP：建立连接，三次握手。
3. HTTP：发送请求，获取页面内容。
4. HTTPS 的话，还有 TLS：加密握手。
5. IP：网络层的传输。
6. 以太网：数据链路层的传输。

网站访问流程是什么？

答案：

1. 浏览器输入 URL，解析域名，DNS 解析到 IP。
2. 浏览器和服务端建立 TCP 连接，三次握手。
3. 如果是 HTTPS，进行 TLS 握手，建立加密连接。
4. 浏览器发送 HTTP 请求，请求页面。
5. 服务器处理请求，返回 HTTP 响应，把页面的内容返回。
6. 浏览器解析响应，渲染页面，同时加载页面里的资源，比如 CSS、JS、图片，每个资源都要做类似的请求。
7. 加载完之后，页面就显示出来了。
8. 传输完成，TCP 四次挥手，关闭连接。

常见的 HTTP 状态码有哪些？

答案：

- 1xx: 信息, 比如 100 Continue。
- 2xx: 成功, 200 OK, 201 Created, 204 No Content。
- 3xx: 重定向, 301 永久重定向, 302 临时重定向, 304 缓存。
- 4xx: 客户端错误, 400 请求错误, 401 未授权, 403 禁止, 404 找不到, 405 方法不允许。
- 5xx: 服务端错误, 500 服务器错误, 502 网关错误, 503 服务不可用, 504 网关超时。

HTTP 响应码 200、400、401、403、404、500、502、503、504 分别代表什么?

答案:

- 200: 请求成功, 服务器正常返回了数据。
- 400: 请求参数错误, 客户端的请求有问题, 服务器无法理解。
- 401: 未授权, 需要登录才能访问。
- 403: 禁止访问, 服务器拒绝了请求, 即使你登录了也不行。
- 404: 找不到资源, 请求的 URL 不存在。
- 500: 服务器内部错误, 服务器处理请求的时候出错了。
- 502: 网关错误, 代理服务器从后端服务器收到了无效的响应。
- 503: 服务不可用, 服务器暂时无法处理请求, 可能是过载了, 或者维护。
- 504: 网关超时, 代理服务器等待后端服务器的响应超时了。

HTTP 请求和响应的基本流程是什么?

答案:

1. 客户端建立和服务器的连接。
2. 客户端发送请求行、请求头、请求体, 比如 `GET /index.html HTTP/1.1`, 然后是头, 然后是体。
3. 服务器收到请求, 处理请求, 然后发送响应行、响应头、响应体, 比如 `HTTP/1.1 200 OK`, 然后头, 然后是页面的内容。
4. 客户端收到响应, 处理内容。
5. 连接关闭。

OSI 七层模型与端口

说一下 OSI 七层模型

答案:

OSI 七层模型从下到上:

1. **物理层**: 负责物理介质的传输, 比如电缆、光纤, 传输比特流。

2. **数据链路层**：负责相邻节点之间的传输，把比特封装成帧，比如以太网，ARP 协议。
3. **网络层**：负责不同网络之间的路由，把帧封装成包，比如 IP 协议，路由器工作在这层。
4. **传输层**：负责端到端的传输，处理流量控制、差错控制，比如 TCP、UDP，端口在这层。
5. **会话层**：负责建立、管理、终止会话，比如会话的建立和断开。
6. **表示层**：负责数据的格式转换，加密、解密，比如 TLS 的加密。
7. **应用层**：负责应用程序的接口，比如 HTTP、FTP、DNS，用户交互在这层。

七层模型有哪些？四层的协议说两个

答案：

七层就是上面的，四层是传输层，协议有 TCP、UDP。

路由器和交换机工作在几层？

答案：

- 路由器工作在网络层（第三层），根据 IP 地址转发。
- 交换机工作在数据链路层（第二层），根据 MAC 地址转发，三层交换机的话，也可以工作在网络层。

四层负载和七层负载之间的区别是什么？

答案：

- **四层负载**：工作在传输层，根据 IP 和端口来转发，比如 LVS，转发速度快，但是不关心应用层的内容，只能做简单的负载。
- **七层负载**：工作在应用层，根据 HTTP 的内容，比如 URL、Header、Cookie 来转发，比如 Nginx，功能更强大，可以做更精细的负载，比如根据路径转发，但是速度比四层慢，因为要解析应用层的内容。

SSH 默认端口、HTTP 默认端口分别是什么？

答案：

- SSH 默认端口是 22。
- HTTP 默认端口是 80。
- HTTPS 默认端口是 443。

常见服务端口有哪些？(FTP 21、SSH 22、Telnet 23、SMTP 25、HTTP 80、HTTPS 443、DHCP 67/68 等)

答案：

- FTP：21（控制），20（数据）
- SSH：22

- Telnet: 23
- SMTP: 25
- HTTP: 80
- HTTPS: 443
- DHCP: 67 (服务端)、68 (客户端)
- MySQL: 3306
- Redis: 6379
- Nginx: 80、443
- Tomcat: 8080
- DNS: 53

系统最大端口号是多少？

答案：

TCP 和 UDP 的端口是 16 位的，所以最大是 65535，也就是 $2^{16}-1$ 。

网段范围 10.10.0.0/19 换算成子网掩码是多少？

答案：

/19 就是前 19 位是网络位，所以子网掩码是 255.255.224.0，因为前两个字节是 16 位，第三个字节的前 3 位是网络位，224 是 11100000。

BGP 协议是几层？

答案：

BGP 是应用层协议，工作在 OSI 的第七层，它是基于 TCP 的，端口 179。

防火墙

iptables 怎么去屏蔽特定网段？

答案：

```
iptables -I INPUT -s 192.168.1.0/24 -j DROP
```

，这样就屏蔽了 [192.168.1.0/24] (192.168.1.0/24) 这个网段的入站请求。

iptables 添加规则的命令是什么？

答案：

```
iptables -A 链 -s 源IP -d 目标IP -p 协议 --dport 端口 -j 动作
```

，比如

```
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

。

iptables 如何限制某个 IP 访问服务器某个端口？说出详细命令

答案：

`iptables -I INPUT -s 192.168.1.100 -p tcp --dport 80 -j DROP`，这样就限制了 192.168.1.100 这个 IP 访问 80 端口。

或者如果是允许的话，就是 `-j ACCEPT`。

如何查看防火墙开放了哪些端口？

答案：

`iptables -L -n`，查看所有的规则，看开放了哪些端口。

或者 `firewall-cmd --list-ports`，firewalld 的话。

对本机放行 8080 端口允许 172.1.1.6 网段来访问，怎么编写 iptables 规则？

答案：

`iptables -I INPUT -s 172.1.1.0/24 -p tcp --dport 8080 -j ACCEPT`，这样就允许 172.1.1.0/24 的网段访问 8080 端口。

对于 Ubuntu 系统除了 ufw 防火墙还用过哪些防火墙？

答案：

还用过 iptables，还有 firewalld，还有云服务器的安全组。

有没有用 iptables 去配置流量转发？

答案：

用过，比如做 NAT，把公网的端口转发到内网的服务器，比如 `iptables -t nat -A PREROUTING -d 公网IP -p tcp --dport 80 -j DNAT --to 内网IP:80`，然后配置 SNAT，`iptables -t nat -A POSTROUTING -s 内网网段 -j SNAT --to 公网IP`。

什么时候开启防火墙？什么时候关闭防火墙？

答案：

生产环境一般都开启防火墙，做访问控制，只开放需要的端口，提升安全，避免被攻击。

测试环境的话，如果是内部的，有时候会关闭，方便测试，或者内部的网络，没有外部的攻击，但是最好还是开启。

网络排查

在现场环境网络方面有遇到什么问题吗？

答案：

遇到过，比如网络不通，端口不通，还有网络延迟高，丢包，还有 K8s 的 Pod 之间网络不通，还有跨可用区的网络延迟高。

IP 通端口不通怎么解决？

和之前的一样，检查服务、防火墙、安全组。

局域网中服务器和服务器网络不通怎么排查？

答案：

1. 先 ping，看能不能通，有没有丢包。
2. 检查是不是在同一个网段，是不是网关对。
3. 检查防火墙，是不是服务器的防火墙拦截了。
4. 检查交换机的配置，是不是 VLAN 隔离了，是不是端口错了。
5. 检查 ARP，是不是 MAC 地址冲突了。
6. 用 traceroute，看数据包到哪里断了。

mtr 和 Traceroute 用过吗？

答案：

用过，都是路由跟踪的工具，traceroute 是跟踪数据包的路径，看经过哪些节点，mtr 是结合了 ping 和 traceroute，不仅看路径，还看每个节点的丢包和延迟，更适合排查网络的问题。

都用过哪些抓包工具？

答案：

用过 tcpdump，还有 wireshark，tcpdump 是命令行的，wireshark 是图形化的，更方便分析。

tcpdump 具体命令是啥？如何指定网卡？用什么参数？

答案：

tcpdump 的基本命令：`tcpdump -i 网卡名 过滤条件`，比如 `tcpdump -i eth0 tcp port 80`，抓 eth0 网卡的 80 端口的 TCP 包。

参数：

- -i：指定网卡。
- -n：不解析域名，显示 IP。
- -c：抓多少个包就停止。
- -w：把包写到文件里，之后用 wireshark 分析。
- 过滤条件：比如 `host [1.1.1.1](1.1.1.1)`，`port 80`，`tcp`，`udp`。

有配置过交换机或者路由器吗？

答案：

配置过，比如配置 VLAN，配置端口聚合，配置静态路由，配置端口的速率。

上不了网排查思路是什么？

答案：

1. 先 ping 网关，看能不能通，能不能到网关。

2. 然后 ping 公网 IP，比如 8.8.8.8，看能不能通，如果能，说明是 DNS 的问题。
3. 如果不能，检查网络配置，IP、子网掩码、网关是不是正确。
4. 检查路由器，是不是路由器断网了。
5. 检查防火墙，是不是拦截了。
6. 检查网卡，是不是网卡坏了，或者驱动有问题。

如何实现不同网段的通信？

答案：

通过路由器，或者三层交换机，配置路由，让不同网段的数据包可以转发，比如配置静态路由，或者动态路由，让两个网段之间可以互相访问。

网络配置

Ubuntu 怎么添加静态 IP？

答案：

编辑 /etc/netplan/00-installer-config.yaml，然后配置：

代码块

```
1  network:
2    ethernets:
3      eth0:
4        addresses: [192.168.1.100/24]
5        gateway4: 192.168.1.1
6        nameservers:
7          addresses: [8.8.8.8, 114.114.114.114]
8  version: 2
```

然后执行 `netplan apply` 生效。

Ubuntu 网络服务叫什么名字？

答案：

新版的 Ubuntu 用 netplan，旧版的用 networking，或者 NetworkManager。

给主机加路由用什么命令？

答案：

`route add -net 192.168.2.0/24 gw 192.168.1.1`，或者 `ip route add 192.168.2.0/24 via 192.168.1.1`。

Nginx/Web 服务

基础配置

Nginx 反向代理如何配置？

答案：

配置反向代理的话，在 server 块里配置 proxy_pass，比如：

代码块

```
1  server {
2      listen 80;
3      server_name example.com;
4      location / {
5          proxy_pass http://127.0.0.1:8080;
6          proxy_set_header Host $host;
7          proxy_set_header X-Real-IP $remote_addr;
8          proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
9      }
10 }
```

这样就把请求转发到本地的 8080 端口了。

Nginx 做过哪些优化？

答案：

1. 调整 worker 进程的数量，设置为 CPU 的核心数，`worker_processes auto;`。
2. 开启 epoll，`use epoll;`，提升 IO 性能。
3. 调整 worker 连接数，`worker_connections 655360;`，支持更多的连接。
4. 开启 gzip 压缩，减少传输的大小。
5. 开启缓存，静态资源的缓存，减少请求。
6. 调整超时参数，比如 `keepalive_timeout`，复用连接。
7. 调整文件描述符，`worker_rlimit_nofile 655360;`。

Nginx 的配置和反向代理怎么写？

就是上面的反向代理的配置。

Nginx 部署后只能公司内部访问如何实现？

答案：

在 Nginx 的配置里，限制 IP，只允许公司的网段访问，比如：

```
代码块 location / {  
2     allow 192.168.0.0/16;  
3     allow 10.0.0.0/8;  
4     deny all;  
5     proxy_pass http://127.0.0.1:8080;  
6 }
```

这样就只允许公司的内网 IP 访问，其他的都拒绝。

Nginx 除了做负载均衡还有哪些地方用到？

答案：

- 反向代理：把请求转发到后端的服务。
- 静态资源服务器：直接提供静态文件的访问。
- 缓存：缓存后端的响应，减少后端的压力。
- 限流：限制请求的频率，防止攻击。
- 重定向：URL 的重写和重定向。
- HTTPS：配置 SSL，做 HTTPS 的卸载。
- 身份验证：做基础的身份验证。

Nginx 都有哪些模块？常用的模块有哪些？

答案：

常用的模块：

- ngx_http_core_module：核心模块，处理 HTTP 的核心功能。
- ngx_http_proxy_module：反向代理模块。
- ngx_http_upstream_module：负载均衡模块。
- ngx_http_gzip_module：gzip 压缩模块。
- ngx_http_rewrite_module：URL 重写模块。
- ngx_http_cache_module：缓存模块。
- ngx_http_limit_conn_module：连接限流模块。
- ngx_http_limit_req_module：请求限流模块。
- ngx_http_ssl_module：SSL 模块，HTTPS。

Nginx 工作原理是什么？为什么消耗资源少并发能力强？

答案：

Nginx 是事件驱动的，用的是 epoll 的 IO 多路复用模型，一个 worker 进程可以处理多个连接，不用为每个连接开一个进程，所以资源消耗少，而且异步非阻塞，处理请求的时候，不会因为 IO 阻塞，所以可以处理很多的并发连接，而 Apache 是每个连接一个进程，所以并发高了之后，资源消耗大，并发能力不如 Nginx。

Nginx 工作在几层？

答案：

Nginx 可以工作在四层和七层，四层的话，就是 TCP 的负载均衡，七层的话，就是 HTTP 的反向代理和负载均衡。

Nginx 有哪些负载均衡模式？

答案：

1. **轮询**：默认的，请求按顺序轮流分到后端节点。
2. **加权轮询**：给节点加权重，权重越高，分到的请求越多。
3. **ip_hash**：根据客户端的 IP 的 hash，分配到固定的节点，解决 session 的问题。
4. **least_conn**：把请求分到连接数最少的节点，哪个节点处理的连接少，就分给谁。
5. **url_hash**：根据 URL 的 hash 分配，适合缓存的场景。
6. **fair**：根据后端的响应时间，响应快的节点，分到更多的请求。

Nginx 日志格式知道吗？

答案：

默认的日志格式是：

代码块

```
1 log_format main '$remote_addr - $remote_user [$time_local] "$request" '
2                 '$status $body_bytes_sent "$http_referer" '
3                 '"$http_user_agent" "$http_x_forwarded_for"';
```

里面包含了客户端 IP、时间、请求、状态、字节、referer、UA、转发的 IP。

如果 Nginx 接收到请求，但不想将客户端的源 IP 向后端服务器传递，应该如何处理？

答案：

就是不设置 X-Real-IP 和 X-Forwarded-For 这些头，默认的话，proxy_pass 不会自动传递这些头，所以如果不配置的话，后端服务器看到的就是 Nginx 的 IP，而不是客户端的 IP，所以只要不添加那些 proxy_set_header 的配置就可以了。

Nginx 的 server_name 配置有什么作用？一般会配什么内容？

答案：

server_name 是用来匹配请求的域名的，Nginx 根据请求的 Host 头，匹配 server_name，找到对应的 server 块，处理请求，所以可以在同一个端口，配置多个域名，不同的域名对应不同的配置。一般配域名，比如 `server_name example.com www.example.com;`。

Nginx 有没有用缓存模块？

答案：

用过，用 proxy_cache，配置后端响应的缓存，比如：

代码块

```
1 proxy_cache_path /tmp/cache levels=1:2 keys_zone=cache:10m inactive=60m;
2 server {
3     location / {
4         proxy_cache cache;
5         proxy_cache_valid 200 10m;
6         proxy_pass http://backend;
7     }
8 }
```

这样就把后端的响应缓存起来，相同的请求，直接返回缓存，不用请求后端，减少后端的压力。

Nginx 404 为啥？如何解决？

答案：

404 是找不到资源，原因：

1. 请求的 URL 不存在，后端没有这个资源。
2. location 匹配错了，没有匹配到正确的 location。
3. 静态资源的话，文件不存在，root 的路径错了。
4. 后端的服务返回 404。

解决：检查 URL，检查 location 的配置，检查 root 的路径，检查后端的服务。

Nginx 优化使用什么参数？性能调优有哪些？

就是之前的那些优化参数，worker_processes、worker_connections、epoll、gzip 这些。

用 shell 脚本统计 Nginx 日志文件里面的平均耗时应该怎么统计？用到哪些关键命令？

答案：

如果日志里有耗时的字段，比如 \$request_time，那么可以用 awk 来统计，比如：

代码块

```
1 awk '{sum+=$NF; count++} END{print sum/count}' access.log
```

就是把所有的耗时加起来，然后除以数量，得到平均耗时，用到 awk 命令。

分析 Nginx 访问日志，一条命令找出访问 IP 前十

答案：

代码块

```
1 awk '{print $1}' access.log | sort | uniq -c | sort -nr | head -10
```

就是取出 IP，然后排序，统计每个 IP 的次数，然后倒序，取前 10。

Nginx 的大日志有 5 个 G, 怎么找出 top10 的 IP 地址？

答案：

和上面的命令一样，awk 处理，因为 awk 是流式处理，不用把整个文件加载到内存，所以即使 5G 的文件，也能处理。

Nginx 和 Apache 的区别是什么？

答案：

- Nginx 是事件驱动的，异步非阻塞，并发能力强，资源消耗少，适合高并发。
- Apache 是进程 / 线程模型，每个连接一个进程，并发高了之后，资源消耗大，但是稳定，模块多。
- Nginx 可以做四层和七层负载，Apache 主要是 Web 服务器。
- Nginx 处理静态资源更快，Apache 处理动态的更成熟。

反向代理与负载均衡

负载均衡的作用是什么？怎么实现的？

答案：

负载均衡的作用是把请求分发到多个后端节点，分担压力，提升系统的并发能力，同时实现高可用，某个节点挂了，就把请求分到其他节点，避免单点故障。

实现的话，用 Nginx、LVS、HAProxy 这些，把请求按照一定的算法，分发到后端的节点。

负载均衡的转发模式有哪些？(轮询、IP 哈希、最少连接、加权轮询)

就是之前的 Nginx 的负载均衡模式，轮询、加权轮询、ip_hash、least_conn 这些。

正向代理与反向代理的区别？

答案：

- **正向代理**：代理的是客户端，客户端通过代理服务器访问外部的服务器，比如 VPN，代理客户端，隐藏客户端的 IP，客户端知道代理的存在。

- **反向代理**：代理的是服务端，客户端访问代理服务器，代理服务器把请求转发到后端的服务端，客户端不知道后端的服务器，隐藏服务端的 IP，对客户端透明。

SLB 和 Nginx 有什么区别？

答案：

SLB 是云厂商的负载均衡，是四层的，或者七层的，是云服务，托管的，不用自己维护，高可用，弹性扩展。

Nginx 是自己部署的，七层的负载均衡，需要自己维护，功能更灵活，可以做更多的自定义的配置，但是需要自己保证高可用，比如用 Keepalived。

Nginx 负载均衡和 LVS 负载均衡有什么区别？

答案：

- LVS 是四层的负载均衡，工作在传输层，转发速度快，性能高，支持更多的并发，但是功能简单，不能做应用层的处理。
- Nginx 是七层的，工作在应用层，功能强大，可以做 URL 转发、缓存、限流等，但是性能比 LVS 差点，并发不如 LVS。

Nginx、LVS、HAProxy 有什么区别？

答案：

- LVS：四层，性能最高，并发最大，功能简单，适合入口的负载。
- HAProxy：四层和七层，性能比 Nginx 高，功能比 LVS 强，适合做负载均衡，支持很多的算法。
- Nginx：七层，功能最丰富，除了负载，还能做 Web 服务器、缓存、重写等，但是性能比前两个差点。

Nginx 在配置上四层和七层代理有什么区别？

答案：

- 四层代理：在 stream 块里配置，转发 TCP 的流量，比如：

代码块

```
1  stream {
2      server {
3          listen 80;
4          proxy_pass 127.0.0.1:8080;
5      }
6  }
```

- 七层代理：在 http 块里配置，处理 HTTP 的请求，解析 HTTP 的头，做转发。

如何配置负载均衡？

答案：

配置 upstream，然后 proxy_pass：

代码块

```
1 upstream backend {
2     server 127.0.0.1:8080;
3     server 127.0.0.1:8081;
4 }
5 server {
6     location / {
7         proxy_pass http://backend;
8     }
9 }
```

location 匹配

location 模块里面代理静态页面时 root 和 alias 有什么区别？

答案：

- **root**：路径拼接，root 的路径加上 location 的路径，比如 root /html; location /img/，那么访问 /img/a.jpg，就会找 /html/img/a.jpg。
- **alias**：路径替换，alias 的路径替换 location 的路径，比如 alias /data/img/; location /img/，那么访问 /img/a.jpg，就会找 /data/img/a.jpg。

location 模块匹配的优先级是什么？

答案：

优先级从高到低：

1. 精确匹配：= 开头的，比如 location = /。
2. 前缀匹配，带 ^~ 的：^~ 开头的，匹配到就不再检查正则。
3. 正则匹配：~ 或者 ~* 开头的，按配置顺序匹配。
4. 普通前缀匹配：最长的前缀匹配。

Nginx 匹配模式是什么？

就是上面的，精确匹配、前缀匹配、正则匹配这些。

HTTPS

https 加密流程是什么？

答案：

1. 客户端发送 Client Hello，告诉服务端支持的加密套件、随机数。
2. 服务端返回 Server Hello，确认用的加密套件，还有服务端的随机数，然后返回证书。
3. 客户端验证证书，然后生成预主密钥，用服务端的公钥加密，发给服务端。
4. 服务端用私钥解密，得到预主密钥，然后双方根据随机数和预主密钥，生成会话密钥。
5. 之后双方就用会话密钥加密传输数据，对称加密。

证书公司怎么配置的？

答案：

先申请证书，比如 Let's Encrypt，或者购买的证书，得到证书文件和私钥文件，然后在 Nginx 里配置：

代码块

```
1  server {
2      listen 443 ssl;
3      server_name example.com;
4      ssl_certificate /path/to/cert.pem;
5      ssl_certificate_key /path/to/key.pem;
6  }
```

Web 服务对比

Nginx 和 Tomcat 什么区别？

答案：

- Nginx 是 Web 服务器，反向代理服务器，处理静态资源快，并发高，用来做入口，转发请求。
- Tomcat 是 Java 的应用服务器，处理动态的请求，比如 JSP、Servlet，运行 Java 的应用。
一般 Nginx 在前面，做反向代理，把动态的请求转发到 Tomcat，静态的自己处理。

除了 Nginx 还部署过什么 Web 服务器？是 Apache 吗？

答案：

部署过 Apache，还有 Tomcat，还有 Jetty。

Tomcat 负载高排查流程是什么？

答案：

1. 先看 CPU、内存的使用，是不是资源不够。
2. 看线程栈，`jstack`，看 Tomcat 的线程，是不是有死锁，或者阻塞的线程。
3. 看 GC，`jstat`，看 GC 是不是频繁，是不是内存不够。
4. 看访问日志，是不是有很多请求，是不是流量太高。

5. 看数据库，是不是慢 SQL，导致 Tomcat 的请求阻塞。
6. 然后优化，比如优化代码，优化 SQL，扩容。

Tomcat 假死，但是进程还在怎么解决？什么原因？

答案：

假死就是进程还在，但是无法处理请求，原因：

1. 内存溢出，OOM 了，进程还在，但是无法工作。
2. 死锁，线程都死锁了，无法处理请求。
3. GC 停顿，Full GC 太久，导致进程卡住。
4. 数据库的连接阻塞，所有的线程都在等数据库。

解决：

5. 先重启 Tomcat，恢复业务。
6. 然后看日志，找原因，看 GC 日志，线程栈，内存快照，分析问题。
7. 优化，比如调整 JVM 参数，优化代码，优化 SQL。

公司里面 Tomcat 和哪些中间件搭配使用，优化过什么方面？

答案：

和 Nginx 搭配，Nginx 做反向代理，Tomcat 做应用服务器，优化过 Tomcat 的线程池，调整最大线程数，还有 JVM 的参数，调整堆内存，GC 算法，还有连接池，数据库的连接池，优化连接数。

如何查看 Tomcat 15 分钟的日志？

答案：

```
tail -n 1000 logs/catalina.out，或者用 sed -n '/时间1/,/时间2/p' logs/catalina.out，过滤 15 分钟内的日志。
```

高可用 / 集群

Keepalived

Keepalived 是干什么用的？

答案：

Keepalived 是用来做高可用的，基于 VRRP 协议，实现虚拟 IP，把多个机器组成一个集群，主节点挂了，备用节点会接管虚拟 IP，实现服务的高可用，比如 Nginx 的高可用，主 Nginx 挂了，备用的就接管 VIP，用户无感知。

Nginx 和 Keepalived 是怎么组合使用的？具体配置流程是怎样的？

答案：

两台 Nginx 服务器，都安装 Keepalived，配置 VRRP，主节点优先级高，备用的低，虚拟 IP 是同一个，然后 Keepalived 会检测 Nginx 的状态，如果主节点的 Nginx 挂了，Keepalived 就会把 VIP 切到备用节点，这样用户访问 VIP，就会到备用的 Nginx，实现高可用。

配置：

主节点的 keepalived.conf：

代码块

```
1  global_defs {
2      router_id NGINX_MASTER
3  }
4  vrrp_instance VI_1 {
5      state MASTER
6      interface eth0
7      virtual_router_id 51
8      priority 150
9      advert_int 1
10     authentication {
11         auth_type PASS
12         auth_pass 1111
13     }
14     virtual_ipaddress {
15         192.168.1.100
16     }
17 }
```

备用节点的 priority 改成 100，state 改成 BACKUP。

Keepalived 除了和 Nginx 搭配，还能和其他组件组合实现高可用吗？

答案：

可以，比如 MySQL 的主从高可用，还有 Redis，还有 LVS，都可以，只要是需要 VIP 的，都可以用 Keepalived。

Keepalived 是几个节点？使用什么样的轮询策略？

答案：

一般是主备，两个节点，一主一备，不是轮询，主节点处理所有的请求，备用的待命，主挂了，备用才上。

也可以做多主多备，但是一般主备就够了。

使用的是抢占还是非抢占模式？

答案：

默认是抢占模式，主节点恢复了，就会把 VIP 抢回去，非抢占的话，主恢复了，也不抢，备用继续当主，避免切换的波动。

主节点挂了的话，怎么去处理轮询策略？优先级如何设置？

答案：

主节点挂了，备用节点检测到，就会把 VIP 接管，然后处理请求，优先级的话，主节点的优先级比备用的高，比如主是 150，备用是 100，这样主节点正常的时候，主是 MASTER。

LVS

LVS 的几种模式是什么？(NAT、DR、TUN)

答案：

1. **NAT 模式**：网络地址转换，请求和响应都经过 LVS，LVS 修改目标 IP 和源 IP，把请求转发到后端，后端的响应再经过 LVS，返回给客户端，优点是后端不用配置 VIP，缺点是 LVS 的压力大，因为所有的流量都经过它。
2. **DR 模式**：直接路由，请求经过 LVS，LVS 修改 MAC 地址，转发到后端，后端的响应直接返回给客户端，不用经过 LVS，所以 LVS 的压力小，性能高，但是后端需要配置 VIP，而且要和 LVS 在同一个网段。
3. **TUN 模式**：隧道模式，把请求封装成 IP 隧道，转发到后端，后端解封装，响应直接返回给客户端，支持跨网段，适合跨地域的集群。

简要介绍一下你对 Nginx 和 LVS 的理解，以及它们在运维工作中的应用场景和区别

答案：

LVS 是四层负载，性能高，并发大，适合做整个集群的入口，处理大流量的入口，Nginx 是七层，功能丰富，适合做业务层的负载，做反向代理，处理应用层的逻辑，比如 URL 转发，缓存，限流等，一般架构是：用户 -> LVS -> Nginx -> 后端服务，LVS 做入口的负载，Nginx 做业务的负载。

所有的负载均衡都有高可用吗？

答案：

不是，负载均衡本身也可能单点，所以需要高可用，比如 LVS 的话，用 Keepalived 做高可用，Nginx 的话，也用 Keepalived，SLB 的话，云厂商已经做了高可用了。

你们的负载有多大？并发是有多大？秒级 PV 是多少？

答案：

我们的负载，峰值的话，并发大概 1 万，秒级 PV 大概 10 万。

脑裂

Keepalived 脑裂问题怎么预防和解决？出现脑裂的原因有哪些？

答案：

脑裂就是主节点和备用节点之间的网络断了，备用节点以为主节点挂了，就把 VIP 抢了，这时候两个节点都有 VIP，导致冲突。

原因：

1. 主备之间的网络断了，无法通信。
2. 主节点的进程卡住了，但是没挂，无法发送 VRRP 报文。
3. 防火墙拦截了 VRRP 的报文。

预防和解决：

1. 双链路，主备之间用多个网络链路，避免单链路断了。
2. 加仲裁，比如用第三方的节点，或者用脚本，检测主节点的状态，同时检测网关，确认是不是网络断了。
3. 调整抢占的时间，避免太快切换。
4. 开启 nopreempt，非抢占模式，减少切换。
5. 监控脑裂，发现了自动处理。

CDN

CDN 加速原理是什么？

答案：

CDN 是内容分发网络，把静态资源缓存到各地的边缘节点，用户访问的时候，会自动调度到最近的边缘节点，从边缘节点获取资源，不用到源站，这样减少延迟，提升速度，同时减少源站的压力。

SLB 和 CDN 怎么配置？

答案：

SLB 是负载均衡，用来做源站的负载，CDN 的话，把 CDN 的源站配置成 SLB 的 IP，这样 CDN 回源的时候，就会到 SLB，SLB 把请求分发到源站的节点。

什么是 CDN？

就是上面的，内容分发网络，加速静态资源的访问。

数据库

MySQL 主从

MySQL 主从原理是什么？

答案：

MySQL 主从复制的原理：

1. 主库把数据的修改，写到 binlog 日志里。

2. 从库的 IO 线程，连接主库，请求 binlog，主库的 log dump 线程，把 binlog 发给从库的 IO 线程。
3. 从库的 IO 线程把 binlog 写到本地的 relay log（中继日志）里。
4. 从库的 SQL 线程，读取 relay log，解析成 SQL，执行，同步数据，这样主从的数据就一致了。

MySQL 主从复制延迟怎么导致的？怎么解决？

答案：

延迟的原因：

1. 主库的写压力太大，binlog 生成的太快，从库跟不上。
2. 从库的性能差，CPU、IO 不够，执行 SQL 慢。
3. 大事务，主库一个大事务，执行很快，从库执行要很久，导致延迟。
4. 从库的读压力太大，占用了资源，导致同步慢。
5. 网络延迟，主从之间的网络慢，binlog 传输慢。

解决方法：

1. 优化从库的性能，升级硬件，提升 IO 和 CPU。
2. 拆分大事务，把大事务拆成小事务。
3. 做读写分离，把读请求分到多个从库，减少从库的读压力。
4. 优化网络，提升主从之间的网络速度。
5. 用并行复制，让从库的 SQL 线程并行执行，提升同步速度。

MySQL 主从同步的底层逻辑是什么？使用什么工具实现主从同步？

答案：

底层逻辑就是 binlog 的复制，主库写 binlog，从库拉取，然后重放，工具的话，原生的 MySQL 就支持，不用额外的工具，也可以用第三方的，比如 MyCat，或者 MaxScale，或者 MHA。

主从同步用户的权限有什么特殊要求吗？

答案：

主从同步的用户，需要有 REPLICATION SLAVE 权限，这样才能读取主库的 binlog。

数据库的负载策略是如何实现的？

答案：

读写分离，主库负责写，从库负责读，然后用中间件，比如 MyCat，或者 Sharding-JDBC，把读请求分发到从库，写请求分到主库，也可以用程序自己实现，读的时候连从库，写的时候连主库。

如何知道数据库主从出问题了？从底层上说说看？

答案：

查看从库的状态，`show slave status\G`，看 `Slave_IO_Running` 和 `Slave_SQL_Running` 是不是都是 Yes，如果不是，说明有问题，还有 `Seconds_Behind_Master`，延迟是不是太大，如果太大，说明同步延迟了。

底层的话，IO 线程是不是正常连接主库，SQL 线程是不是正常执行 relay log。

你们数据同步状态怎么保证从库是最新的状态？

答案：

监控 `Seconds_Behind_Master`，监控延迟，如果延迟太大，就告警，同时监控 IO 和 SQL 线程的状态，如果线程停了，就告警，同时定期校验数据，用 `pt-table-checksum`，检查主从的数据是不是一致。

MySQL 的架构是什么？MHA 还是什么？

答案：

我们的 MySQL 架构是一主多从，然后用 MHA 做高可用，主库挂了，MHA 自动把从库提升为主库，保证高可用。

假如有 100 个读写请求，按照轮询策略应该怎么分配？

答案：

写请求都分到主库，读请求轮询分到从库，比如有 2 个从库，那么 50 个读请求，每个从库分 25 个，写的话，所有的写都到主库。

MySQL 备份恢复

MySQL 的备份方案是什么？

答案：

我们用的是 xtrabackup 做物理备份，每天全量备份，然后每小时增量备份，同时开启 binlog，这样可以恢复到任意时间点，备份的文件会传到 OSS，做异地备份。

数据库的备份和恢复怎么做？

答案：

备份用 xtrabackup，`xtrabackup --backup --target-dir=/backup`，全量备份，然后增量的话，`xtrabackup --backup --target-dir=/inc --incremental-basedir=全量的目录`。

恢复的话，先准备全量备份，`xtrabackup --prepare --target-dir=/backup`，然后应用增量，然后把文件复制到 MySQL 的数据目录，然后启动 MySQL，或者如果是用 binlog 的话，恢复到备份的时间点，然后应用 binlog，恢复到故障的时间点。

备份是怎么实现的？自动化备份怎么实现？

答案：

写脚本，然后用 crontab 定时执行，每天凌晨执行备份，然后备份完之后，把备份文件传到 OSS，然后检查备份的结果，如果失败了，就告警。

怎么利用 binlog 恢复数据？

答案：

首先，找到备份的时间点，然后把 binlog 里，备份之后的操作，解析出来，然后执行，比如：

```
mysqlbinlog --start-position=xxx binlog.000001 | mysql -u root -p
```

，这样就把备份之后的操作重放，恢复到指定的时间点。

数据损坏了需要恢复数据怎么办？

答案：

先停业务，然后用最近的备份恢复数据，然后用 binlog 恢复备份之后的数据，尽量减少数据的丢失，然后启动业务，然后排查数据损坏的原因。

实际工作有没有遇到过数据恢复的情况？

答案：

遇到过，比如有一次开发误删了表，然后用备份恢复，然后用 binlog 恢复了删除之前的数据，把数据找回来了。

MySQL 备份文件存放地点，本地还是服务器？

答案：

本地先存一份，然后传到 OSS，异地存储，避免本地的磁盘坏了，备份也没了。

MySQL 优化与基础

MySQL 你做过调优吗？做过哪个方面？

答案：

做过，比如 SQL 的调优，给慢查询加索引，优化 SQL 的写法，还有参数的调优，调整缓冲池的大小，调整连接数，还有架构的调优，读写分离，分库分表，还有主从的调优，减少复制延迟。

数据库查询突然变慢 (从毫秒级变成几秒甚至十几秒), 可能的原因有哪些？针对这些原因做哪些优化？

答案：

原因：

1. 索引失效，比如对索引列用了函数，导致索引没用，全表扫描。
2. 锁等待，比如有锁表，或者行锁，查询等待锁。
3. 缓存失效，之前的缓存失效了，所有的请求都到数据库。
4. 数据量太大，表的数据太多，查询慢。
5. 数据库的资源不够，CPU、IO、内存不够，负载高。

6. 执行计划变了，比如统计信息过期，MySQL 选了错误的索引。

优化：

1. 优化 SQL，加索引，避免索引失效。
2. 优化事务，减少锁的时间，避免大事务。
3. 优化缓存，预热缓存，避免缓存击穿。
4. 分库分表，拆分大表，减少单表的数据量。
5. 扩容数据库的资源，提升性能。
6. 更新统计信息，重新分析表，让 MySQL 选对索引。

除了资源维度和 "缺索引" 外，数据库查询突然变慢还有哪些数据库自身技术层面的原因？

答案：

1. 事务锁，比如长事务，导致锁等待，查询阻塞。
2. 执行计划选错，MySQL 的优化器选错了索引，导致全表扫描。
3. 统计信息过期，导致优化器判断错误。
4. 缓冲池满了，内存不够，数据都读到磁盘，慢了。
5. 页分裂，表的碎片太多，查询慢。
6. redo log、undo log 的瓶颈，写的太慢。

当业务数据量或请求量激增导致数据库查询延迟飙升，从系统架构层面应该如何优化？

答案：

1. 读写分离，把读请求分到从库，分担主库的压力。
2. 分库分表，把大表拆成小表，减少单表的压力。
3. 加缓存，把热点数据放到 Redis，减少数据库的请求。
4. 读写分离，加更多的从库，分担读的压力。
5. 用 NewSQL，比如 TiDB，分布式数据库，弹性扩展。
6. 归档冷数据，把冷数据归档到其他存储，减少热数据的量。

MySQL 的 CPU 会突然飙升，作为运维排查思路是什么？

答案：

1. 先看 show processlist，看有没有慢查询，有没有很多的查询在执行。
2. 看慢查询日志，找慢 SQL，是不是有新的慢 SQL 导致的。
3. 看 innodb 的状态，有没有锁等待，有没有事务阻塞。
4. 看 CPU 的使用，是不是 MySQL 的进程占用了 CPU，是的话，说明是 SQL 的问题。

5. 然后优化慢 SQL，加索引，或者杀掉异常的查询。

MySQL 左右内外连接如何连接？左表连接跟其他连接有什么不同？

答案：

- **左连接**：以左表为基础，左表的所有行都保留，右表匹配不到的话，就填 NULL，比如 A left join B，A 的所有行都有，B 的匹配的才加。
- **右连接**：和左连接相反，以右表为基础，右表的所有行都保留，左表匹配不到的填 NULL。
- **内连接**：只保留两边都匹配的行，只有两边都有的，才会出现在结果里。
- **全连接**：保留两边的所有行，匹配不到的填 NULL，MySQL 不支持，需要用 union。

MySQL 子表查询你会吗？

答案：

会，子查询就是在查询里嵌套查询，比如 `select * from a where id in (select id from b)`，不过子查询有时候性能不好，所以一般改成 join。

如何查看 MySQL 一个语句的执行过程？

答案：

用 explain，`explain select * from a where id=1`，然后看执行计划，看 type、key、rows、Extra 这些字段，看有没有用索引，有没有全表扫描。

在 MySQL 中需要删除一张大表，怎么删除能防止产生死锁？

答案：

不能直接 drop table，因为 drop table 会锁表，导致阻塞，所以可以先把表重命名，`rename table old_table to tmp_table`，这样很快，然后再慢慢删 tmp_table，这样就不会阻塞业务了，因为重命名是很快的，不会锁很久。

数据库锁表了如何处理？

答案：

1. 先 show processlist，找到导致锁表的进程，看是哪个事务。
2. 然后看 innodb_lock_wait_timeout，还有 show engine innodb status，看锁的信息。
3. 如果是长事务导致的，杀掉那个事务，`kill PID`，释放锁。
4. 然后优化事务，避免大事务，避免长事务。

数据库 ACID 机制都有什么？

答案：

ACID 是事务的四个特性：

- **A（原子性）**：事务的操作要么全部成功，要么全部失败，不会部分成功。

- **C（一致性）**：事务执行前后，数据库的完整性约束没有被破坏，比如主键唯一，外键一致。
- **I（隔离性）**：多个事务之间是隔离的，互相不影响，有不同的隔离级别。
- **D（持久性）**：事务提交之后，修改就永久保存了，即使系统崩溃，也不会丢。

MySQL 常用的存储引擎以及区别？

答案：

- **InnoDB**：默认的，支持事务，支持行锁，支持外键，崩溃恢复，适合大部分的业务，支持崩溃恢复，安全。
- **MyISAM**：不支持事务，不支持行锁，只支持表锁，查询快，适合读多写少的场景，但是崩溃恢复慢。
- **Memory**：数据存在内存里，速度快，但是重启就丢了，适合临时表。

日常工作中优化 SQL 的方案有哪些？

答案：

1. 加索引，给查询的字段加索引，避免全表扫描。
2. 避免索引失效，比如不要对索引列用函数，不要用 like % xxx，不要用 or。
3. 避免 select *，只查需要的字段，减少数据传输。
4. 分页优化，大分页的话，用 id 过滤，避免 offset 太大。
5. 用 join 代替子查询，子查询性能差。
6. 拆分大 SQL，把大 SQL 拆成小的，避免锁太久。
7. 批量操作，代替循环的单条操作，减少交互。

MHA

MHA 的底层原理是什么？

答案：

MHA 是 MySQL 的高可用工具，原理是：

1. 监控主库的状态，当主库挂了，MHA 会从从库里选一个最新的，提升为新的主库。
2. 然后把其他的从库，指向新的主库，重新建立主从，整个过程自动，不用人工干预，保证数据的一致性，尽量减少数据的丢失。

MySQL 的 MHA 高可用如何搭建？

答案：

1. 先配置主从复制，所有的节点之间配置 SSH 免密登录。
2. 安装 MHA 的节点和 manager 包。

3. 配置 MHA 的配置文件，配置各个节点的信息，用户，密码。
4. 测试 SSH 的连通性，测试主从的状态。
5. 启动 MHA 的 manager，就可以了。

Redis

Redis 有哪些基本数据类型？

答案：

- String：字符串，最基本的，比如缓存、计数器。
 - List：列表，有序的，比如队列。
 - Set：集合，无序的，去重，比如共同好友。
 - Hash：哈希，键值对，比如存储用户信息。
 - ZSet：有序集合，有序的，比如排行榜。
- 还有高级的，比如 Bitmap、HyperLogLog、Geo。

Redis 默认端口是什么？

答案：

6379。

Redis 怎么搭集群？

答案：

Redis Cluster，至少 3 主 3 从，把数据分到 16384 个槽，每个节点负责一部分槽，然后节点之间互相通信，客户端访问任意节点，就可以路由到正确的节点，自动分片，高可用，主挂了，从就提升为主。

Redis 集群的作用是什么？

答案：

1. 分片，把数据分到多个节点，突破单节点的内存限制，存储更多的数据。
2. 高可用，每个主节点有从节点，主挂了，从可以顶上，避免单点故障。
3. 分担压力，多个节点，处理更多的请求，提升并发。

Redis cluster 模式是什么？

就是上面的 Redis Cluster，分布式的集群模式，分片 + 高可用。

Redis 有多少个数据库？

答案：

默认是 16 个，0-15，是逻辑的数据库，每个数据库是独立的，默认用 0 号，集群模式的话，不支持多数据库。

Redis 正在运行看他哪些指标？

答案：

看内存的使用，连接数，QPS，命中率，过期键的数量，阻塞的客户端，持久化的状态，有没有慢查询。

你们用 Redis 来干什么？

答案：

用来做缓存，缓存热点数据，减少数据库的压力，还有做计数器，比如限流，还有做分布式锁，还有做消息队列，还有 session 存储。

Redis 缓存用过没有？搭建的什么结构的 Redis？

答案：

用过，搭建的 Redis Cluster 集群，3 主 3 从，用来做缓存，还有部分用的主从，用来做分布式锁。

如何监控一个云产品 Redis 的状态？

答案：

用云厂商的监控，监控 Redis 的 CPU、内存、连接数、QPS、命中率这些指标，然后配置告警，当指标超过阈值，就告警，同时用 Prometheus，采集 Redis 的指标，自己监控。

国产数据库

国产数据库你了解哪些？会用国产数据库吗？

答案：

了解，比如达梦、人大金仓、OceanBase、TiDB、TDSQL，这些，会用，之前做信创项目的时候，用过达梦数据库，和 MySQL 兼容，迁移也比较方便。

Shell 脚本

awk/sed/grep

awk 用过吗？一般都用来干啥？

答案：

用过，awk 是文本处理工具，用来处理结构化的文本，比如日志，用来做统计，比如统计 IP 的次数，统计平均耗时，取列，比如取日志的第一列，第二列，处理数据，很方便。

用 awk、sed、grep 做数据统计分析

用过，比如统计 Nginx 的访问量，统计 IP，统计状态码，都是用这三个工具。

grep 有哪些参数？

答案：

- -i: 忽略大小写。
- -v: 反向匹配，排除匹配的行。
- -n: 显示行号。
- -c: 统计匹配的行数。
- -A: 显示匹配行的后面 n 行。
- -B: 显示前面 n 行。
- -C: 显示前后 n 行。
- -r: 递归目录。

find 用过哪些？

答案：

find 用来查找文件，比如找大文件，找过期的文件，比如 `find / -size +100M`，找大文件，`find / -mtime +30 -delete`，找 30 天前的文件删除，`find -name "*.log"`，找日志文件。

find 和 grep 的区别是什么？

答案：

- find: 是查找文件，根据文件的属性，比如名字、大小、时间，找文件。
- grep: 是查找文件里的内容，根据文件里的文本，找匹配的行。

实时去查看日志文件用什么命令？

`tail -f`。

sed 如何替换文件中的某个字符？

答案：

`sed -i 's/原字符/新字符/g' 文件名`，比如 `sed -i 's/北京/天津/g' file.txt`，全局替换。

过滤出文件中包含 root 的行 (grep/sed/awk 三种方式)

答案：

- grep: `grep root file.txt`
- sed: `sed -n '/root/p' file.txt`

- awk: `awk '/root/' file.txt`

取出文件的第某行第某列

答案:

比如取第 2 行第 3 列: `awk 'NR==2 {print $3}' file.txt`。

要将文件中所有的某字符替换成某字符

就是 sed 的 s 命令, `sed -i 's/原/新/g' file`。

想看过滤到内容的上和下面的内容 (grep -A/-B/-C)

答案:

比如 `grep -C 5 root file.txt`, 就是显示 root 行的前后 5 行, -A 是后面, -B 是前面, -C 是前后。

分析 Nginx 访问日志, 一条命令找出访问 IP 前十

之前的那个, `awk '{print $1}' access.log | sort | uniq -c | sort -nr | head -10`。

怎么替换一个文件中的空行? 具体命令是什么?

答案:

`sed -i '/^$/d' file.txt`, 删除空行。

用 shell 脚本怎么把一个文档空格都去掉?

答案:

`sed -i 's/ //g' file.txt`, 把所有的空格去掉, 或者 `tr -d ' ' < file.txt`。

脚本编写

写过哪些脚本? 讲讲逻辑

答案:

写过很多, 比如备份的脚本, 自动备份 MySQL, 然后传到 OSS, 还有巡检的脚本, 自动检查系统的状态, 生成报告, 还有清理日志的脚本, 自动清理过期的日志, 还有监控的脚本, 检查服务的状态, 告警, 还有部署的脚本, 自动部署服务, 更新代码。

用 shell 脚本怎么计算浮点数?

答案:

用 bc, 比如 `echo "1.1+2.2" | bc`, 或者 awk, `awk 'BEGIN{print 1.1+2.2}'`。

如何用 shell 写一个替换域名的脚本?

答案:

比如，遍历所有的配置文件，然后用 sed 替换，比如：

代码块

```
1  for file in /etc/nginx/conf.d/*.conf; do
2      sed -i 's/old.com/new.com/g' $file
3  done
```

然后重启 Nginx。

Shell 中第一行 #!/bin/bash 代表什么意思？

答案：

指定这个脚本的解释器，告诉系统，用 /bin/bash 来执行这个脚本，这样即使你用 sh 来执行，也会用 bash，避免不同的 shell 的差异。

Shell 脚本的循环语句怎么写？循环里的 break 和 continue 的区别是什么？

答案：

for 循环：

代码块

```
1  for i in {1..10}; do
2      echo $i
3  done
```

while 循环：

代码块

```
1  while [ $i -lt 10 ]; do
2      i=$((i+1))
3  done
```

break 是跳出整个循环，continue 是跳过当前这次循环，继续下一次。

判断磁盘占用率，当使用率大于 95% 时，删除 /var/log/ 下的 7 天之前的日志，怎么写？

答案：

代码块

```
1  usage=$(df / | awk 'NR==2 {print $5}' | sed 's/%//')
2  if [ $usage -gt 95 ]; then
3      find /var/log -type f -mtime +7 -delete
```

. 2 > xx 、  无效公式 、 &>xx 这三个命令分别代表什么意思？

答案：

- 2>xx ：把标准错误（stderr）重定向到 xx 文件。
- 2>&1 ：把标准错误重定向到标准输出（stdout），也就是把错误和输出合并。
- &>xx ：把标准输出和标准错误都重定向到 xx 文件，相当于 >xx 2>&1 。

正则表达式中 .、*、+、[]、^、\$、\w、\d、\s 分别代表什么？

答案：

- . ：匹配任意单个字符，除了换行。
- * ：匹配前面的字符 0 次或多次。
- + ：匹配前面的字符 1 次或多次。
- [] ：匹配括号里的任意一个字符，比如 [abc]，匹配 a 或 b 或 c。
- ^ ：匹配行的开头。
- \$ ：匹配行的结尾。
- \w ：匹配单词字符，字母、数字、下划线。
- \d ：匹配数字。
- \s ：匹配空白字符，空格、制表符。

用 shell 脚本统计文本的次数跟排序，用具体命令

比如统计单词的次数：`awk '{for(i=1;i<=NF;i++) print $i}' text.txt | sort | uniq -c | sort -nr`。

Docker

基础命令

你经常用的 docker 命令有哪些？

答案：

- docker ps ：查看运行的容器。
- docker logs ：查看容器的日志。
- docker exec ：进入容器。

- `docker build`：构建镜像。
- `docker pull/push`：拉取推送镜像。
- `docker images`：查看镜像。
- `docker rm/rmi`：删除容器镜像。
- `docker cp`：复制文件。
- `docker restart/start/stop`：管理容器。

如何查看正在运行的容器？

`docker ps`，`docker ps -a` 查看所有的，包括停止的。

如何查看容器日志异常？

`docker logs 容器名`，看日志，有没有错误，`docker logs --tail 100 -f`，实时看。

Docker 怎么查看守护进程？

答案：

`systemctl status docker`，查看 docker 守护进程的状态，或者 `ps aux | grep docker`。

Docker 容器的数据持久化方案包括什么？

答案：

三种：

1. **Volume**：Docker 管理的，存到 `/var/lib/docker/volumes/`，推荐，持久化，容器删了数据还在，跨平台好。
2. **Bind Mount**：绑定挂载，把宿主机的目录挂载到容器，宿主机的文件，直接挂载，方便，但是依赖宿主机的路径。
3. **tmpfs Mount**：挂载到内存，临时的，容器停了就没了，适合敏感数据，不用存到磁盘。

如何实现容器数据持久化？

就是用上面的，比如 `docker run -v myvolume:/data`，用 volume，或者 `-v /host/path:/container/path`，绑定挂载。

批量停止所有运行中容器、批量删除所有已停止容器的命令是什么？

答案：

- 批量停止：`docker stop $(docker ps -q)`
- 批量删除停止的：`docker container prune`，或者 `docker rm $(docker ps -aq -f status=exited)`

Docker 如何解决与宿主机的时间同步问题？

答案：

挂载宿主机的 localtime 文件，`docker run -v /etc/localtime:/etc/localtime:ro`，这样容器的时间就和宿主机一样了，或者设置时区环境变量，`-e TZ=Asia/Shanghai`。

docker 后面跟什么参数？

比如 -d 后台运行，-p 端口映射，-v 挂载，-e 环境变量，--name 容器名，--network 网络。

如何还原一个 Docker 容器的启动命令？

答案：

用 `docker runlike`，或者用 `docker inspect 容器名`，然后解析里面的配置，或者用第三方的工具，比如 `docker-runlike`。

查看磁盘占用空间，知道哪些位置满了，但不知道哪些容器日志满了怎么办？

答案：

用 `docker system df`，查看 Docker 的磁盘使用，然后 `docker system prune`，清理无用的镜像和容器，然后用 `du -sh /var/lib/docker/containers/*`，看哪个容器的日志目录大，找到对应的容器，清理日志。

如何删除镜像？

`docker rmi 镜像ID`。

如何查看容器资源占用以及使用情况？

`docker stats`，查看容器的 CPU、内存、网络、磁盘的使用情况。

Dockerfile

用过 Dockerfile 里的哪些指令？

答案：

FROM、RUN、COPY、ADD、CMD、ENTRYPOINT、ENV、WORKDIR、EXPOSE、USER、VOLUME 这些。

Dockerfile 的常用字段有哪些？

就是上面的这些。

Dockerfile 中 COPY 和 ADD 有什么区别？

答案：

- COPY：只是简单的把本地的文件复制到镜像里，支持本地文件，也支持远程的 URL？不，COPY 不支持，COPY 只能复制本地的。

- ADD: 比 COPY 多了功能, 比如可以自动解压 tar、gz 等压缩文件, 还支持远程的 URL, 下载文件, 但是一般推荐用 COPY, 因为 ADD 的功能有时候会有意外, 比如不小心解压了文件。

CMD 和 ENTRYPOINT 有什么区别? 什么时候用 ENTRYPOINT 什么时候用 CMD?

答案:

- CMD: 是容器的默认命令, 但是可以被 docker run 的命令覆盖, 比如 `docker run image echo hello`, 就会覆盖 CMD。
- ENTRYPOINT: 是容器的入口命令, 不会被覆盖, docker run 的参数会作为参数传给 ENTRYPOINT。
比如, 如果容器是一个工具, 比如 nginx, 你希望用户运行的时候, 默认启动 nginx, 但是也可以传参数, 就用 ENTRYPOINT, CMD 是默认的参数, 如果只是简单的运行一个脚本, 用 CMD 就可以了。

Dockerfile 中环境变量的关键字是啥?

ENV, 比如 `ENV JAVA_HOME /usr/local/jdk`。

一个 Dockerfile 可以有几个 FROM?

答案:

多阶段构建的话, 可以有多个 FROM, 每个阶段一个, 比如先构建, 然后把构建的结果复制到新的镜像, 这样可以减小镜像的大小, 比如:

代码块

```
1 FROM golang as builder
2 WORKDIR /app
3 COPY . .
4 RUN go build -o app .
5
6 FROM alpine
7 COPY --from=builder /app/app /app
8 CMD ["/app"]
```

这样就两个 FROM, 最后镜像只有后面的, 很小。

多个 RUN 怎么写在一起?

答案:

用 && 连接, 比如:

代码块

```
1 RUN yum install -y gcc && \
```

这样就把多个 RUN 合并成一个，减少镜像的层数，减小镜像大小。

Dockerfile 层的概念是什么？

答案：

Docker 的镜像是分层的，每个 RUN、COPY 等指令，都会生成一个层，层是只读的，可以共享，比如多个镜像用同一个基础镜像，就可以共享基础的层，节省空间，构建的时候，也可以缓存层，不用重新构建，所以合并 RUN 指令，减少层数，减小镜像大小。

在编写 Dockerfile 时，镜像是动态的，打包时可以传进来，这个机制是怎么做的？

答案：

用 ARG，构建参数，比如 ARG VERSION=latest，然后构建的时候，docker build --build-arg VERSION=1.0，这样就可以把参数传进来，动态的构建镜像。

镜像管理

怎么在构建 Dockerfile 的时候减小镜像体积和构建速度？

答案：

1. 用多阶段构建，把构建的阶段和运行的阶段分开，最后镜像只保留运行的，去掉构建的工具。
2. 合并 RUN 指令，减少层数。
3. 用更小的基础镜像，比如 alpine，比 centos 小很多。
4. 清理缓存，比如 yum install 之后，yum clean all，删除缓存。
5. 利用层缓存，把变化少的指令放在前面，变化多的放在后面，利用缓存，提升构建速度。
6. 不要把无用的文件放到镜像里，用 .dockerignore，忽略不需要的文件。

怎么保证打出来的 image 是最小的？

就是上面的，多阶段，小基础镜像，清理缓存，合并指令。

构建镜像的时候有什么优化的方法？

就是上面的这些。

在业务容器里安装服务，哪种安装方式会更节省容器体积？

答案：

用 alpine 的包，apk 安装，比 yum 小很多，因为 alpine 本身很小，或者用静态编译的二进制，不用安装依赖，直接运行。

怎么把 Docker 镜像传到 Harbor 仓库，再拉下来？

答案：

1. 先给镜像打标签，`docker tag image harbor.example.com/project/image:tag`。
2. 然后登录 Harbor，`docker login harbor.example.com`。
3. 然后 push，`docker push harbor.example.com/project/image:tag`。
4. 拉的话，`docker pull harbor.example.com/project/image:tag`。

Harbor 仓库高可用原理是什么？

答案：

Harbor 的高可用，是用共享存储，比如 NAS，所有的节点都挂载同一个存储，然后前面用负载均衡，多个 Harbor 节点，共享存储，这样一个节点挂了，其他的还能工作，数据都在共享存储里。

docker buildx 知道吗？

答案：

知道，buildx 是 Docker 的构建工具，支持多架构的构建，比如构建 x86 和 ARM 的镜像，一次构建，多个架构的镜像。

网络

Docker 有多少种网络模式？

答案：

1. **bridge**：默认的，桥接网络，容器有独立的网络，通过网桥连接，NAT 访问宿主机的网络。
2. **host**：主机网络，容器和宿主机共享网络，没有隔离，容器的端口就是宿主机的端口。
3. **none**：没有网络，只有 lo。
4. **overlay**：跨主机的网络，Swarm 的，多主机的容器通信。
5. **macvlan**：给容器分配 MAC 地址，像物理机一样。

bridge 和 host 的区别，各有什么优缺点？

答案：

- **bridge**：有网络隔离，容器的网络是独立的，端口需要映射，安全，但是有一点性能损耗，NAT 的损耗。
- **host**：没有隔离，共享宿主机的网络，性能好，没有损耗，但是没有隔离，端口会冲突，不安全。

容器里的 DNS 是怎么配置的？

答案：

Docker 默认会把宿主机的 `/etc/resolv.conf` 复制到容器里，或者你可以用 `--dns` 指定 DNS 服务器，比如 `docker run --dns 8.8.8.8`。

Docker Compose

docker-compose 几种类型？

答案：

docker-compose 是用来定义和运行多容器的应用的，用一个 yaml 文件，定义服务、网络、存储，然后一键启动，类型的话，服务的类型，比如 deployment，或者什么？不对，docker-compose 的配置里，服务的类型，其实就是容器的配置，还有网络、卷的类型。

用过 docker-compose 吗？

用过，用来部署多容器的应用，比如 WordPress，或者微服务，一个文件定义所有的服务，一键启动。

depends_on 是做什么的？

答案：

depends_on 是用来定义服务的依赖，比如 web 服务依赖 db 服务，depends_on: db，这样 docker-compose 启动的时候，会先启动 db，再启动 web，保证启动的顺序，但是它不会等 db 完全准备好，只是启动顺序。

使用 docker-compose 如何实现有顺序依赖的服务的启动顺序？

答案：

用 depends_on，定义依赖，然后启动的时候，就会按顺序启动，但是如果等要等服务完全准备好，就需要自己写脚本，等待 db 的端口通了，再启动 web。

如果在 docker-compose 启动过程中，其中一个项目出问题如何排查？

答案：

看日志，`docker-compose logs 服务名`，看服务的日志，有没有错误，然后看容器的状态，`docker-compose ps`，看服务是不是退出了，然后排查配置，是不是配置错了，端口冲突了。

Docker 与虚拟机

Docker 跟虚拟机的区别是什么？

答案：

- Docker 是容器，共享宿主机的内核，轻量级，启动快，资源占用小，但是隔离性差点，只能运行相同内核的系统。
- 虚拟机是完全的操作系统，有自己的内核，隔离性好，能运行不同的系统，但是启动慢，资源占用大，重。

Docker 为什么比 VM 快？

答案：

因为容器共享宿主机的内核，不用模拟硬件，不用启动整个操作系统，启动的时候，只是启动一个进程，所以启动快，秒级，资源占用小，而 VM 要启动整个操作系统，模拟硬件，所以慢。

Docker 的优缺点有哪些？

答案：

优点：

1. 轻量，启动快，秒级。
2. 资源占用小，比 VM 省资源。
3. 环境一致，打包镜像，一次构建，到处运行。
4. 快速迭代，部署快。

缺点：

5. 隔离性不如 VM。
6. 只能运行 Linux 的容器，Windows 的差点。
7. 安全比 VM 差点，因为共享内核。

容器化和 VM 的对比是什么？

就是上面的，容器轻量，快，资源省，隔离差点，VM 重，慢，隔离好。

其他

Docker 日志保存在什么地方？

答案：

默认存在 /var/lib/docker/containers/ 容器 ID / 容器 ID-json.log，这个文件里。

一体机开机自启，Docker 怎么实现自动启动？容器启动后怎么启动服务？

答案：

Docker 的守护进程，设置开机自启，`systemctl enable docker`，然后容器的话，启动的时候，加 `--restart=always`，这样容器就会开机自动启动，Docker 启动了，就会启动容器，容器启动了，里面的服务就启动了。

Docker 的原理有了解过吗？

答案：

Docker 的原理是用了 Linux 的 Namespace 和 Cgroups，Namespace 做隔离，把进程的资源隔离，比如 PID、网络、Mount，让容器以为自己是独立的系统，Cgroups 做资源限制，限制容器的 CPU、内存的使用，然后联合文件系统，做镜像的分层，这样就实现了容器。

组件与架构

K8S 组件有哪些？各自的作用是什么？

答案：

Master 节点的组件：

1. **kube-apiserver**：集群的入口，所有的操作都通过它，提供 REST API，认证、授权。
2. **etcd**：分布式的键值存储，保存集群的所有数据，比如节点、Pod、Service 的信息。
3. **kube-scheduler**：调度器，负责把新的 Pod 调度到合适的 Node 节点。
4. **kube-controller-manager**：运行各种控制器，比如 Node 控制器、Pod 控制器，保证集群的状态和预期一致。
5. **cloud-controller-manager**：云平台的控制器，对接云厂商的 API。

Node 节点的组件：

1. **kubelet**：每个 Node 上的代理，负责管理 Node 上的 Pod，和 Master 通信，保证 Pod 运行。
2. **kube-proxy**：每个 Node 上的网络代理，负责 Service 的网络，把 Service 的 IP 转发到 Pod，做负载均衡。
3. **容器 runtime**：比如 Docker、containerd，运行容器。

K8S 的架构是怎样的？

答案：

K8S 是主从架构，Master 节点负责管理集群，Node 节点负责运行工作负载，Master 的 apiserver 是入口，所有的组件都和 apiserver 通信，etcd 存储数据，scheduler 调度，controller 管理控制器，Node 上的 kubelet 管理 Pod，kube-proxy 管理网络。

K8S 和容器什么关系？

答案：

容器是运行应用的，K8S 是编排容器的，管理容器的部署、扩展、生命周期，容器是 K8S 管理的对象。

K8S 的最小部署单元是什么？

答案：

Pod，Pod 是 K8S 的最小单元，一个 Pod 里可以有多个容器，共享网络和存储。

kube-proxy 的模式有哪些？(iptables、IPVS、userspace)

答案：

1. **userspace**：最早的，用户空间的代理，性能差，现在不用了。
2. **iptables**：默认的，用 iptables 的规则，转发流量，性能不错，但是大规模的话，规则太多，性能差。

3. **IPVS**：用 IPVS 的负载均衡，性能更高，支持更多的 Service，大规模集群用这个。

kube-apiserver 高可用是如何实现的？

答案：

部署多个 kube-apiserver，前面用负载均衡，把请求分发到多个 apiserver，因为 apiserver 是无状态的，所以可以水平扩展，然后 etcd 做集群，保证高可用。

etcd 的快照恢复流程是什么？

答案：

1. 先停止 apiserver，避免写入。
2. 然后用 `etcdctl snapshot save`，备份快照。
3. 恢复的话，停止所有的 etcd 节点，然后用 `etcdctl snapshot restore`，恢复快照，生成新的数据目录。
4. 然后启动 etcd，然后启动 apiserver。

etcd 如何保证高可用？

答案：

etcd 是 Raft 协议的集群，部署多个节点，一般 3 个或者 5 个，只要大部分节点活着，集群就正常，比如 3 节点的话，最多挂 1 个，5 节点的话，最多挂 2 个，保证高可用。

scheduler 底层的调度算法有哪些？

答案：

调度的过程是，先过滤节点，排除不符合的，比如资源不够的，然后打分，给剩下的节点打分，选分数最高的，打分的算法有很多，比如优先调度，比如资源的剩余，有没有亲和性，有没有负载，比如 `LeastRequestedPriority`，优先调度到资源剩余多的节点。

K8S 你负责的集群都是自建的吗？

答案：

是的，我们是自建的 K8S 集群，用 `kubeadm` 搭建的，自己维护。

K8S 有哪些优缺点？

答案：

优点：

1. 自动部署、扩展、管理容器，自动化运维。
2. 自愈能力，Pod 挂了自动重启，节点挂了，Pod 自动调度到其他节点。
3. 滚动更新，零停机更新。
4. 自动扩缩容，根据负载自动调整 Pod 的数量。

5. 服务发现和负载均衡，内置的。

缺点：

6. 复杂度高，学习成本高，运维难度大。

7. 有一定的性能损耗，比直接部署差点。

8. 小项目的话，太重了，没必要。

Pod 生命周期与状态

Pod 的生命周期是什么？具体过程是怎样的？

答案：

Pod 的生命周期：

1. Pending：Pod 创建了，但是还没调度到节点，或者容器还在创建。
2. Running：Pod 的所有容器都启动了，运行中。
3. Succeeded：所有的容器都正常退出了，完成了，一次性的任务。
4. Failed：所有的容器都退出了，而且有异常退出的。
5. Unknown：不知道 Pod 的状态，可能网络不通。

Pod 有几种状态？分别代表什么意思？(Pending、Running、Succeeded、Failed、Unknown)

就是上面的。

如果 Pod 的状态是 Pending, 一般有哪些原因？

答案：

1. 资源不够，节点的 CPU、内存不够，调度不了。
2. 镜像拉取失败，镜像不存在，或者权限不够。
3. 节点不可用，节点挂了。
4. 调度失败，比如亲和性的问题，调度不到节点。
5. 卷挂载失败，比如 PVC 绑定不了。

Pod 一直重启什么原因导致的？

答案：

1. 容器的主进程退出了，异常退出，比如程序崩溃。
2. 健康检查失败，存活探针失败，kubelet 重启了容器。
3. 资源不够，OOM，内存不够，内核杀死了进程。
4. 镜像有问题，启动命令错了，容器启动就退出。

5. 节点的问题，比如节点的资源不够。

Pod 一直处于 CrashLoopBackOff 状态怎么排查？

答案：

1. 看 Pod 的日志，`kubectl logs pod名`，看容器的日志，有没有错误。
2. 看 Pod 的事件，`kubectl describe pod`，看事件，有没有什么错误，比如 OOM。
3. 看重启的原因，是不是探针失败了，还是进程退出了。
4. 手动运行镜像，看能不能启动，是不是启动命令错了。

Pod 状态为 Error 怎么排查？

答案：

看 describe pod，看事件，看日志，看是不是容器启动失败了，比如配置错了，或者卷挂载错了。

怎么查看上一个 Pod 的启动日志？

答案：

`kubectl logs pod名 --previous`，查看上一个容器的日志。

说一下 Pod 的重启策略 (Always、OnFailure、Never)

答案：

- Always：总是重启，只要容器退出了，就重启，默认的。
- OnFailure：只有容器异常退出，退出码非 0，才重启，正常退出的话，不重启。
- Never：从不重启，不管什么情况。

删除 Pod 会发生什么事情？

答案：

如果 Pod 是被控制器管理的，比如 Deployment，那么控制器会新建一个 Pod，代替它，因为控制器要保证副本数，所以删除 Pod，会自动新建一个。

如果是自主的 Pod，没有控制器，那么就直接删除了，不会新建。

什么是静态 Pod、自主式 Pod、动态 Pod？

答案：

- **静态 Pod**：是 kubelet 直接管理的，不用 apiserver，配置文件放在 Node 的 `/etc/kubernetes/manifests/`，kubelet 会自动加载，一般用来部署集群的组件，比如 etcd、apiserver。
- **自主式 Pod**：自己创建的，没有控制器的 Pod，删除了就没了，不会重建。
- **动态 Pod**：被控制器管理的 Pod，比如 Deployment、StatefulSet 创建的，控制器会管理它们，保证副本数。

Pod 地址是唯一的吗？Pod 和 Pod 之间共享资源吗？

答案：

Pod 的 IP 是唯一的，每个 Pod 有自己的 IP，同一个 Pod 里的容器，共享网络和存储，共享 PID namespace，所以它们可以用localhost通信，共享卷，不同的 Pod 之间，不共享资源，除非用共享卷。

Pod 的创建流程是怎样的？各组件都做了哪些工作？

答案：

1. 用户创建 Pod，请求发给 apiserver。
2. apiserver 把 Pod 的信息写到 etcd。
3. scheduler 发现有新的 Pod，没调度，然后调度，选一个节点，更新 Pod 的节点信息，写到 etcd。
4. 那个节点的 kubelet，发现有新的 Pod，然后创建容器，拉取镜像，启动容器。
5. kubelet 更新 Pod 的状态，写到 etcd。

Service

K8S 的 Service 是干什么用的？

答案：

Service 是用来抽象 Pod 的访问，因为 Pod 会变，IP 会变，Service 给 Pod 一个固定的 IP，然后把请求负载均衡到后端的 Pod，同时做服务发现，这样客户端不用管 Pod 的 IP，直接访问 Service 的 IP 就可以了。

Service 有几种类型？(ClusterIP、NodePort、LoadBalancer、ExternalName)

答案：

1. **ClusterIP**：默认的，集群内部的 IP，只有集群内部可以访问，集群内的服务之间访问。
2. **NodePort**：在每个 Node 上开一个端口，外部可以通过 NodeIP:NodePort 访问服务。
3. **LoadBalancer**：云厂商的负载均衡，Service 会得到一个外部的 IP，外部可以直接访问，云厂商的负载均衡把请求转发到 Service。
4. **ExternalName**：把 Service 映射到外部的域名，比如 CNAME。

ClusterIP 和 NodePort 是什么关系？

答案：

NodePort 是在 ClusterIP 的基础上，在每个 Node 上开一个端口，外部可以访问，ClusterIP 是内部的，NodePort 是外部的，NodePort 的 Service，也有 ClusterIP，内部可以用 ClusterIP 访问，外部可以用 NodePort。

Service 无法访问其后端 Pod, 该如何排查?

答案:

1. 先看 Service 的 endpoints, `kubectl get endpoints service名`, 看有没有后端的 Pod 的 IP, 没有的话, 说明 Service 的 selector 没匹配到 Pod。
2. 看 Pod 的状态, 是不是 Running, Pod 的端口是不是对的, 是不是监听了。
3. 看 Pod 之间能不能通, 能不能访问 Pod 的 IP: 端口。
4. 看 kube-proxy 是不是正常, 有没有生成 iptables 的规则。
5. 看网络插件是不是正常, Pod 之间的网络通不通。

通过 NodePort 暴露服务, 客户通过 IP 加端口访问, 怎么获得客户的真实 IP?

答案:

需要配置 Service 的 externalTrafficPolicy 为 Local, 这样的话, 请求的源 IP 就会保留, 不会被 SNAT, 这样后端的 Pod 就能拿到客户端的真实 IP, 但是如果 Node 上没有 Pod 的话, 请求就会失败。

Stateful Service 和普通的 Service 有什么区别?

答案:

Stateful Service 是给 StatefulSet 用的, Headless Service, 也就是 ClusterIP 是 None, 没有负载均衡, 会返回所有的 Pod 的 IP, 这样客户端可以自己发现所有的 Pod, 适合有状态的服务, 比如数据库, 需要知道每个 Pod 的地址。

Deployment 与控制器

Deployment 和 StatefulSets 的区别是什么?

答案:

- Deployment: 用来部署无状态的服务, Pod 是无状态的, 没有顺序, 名字是随机的, 重启之后, 数据没了, 适合 Web 服务, 无状态的。
- StatefulSet: 用来部署有状态的服务, Pod 是有顺序的, 名字是固定的, 有稳定的存储, 稳定的网络, 启动和更新是按顺序的, 适合数据库、中间件, 有状态的。

Deployment、DaemonSet、StatefulSet 之间的区别和特性是什么?

答案:

- **Deployment:** 无状态, 管理 Pod 的副本, 滚动更新, 适合无状态的服务。
- **DemonSet:** 每个 Node 上都运行一个 Pod, 新的 Node 加入, 自动加一个, 适合日志采集、监控代理, 每个节点都要的。
- **StatefulSet:** 有状态, 稳定的网络、存储, 顺序, 适合有状态的服务。

Deployment 的滚动更新是怎么配置的？

答案：

在 Deployment 的配置里，配置 strategy：

代码块

```
1  strategy:
2    rollingUpdate:
3      maxSurge: 1
4      maxUnavailable: 1
5    type: RollingUpdate
```

maxSurge 是最多可以超出多少个副本，maxUnavailable 是最多可以有多少个不可用，这样更新的时候，先启动新的 Pod，然后停旧的，逐步更新，零停机。

DaemonSet 类型的资源特性是什么？

答案：

每个 Node 上运行一个 Pod，所有的 Node，或者匹配的 Node，都会有一个，新的 Node 加入，自动创建，Node 删除，Pod 也删除，适合每个节点都要运行的服务，比如日志采集、监控、网络插件。

现在有一个需求想使用 Deployment 的模式达到 DaemonSet 的效果，该怎么实现？

答案：

可以给 Pod 做节点亲和，或者用拓扑 Spread，或者把 replicas 设置成节点的数量，然后调度到每个节点一个，但是不如 DaemonSet 方便，因为节点加了，Deployment 不会自动加副本。

Deployment 的 YAML 文件怎么编写？有哪些关键字段？

答案：

apiVersion、kind、metadata、spec，spec 里有 replicas、selector、template、strategy，template 里是 Pod 的模板，metadata 和 spec。

容器控制器如果要在每个节点部署 Pod 用什么控制器？

DaemonSet。

docker 控制器有哪些？各自的作用？

不对，是 K8S 的控制器，Deployment、DaemonSet、StatefulSet、Job、CronJob 这些。

故障排查

K8S 中容器出现故障怎么排查？排查流程？

答案：

1. 先看 Pod 的状态，`kubectl get pod`，看是什么状态，Pending 还是 Crash。

2. 然后看 Pod 的事件，`kubectl describe pod`，看事件里的错误，比如镜像拉取失败，还是 OOM。
3. 然后看 Pod 的日志，`kubectl logs pod`，看容器的日志，有没有错误。
4. 然后进入容器，`kubectl exec`，看里面的进程是不是正常。
5. 然后看节点的状态，节点的资源是不是够，是不是节点出问题了。

K8S 中遇见的网络故障有哪些？

答案：

Pod 之间网络不通，Service 访问不通，CNI 的问题，Flannel 的问题，比如网络插件挂了，Pod 没有 IP，或者跨节点的 Pod 不通。

K8S 集群中 Node 节点状态为 NotReady 怎么排查？

答案：

1. 先 ssh 到节点，看 kubelet 是不是正常运行，`systemctl status kubelet`。
2. 看 kubelet 的日志，`journalctl -u kubelet`，看有没有错误。
3. 看节点的资源，是不是磁盘满了，内存满了，导致 kubelet 无法工作。
4. 看网络是不是通的，节点能不能访问 apiserver。
5. 看 docker 是不是正常，容器运行时是不是正常。

K8S 集群前面负载的架构是什么？

答案：

前面用 Ingress Controller，比如 Nginx Ingress，然后前面用 SLB，把请求转发到 Ingress Controller，然后 Ingress 把请求转发到后端的 Service。

K8S 中 Service 访问不通排查用什么命令？

答案：

`kubectl get endpoints`，看 endpoints，然后 `curl PodIP:端口`，看 Pod 通不通，然后 `telnet`，看端口，然后看 kube-proxy 的日志。

维护 K8S 集群的时候都遇到过哪些问题？怎么解决的？

答案：

遇到过节点磁盘满了，导致 Pod 无法启动，然后清理日志，扩容磁盘，还有 Pod 的 CrashLoopBackOff，排查是程序的问题，还有节点 NotReady，kubelet 挂了，重启 kubelet，还有镜像拉取失败，配置 Harbor 的证书。

K8S 突发故障怎么处理？

答案：

先恢复业务，比如把 Pod 调度到其他节点，或者重启 Pod，然后排查原因，解决问题，然后复盘。

K8S 集群中 Pod 出故障不会转移到其他节点吗？

答案：

如果节点挂了，那么 Pod 会被驱逐，转移到其他节点，如果只是 Pod 出故障，节点是好的，那么会在本节点重启 Pod，不会转移，除非你配置了 Pod 的驱逐。

Pod 拉取不动镜像要怎么排查？

答案：

1. 看镜像是不是存在，是不是名字错了。
2. 看节点能不能访问镜像仓库，能不能 ping 通。
3. 看是不是需要认证，节点有没有登录镜像仓库，Secret 是不是配置对了。
4. 看镜像的架构，是不是和节点的架构不一样，比如 ARM 的镜像，x86 的节点拉了。
5. 看网络是不是有问题，防火墙是不是拦截了。

K8S 各组件及作用是什么？

就是之前的组件的作用。

现在有一个系统访问缓慢，排查思路是什么？

答案：

1. 先看 Pod 的资源，CPU、内存是不是够，是不是负载高。
2. 看数据库，是不是慢 SQL，数据库的负载高。
3. 看网络，是不是网络延迟高，丢包。
4. 看监控，看 QPS 是不是太高，是不是需要扩容。
5. 看日志，有没有错误，有没有慢请求。

如果磁盘 IO 压力过大，应该如何排查？

答案：

用 iostat，看磁盘的 IO，然后用 iotop，看哪个进程的 IO 高，然后看是不是容器的日志，是不是容器的读写太多，然后优化，比如把日志写到外部的存储，或者扩容磁盘。

探针

K8S 探针用过吗？了解探针吗？

用过，探针是 kubelet 用来检查容器的状态的，保证容器正常。

K8S 探针类型有哪些？(livenessProbe、readinessProbe、startupProbe)

答案：

1. **livenessProbe**：存活探针，检查容器是不是活着，如果失败了，kubelet 就会重启容器，用来检测程序是不是死锁了，卡住了。
2. **readinessProbe**：就绪探针，检查容器是不是可以处理请求，如果失败了，就会把 Pod 从 Service 的 endpoints 里移除，不会把请求转发给它，用来检测程序是不是启动好了，能不能处理请求。
3. **startupProbe**：启动探针，检查容器是不是启动完成，启动的时候，会禁用存活和就绪探针，直到启动完成，避免启动慢的程序，被存活探针杀死。

探针的探测方法有哪些？(httpGet、exec、tcpSocket)

答案：

1. **httpGet**：发送 HTTP 请求，如果返回 2xx-3xx，就成功，适合 Web 服务。
2. **exec**：执行一个命令，如果命令的退出码是 0，就成功，适合自定义的检查。
3. **tcpSocket**：检查 TCP 端口是不是通的，如果通，就成功，适合 TCP 的服务。

健康检查怎么写的？是用的探针请求接口吗？

答案：

是的，一般用 httpGet，请求程序的 /health 接口，程序返回 200，说明健康，比如：

代码块

```
1  livenessProbe:
2    httpGet:
3      path: /health
4      port: 8080
5    initialDelaySeconds: 5
6    periodSeconds: 10
7  readinessProbe:
8    httpGet:
9      path: /ready
10     port: 8080
11    initialDelaySeconds: 5
12    periodSeconds: 10
```

存活探针是由哪个服务发起请求的？

答案：

是 Node 上的 kubelet 发起的，kubelet 定期检查，不是 Master 的组件。

PV/PVC

PV 和 PVC 是什么？

答案：

- PV: PersistentVolume，是集群里的存储，管理员创建的，比如 NFS、云盘，是存储的抽象。
- PVC: PersistentVolumeClaim，是用户的存储请求，用户创建 PVC，请求存储，然后 K8S 会把 PVC 和合适的 PV 绑定，然后 Pod 就可以用这个 PVC，挂载到容器里，用户不用关心存储的细节。

PV 的生命周期状态有哪些？(Available、Bound、Released、Failed)

答案：

- Available: PV 可用，还没绑定。
- Bound: PV 已经和 PVC 绑定了。
- Released: PVC 删除了，PV 的资源被释放了，但是还没回收。
- Failed: PV 绑定失败了。

StorageClass 的作用是什么？

答案：

StorageClass 是用来动态创建 PV 的，不用管理员手动创建 PV，用户创建 PVC 的时候，指定 StorageClass，K8S 就会根据 StorageClass 的配置，自动创建 PV，比如云盘的 StorageClass，自动创建云盘，这样就实现了动态存储，不用手动管理 PV。

Ingress

K8S 中 Ingress 和普通 Nginx 有什么区别？

答案：

普通的 Nginx 是自己部署的，Ingress 是 K8S 的 Ingress Controller，比如 Nginx Ingress，是集成了 K8S 的，自动监听 Ingress 的资源，自动更新 Nginx 的配置，不用手动改 Nginx 的配置，当 Service 和 Pod 变化的时候，自动更新，很方便，而普通的 Nginx，需要手动改配置。

请结合业务描述 Nginx Ingress 的工作原理。

答案：

Nginx Ingress Controller，会监听 K8S 的 Ingress 资源，然后把 Ingress 的配置，转换成 Nginx 的配置文件，然后 reload Nginx，这样就自动配置了反向代理，当用户访问域名的时候，Ingress Controller 根据 Ingress 的规则，把请求转发到对应的 Service，然后 Service 转发到 Pod，这样就不用手动配置 Nginx 了，自动的，比如我们的业务，每个服务有自己的 Ingress，配置域名和路径，Ingress Controller 自动更新，很方便。

你了解 Ingress Controller 的 Webhook 或 Admission Controller 吗？

答案：

了解，Admission Controller 是 K8S 的准入控制器，在资源创建的时候，拦截请求，验证或者修改，Ingress 的 Webhook，就是在创建 Ingress 的时候，验证 Ingress 的配置是不是合法，比如域名是不是重复，配置是不是对的，避免错误的配置。

镜像与配置

K8S 的镜像拉取策略有哪些？(Always、IfNotPresent、Never)

答案：

- Always：总是去拉取镜像，不管本地有没有。
- IfNotPresent：如果本地有，就用本地的，没有的话，才拉取，默认的。
- Never：从不拉取，只用本地的，适合离线的环境。

K8S 中如何给节点打污点？

答案：

`kubectl taint nodes 节点名 key=value:effect`，比如 `kubectl taint nodes node1 key=value:NoSchedule`，这样 Pod 就不会调度到这个节点，除非 Pod 有容忍。

K8S 中需要将 Pod 中的文件拉取到本地目录，用什么命令？

答案：

`kubectl cp pod名:容器内的路径 本地路径`，比如 `kubectl cp pod:/app/log ./log`。

configmap 的作用是什么？

答案：

ConfigMap 是用来存储配置的，把配置和镜像分开，不用把配置写到镜像里，然后 Pod 可以挂载 ConfigMap，或者用环境变量，把配置注入到容器里，这样配置可以动态的改，不用重新构建镜像。

资源清单里的 kind 字段有哪些类型？

答案：

比如 Deployment、Service、Pod、ConfigMap、Secret、Ingress、StatefulSet、DaemonSet、Job、CronJob 这些，不同的 kind，代表不同的资源类型。

网络插件

K8S 中的网络插件有哪些？(Flannel、Calico、Canal、Weave Net、Cilium)

答案：

- Flannel：简单，Overlay 网络，配置简单，适合小集群。
- Calico：BGP 的，三层网络，性能好，支持网络策略，安全，适合大集群。
- Canal：Flannel+Calico，用 Flannel 做网络，Calico 做网络策略。

- Weave：简单，自动组网，不用配置。
- Cilium：eBPF 的，性能高，安全，支持很多高级功能。

Flannel 的几种模式是什么？(VXLAN、Host-GW、UDP)

答案：

- VXLAN：Overlay 网络，封装成 UDP，跨节点，兼容性好，但是有一点性能损耗。
- Host-GW：直接路由，不用封装，性能高，但是要求节点在同一个二层网络。
- UDP：最早的，性能差，现在不用了。

为什么选择 Flannel 作为 CNI 插件？和 Calico 相比有何考虑？

答案：

Flannel 简单，容易部署，运维简单，我们的集群不大，不需要 Calico 的网络策略，所以用 Flannel，足够用了，简单稳定。

调度与资源

Pod 的调度依据是什么？具体的调度机制 (驱逐机制) 是什么？

答案：

调度的依据是，节点的资源，有没有足够的 CPU、内存，然后亲和性，节点亲和、Pod 亲和，然后污点和容忍，然后调度算法，过滤，然后打分，选最好的节点。

驱逐机制是，当节点的资源不够的时候，kubelet 会驱逐一些 Pod，腾出资源，保证节点的稳定，驱逐的话，根据 Pod 的优先级，先驱逐优先级低的。

K8S 中 Pod 调度不均匀怎么办？

答案：

1. 检查是不是有节点的污点，导致 Pod 调度不到。
2. 检查是不是亲和性的问题，导致 Pod 都调度到某些节点。
3. 用 descheduler，把不均衡的 Pod，重新调度，平衡节点的负载。
4. 调整调度的算法，让调度更均衡。

K8S 怎么节省资源？

答案：

1. 调整 Pod 的 Request 和 Limit，合理的设置，不要设置太大，浪费资源。
2. 做 Pod 的优先级，重要的 Pod 优先调度。
3. 做超配，因为 Pod 不会一直用满资源，所以可以超配，提升资源利用率。
4. 自动扩缩容，根据负载，自动调整 Pod 的数量，闲的时候缩容。

5. 驱逐闲置的 Pod，回收资源。

Request 和 Limit 这两个值会设置成一样吗？

答案：

一般不会，Request 是调度的时候用的，是 Pod 最少需要的资源，Limit 是最大的资源，限制 Pod 不能用太多，所以 Request 比 Limit 小，比如 CPU 的 Request 是 1 核，Limit 是 2 核，这样调度的时候，给它留 1 核，但是它最多可以用 2 核，突发的时候可以用，但是如果设置成一样的话，就是刚性的，最多用这么多，突发的时候就不够了。

YAML 文件一般都会定义哪些主要的参数？

答案：

apiVersion、kind、metadata (name、namespace、labels) 、spec，然后根据资源的不同，spec 的内容不同，比如 Deployment 的 spec 有 replicas、selector、template，Pod 的 spec 有 containers、volumes、nodeSelector 这些。

Pod 亲和是什么？

答案：

Pod 亲和性，是把 Pod 调度到和某个 Pod 同一个节点，或者同一个拓扑域，比如两个服务需要在一起，网络延迟低，就用 Pod 亲和，把它们调度到同一个节点，或者同一个可用区。

反亲和的话，就是把 Pod 调度到不同的节点，避免单点，比如同一个服务的 Pod，调度到不同的节点，这样一个节点挂了，不会所有的 Pod 都挂了。

HPA

HPA 了解过吗？HPA 底层的逻辑怎么实现的？底层的原理是什么？

答案：

HPA 是 Horizontal Pod Autoscaler，Pod 的水平扩缩容，根据 Pod 的 CPU、内存，或者自定义的指标，自动调整 Pod 的副本数，比如 CPU 超过 80%，就加 Pod，低于，就减。

底层的原理是，HPA 控制器，定期获取 Pod 的指标，然后和目标值对比，计算需要的副本数，然后更新 Deployment 的副本数，实现自动扩缩容。

同一个服务，想增加 Pod 的数量怎么做？

答案：

可以手动改 Deployment 的 replicas，`kubectl scale deployment --replicas=3`，或者用 HPA，自动扩缩容。

日志采集

K8S 上 Pod 的工作日志如何管理？

答案：

用 ELK，或者 EFK，Filebeat 在每个节点上，采集 Pod 的日志，然后发到 Elasticsearch，然后 Kibana 展示，这样日志就集中管理了，不用到每个节点上看。

如果 Pod 漂移到了其他节点日志是如何采集的？

答案：

因为 Filebeat 是在节点上的，不管 Pod 漂移到哪个节点，那个节点的 Filebeat 都会采集它的日志，所以没问题，日志的元信息会带 Pod 的名字，所以可以统一管理。

怎么采集 K8S 日志？

就是上面的，EFK，Filebeat 采集，然后存到 ES。

部署与迁移

K8S 上都做过什么实际项目？

答案：

做过业务的容器化迁移，把原来的物理机的业务，迁移到 K8S 上，然后做了 CI/CD，自动部署，还有监控的搭建，日志的采集，还有集群的维护，升级。

你们现在 K8S 上项目是怎么部署的？

答案：

用 CI/CD，开发提交代码，Jenkins 自动构建镜像，然后更新 Deployment 的镜像，然后 K8S 滚动更新，自动部署，不用手动操作。

从物理机迁移到 K8S 后，在服务交付、CI/CD 和监控方案上有哪些具体差异？

答案：

- 服务交付：原来的物理机，部署慢，现在 K8S，部署快，滚动更新，零停机，交付快。
- CI/CD：原来的部署是脚本，现在 K8S 的 CI/CD，自动构建镜像，自动部署，更自动化。
- 监控：原来的是节点的监控，现在是 Pod 的监控，还有 K8S 的组件的监控，更全面，而且日志集中管理。

环境迁移到 K8S 集群怎么做的？

答案：

先把应用容器化，制作镜像，然后编写 K8S 的资源清单，Deployment、Service、Ingress，然后测试环境部署，验证，然后生产环境，先迁移一部分，验证，然后全量迁移，同时把数据也迁移过去，然后切换流量。

如果想从 docker-compose 业务迁移到 K8S 集群怎么去做？

答案：

先把 docker-compose 的服务，转换成 K8S 的资源，比如每个服务，做成 Deployment，然后卷转换成 PVC，网络转换成 Service，然后把配置转换成 ConfigMap，然后部署到 K8S，测试，验证。

K8S 迁移应该注意哪些问题？比如说正在运行的业务？

答案：

要注意数据的迁移，不要丢数据，然后要灰度迁移，先迁一部分，验证，然后全量，还要注意流量的切换，不要中断业务，还要注意兼容性，应用在 K8S 上能不能正常运行，有没有权限的问题，存储的问题。

假设现在业务基于 K8S 环境运行，要把业务迁到 K8S 环境，思路是什么？

不对，应该是从其他环境迁到 K8S，思路就是容器化，然后编写资源，测试，灰度迁移，切换流量。

假设业务在线下的 K8S 环境运行，怎么把 K8S 这套业务迁移到云上？

答案：

把镜像同步到云上的镜像仓库，然后把资源清单导出，在云上的 K8S 创建，然后把数据迁移过去，然后切换 DNS，把流量切到云上。

有一个 Java 代码怎么部署到 K8S 集群？

答案：

1. 把 Java 代码构建成为 Jar 包，然后制作 Docker 镜像，把 Jar 包放到镜像里。
2. 编写 Deployment 的 YAML，配置容器，端口，资源，探针。
3. 编写 Service，暴露服务。
4. 编写 Ingress，配置域名。
5. 然后 `kubectl apply`，部署到 K8S。

Java 服务迁移到 K8S 后，编写了哪些 K8S 资源清单 (YAML)？

答案：

Deployment、Service、Ingress、ConfigMap、Secret、PVC，这些，配置应用的部署，服务，配置，存储。

部署一个 Java 服务，它在容器里不同状态对应的服务状态是怎样的？

答案：

Pod 的状态，Pending 的时候，还在创建，Running 的时候，启动了，就绪探针成功了，就可以处理请求，然后如果探针失败了，就会重启，或者从 Service 移除。

在什么场景下你会使用 `kubectl describe pod`？又在什么情况下使用 `kubectl logs`？

答案：

- `kubectl describe pod`: 看 Pod 的事件, 看 Pod 的状态, 比如 Pending 的时候, 看为什么调度不了, 看事件里的错误, 比如镜像拉取失败, 资源不够。
 - `kubectl logs`: 看容器的日志, 看程序的输出, 比如 CrashLoopBackOff 的时候, 看程序的日志, 为什么退出了, 有没有错误。
-

CI/CD

Jenkins

Jenkins 部署业务服务是单节点还是集群的?

答案:

我们是单节点的, 因为规模不大, 足够用了, 也可以用集群, 主从, 分布式构建。

Jenkins 都是先部署在内部服务器还是直接上线?

答案:

部署在内部的, CI/CD 的服务, 内部用, 不对外。

Jenkins 工作流程是什么?

答案:

1. 开发提交代码到 GitLab。
2. GitLab 触发 Jenkins 的 Webhook。
3. Jenkins 拉取代码。
4. 构建, 编译代码, 构建 Docker 镜像。
5. 推送镜像到 Harbor。
6. 更新 K8S 的 Deployment, 滚动更新。
7. 完成, 通知开发。

Jenkins 如何实现代码的回滚? 具体是配了什么?

答案:

记录每次构建的镜像版本, 然后回滚的时候, 把 Deployment 的镜像改回之前的版本, 然后 K8S 滚动回滚, 或者用 K8S 的 rollout, `kubectl rollout undo deployment`, 回滚到上一个版本。

Jenkinsfile 如何安全传输像密码、安全地址这种敏感信息?

答案:

用 Jenkins 的 Credentials, 把密码存在 Jenkins 的凭证里, 然后在 Jenkinsfile 里, 用凭证的 ID 引用, 不会明文显示, 安全。

Jenkins 的多分支并行构建怎么做？

答案：

用多分支流水线，每个分支自动创建流水线，然后并行构建，或者在流水线里，用 parallel，并行执行多个步骤。

Jenkins 构建失败之后你们是怎么做的？

答案：

通知开发，告诉构建失败了，然后看日志，排查原因，解决问题，然后重新构建。

如果给你一台空白的 Jenkins 机器，如何从零搭建？

答案：

1. 安装 JDK，因为 Jenkins 需要 Java。
2. 安装 Jenkins，配置 yum 源，安装 Jenkins 包。
3. 启动 Jenkins，配置端口，然后初始化，输入管理员密码。
4. 安装需要的插件，比如 Git、Docker、Kubernetes、Pipeline。
5. 配置凭证，配置 Git 的凭证，Harbor 的凭证。
6. 配置节点，如果是集群的话，配置从节点。
7. 创建流水线任务，配置 Jenkinsfile。

使用哪一种 Jenkins 方式实现自动化流程？

用 Pipeline as Code，用 Jenkinsfile，把流水线的代码写到代码仓库里，和代码一起版本管理。

Jenkins 中涉及到和 GitLab 的一个认证，怎么认证的？

答案：

用 SSH 密钥，Jenkins 生成 SSH 密钥，然后把公钥加到 GitLab 的部署密钥里，这样 Jenkins 就可以拉取代码了，或者用账号密码，或者 Token。

GitLab

GitLab 的钩子函数怎么配？这个东西是哪里面的？

答案：

GitLab 的 Webhook，在项目的设置里，配置 Jenkins 的地址，当代码提交的时候，GitLab 就会发送请求到 Jenkins，触发构建，这个是 GitLab 的功能，用来触发事件。

发布流程

CI/CD 流程是怎么样子的？

就是上面的，代码提交，触发构建，构建镜像，推送，部署。

滚动更新如何实现？

K8S 的 Deployment 的滚动更新，先启动新的 Pod，然后停旧的，逐步更新。

业务更新的时候，保障服务的持续稳定的运作有什么方式吗？

答案：

用滚动更新，零停机，同时做好回滚的准备，如果有问题，立刻回滚，还有灰度发布，先更一部分，验证没问题，再全量。

蓝绿发布和金丝雀发布如何在 K8S 上落地？各自有什么优缺点？

答案：

- **蓝绿发布**：部署两套环境，蓝是旧的，绿是新的，都部署好，然后切换流量，优点是切换快，回滚快，缺点是需要双倍的资源。
- **金丝雀发布**：先发布一小部分，给少量用户，验证没问题，然后全量，优点是节省资源，风险小，缺点是切换慢，需要流量的拆分。

在 K8S 上，蓝绿的话，就是两个 Deployment，切换 Service 的 selector，金丝雀的话，用 Ingress 的权重，或者 Service 的分流，把一部分流量分到新的版本。

你们是多久更新一次？发布一次新版本？

答案：

一般是一周一次，或者两周，小的话，随时可以发布。

你们滚动更新，Server 怎么去挂的容器？

不对，滚动更新的时候，K8S 先启动新的容器，然后把旧的停掉，流量切到新的。

如何控制发布的 Pod？

用 Deployment 的 maxSurge 和 maxUnavailable，控制更新的速度，比如一次更 1 个，或者 25%。

CICD 执行失败了，但有些配置已经生效了，怎么进行恢复？

答案：

回滚，把配置改回之前的，然后 K8S rollout undo，回滚到上一个版本，恢复到之前的状态。

灰度发布有哪些方式？

答案：

1. 按权重，比如 10% 的流量到新的版本。
2. 按用户，比如部分用户用新的。
3. 按时间，先小部分，然后逐步加。

在 CI/CD 流程中，如何使用 Jenkins 与 Docker 集成？

答案：

Jenkins 里安装 Docker 插件，然后在流水线里，执行 docker build，构建镜像，然后 docker push，推送到仓库。

打包镜像上传到仓库的动作详细流程是什么？

答案：

1. 拉取代码。
2. 编译代码，得到二进制。
3. 构建 Docker 镜像，docker build -t 镜像名。
4. 登录 Harbor，docker login。
5. 推送镜像，docker push 镜像名。

DevOps 中的 CI 和 CD 分别是做什么用的，优点是什么？

答案：

- CI：持续集成，开发的代码，持续的合并到主干，自动构建、测试，发现问题，优点是快速发现问题，减少集成的成本。
- CD：持续交付 / 部署，持续的把代码部署到生产环境，自动的，优点是快速交付，减少手动操作，提升效率，减少错误。

代码上线流程是什么？会遇到什么问题？

答案：

开发提交代码，CI 构建，测试，然后部署到测试环境，测试，然后部署到生产，遇到的问题，比如构建失败，测试失败，部署失败，回滚的问题。

监控

Prometheus

Prometheus 监控过什么？是怎么实现的？

答案：

监控过系统的指标，Node 的 CPU、内存、磁盘，还有 K8S 的组件的指标，还有业务的指标，还有中间件的，比如 MySQL、Redis、Nginx 的指标，通过 exporter，比如 node_exporter，收集节点的指标，mysqld_exporter 收集 MySQL 的，然后 Prometheus 拉取这些指标，存储起来。

监控数据的采集和实现是什么？

用 exporter，每个服务的 exporter，暴露指标，Prometheus 用 pull 的方式，定期拉取指标，然后存储。

Prometheus 架构是什么？监控了什么？怎么安装部署？

答案：

Prometheus 的架构：

- Prometheus Server：拉取指标，存储，查询。
- Exporter：暴露指标的组件。
- AlertManager：告警管理，把告警发到邮箱、钉钉。
- Grafana：展示指标，做 Dashboard。

监控了 Node、K8S、MySQL、Redis、Nginx、业务的指标，部署的话，用 Docker，或者 K8S 的 Operator，部署 Prometheus Operator。

Prometheus 如何自定义监控 Java 项目？

答案：

Java 项目用 Prometheus 的客户端，埋点，暴露指标，比如用 micrometer，然后 Prometheus 拉取这些指标，就可以监控 Java 项目的自定义指标了。

如何在 K8S 集群外部搭建 Prometheus 对集群进行监控？

答案：

把 K8S 的 APIServer 的地址暴露出来，然后 Prometheus 配置 K8S 的 SD，自动发现 K8S 的服务和 Pod，然后拉取指标，网络通的话，就可以了。

Prometheus 监控 K8S 的原理是什么？

答案：

通过 K8S 的 API，自动发现 K8S 的节点、Pod、Service，然后找到对应的 exporter，拉取指标，比如 kube-state-metrics，暴露 K8S 的资源的指标，然后 Prometheus 拉取，监控 K8S 的状态。

Prometheus 如何收集 K8S 指标？

就是上面的，通过 kube-state-metrics，还有各个 exporter，然后 Prometheus 拉取。

Prometheus 监控 Pod 的哪些指标？

答案：

Pod 的 CPU、内存、网络、磁盘的使用，还有重启次数，状态，就绪探针的状态。

监控 K8S 的状态关注哪些关键指标？

答案：

节点的状态，Pod 的状态，资源的使用率，etcd 的健康，apiserver 的请求延迟，调度的成功率。

一个服务上线要监控哪些核心指标？

答案：

四个黄金指标：延迟、流量、错误、饱和度，也就是请求的延迟，QPS，错误率，资源的使用率。

rate () 和 irate () 的区别和使用场景是什么？

答案：

- rate：计算区间内的平均增长率，比如 rate (http_requests [5m])，是 5 分钟的平均，比较平滑，适合慢变化的指标，适合告警。
- irate：计算区间内的最后两个点的增长率，是瞬时的，比较灵敏，适合图表，展示瞬时的峰值，但是不适合告警，因为太灵敏，容易波动。

Prometheus 输出的都是数字对吧？这个数字分为什么类型？(Counter、Gauge、Histogram、Summary)

答案：

- Counter：计数器，只增不减，比如请求数，重启次数。
- Gauge：仪表盘，可增可减，比如 CPU 使用率，内存使用。
- Histogram：直方图，统计样本的分布，比如请求的延迟的分布，分位数。
- Summary：摘要，和 Histogram 类似，但是客户端计算分位数。

这四个有什么区别？使用场景是什么？

就是上面的，Counter 用来计数，Gauge 用来测当前的状态，Histogram 和 Summary 用来统计延迟的分布。

Prometheus 如何自定义监控指标？举个例子

答案：

比如业务的订单数，在代码里，加一个 Counter，订单创建了，就 incr，然后暴露指标，Prometheus 拉取，就可以监控订单的 QPS 了。

你们普罗米修斯的指标有多少？做了哪些预置？

答案：

大概几千个指标，预置了 Node 的监控，K8S 的监控，MySQL、Redis 的监控，还有业务的监控，还有告警规则。

普罗米修斯具体指标怎么设置？设置多少？

答案：

比如 CPU 使用率，超过 80% 告警，内存超过 80% 告警，磁盘超过 85% 告警，Pod 重启，告警，节点 NotReady，告警。

普罗米修斯计算七天的平均量是怎么算的？函数叫什么？语句是什么？

答案：

用 avg_over_time，比如 `avg_over_time(rate(http_requests[5m])[7d:1m])`，计算 7 天的平均的 QPS。

Prometheus 各个组件和作用是什么？主要监控哪些指标？

就是之前的，Server、Exporter、AlertManager、Grafana，监控系统、业务、中间件的指标。

Prometheus 是部署在 K8S 集群内还是独立物理机上？

答案：

我们是部署在 K8S 集群内的，用 Prometheus Operator，方便管理。

用 Prometheus Operator 还是其他方式部署？两种部署方式各有什么优劣？

答案：

用的 Operator，Operator 的话，自动管理，方便升级，配置，自动处理集群的状态，缺点是复杂一点，手动部署的话，简单，但是维护麻烦，升级麻烦。

Grafana

用过 Grafana 吗？如何让 Prometheus 和 Zabbix 的监控项一起展示？

答案：

用过，Grafana 可以加多个数据源，Prometheus 和 Zabbix，然后在同一个 Dashboard 里，展示两个数据源的指标，一起展示。

Grafana 自己做的界面是怎么做的？用的哪些标签？

答案：

自己创建 Dashboard，然后加 Panel，每个 Panel 展示不同的指标，用 Graph、Stat、Table 这些面板，标签的话，用 Pod、Node 这些标签，过滤指标。

Grafana 和 Prometheus 之间是怎样通信的？

答案：

Grafana 把 Prometheus 作为数据源，Grafana 查询 Prometheus 的 API，获取指标，然后展示。

Grafana 自定义面板、数据展示 Dashboard 怎么做？

答案：

在 Grafana 里，创建 Dashboard，然后添加 Panel，选择数据源，然后写 PromQL，查询指标，然后配置面板的样式，保存，就可以了。

Zabbix

Zabbix 和 Prometheus 有什么区别？

答案：

- Zabbix：传统的监控，push 的模式，Agent 把数据推给 Server，适合传统的物理机、虚拟机，功能全，但是不够灵活，对容器和云原生的支持差点。

- Prometheus：云原生的，pull 的模式，适合动态的环境，比如 K8S，自动发现，灵活，支持自定义指标，但是对传统的监控，差点，而且存储是时序的。

Zabbix 的基本架构是什么？

答案：

Zabbix Server，Agent，数据库，前端，Agent 收集数据，发给 Server，Server 存到数据库，前端展示。

Zabbix 和 Prometheus 后端的数据库有什么不同？

答案：

Zabbix 用 MySQL 或者 PostgreSQL，关系型数据库，Prometheus 用自己的时序数据库，TSDB，适合时序数据的存储，压缩比高，查询快。

Zabbix 自动发现有研究过吗？

答案：

Zabbix 的自动发现，自动发现服务器、服务，不用手动添加，适合动态的环境。

Zabbix 监控突然缺失了一部分怎么排查？

答案：

看 Agent 是不是正常，能不能通，看网络是不是通，看 Server 的日志，有没有错误，看数据库是不是满了。

如果 Zabbix 监控突然缺失了一部分，可能是哪些原因？

答案：

Agent 挂了，网络断了，Server 的问题，数据库的问题，前端的缓存。

Zabbix 服务端优化怎么做？

答案：

分 Proxy，分布式的，把数据分到多个 Proxy，减轻 Server 的压力，优化数据库，分表，索引，缓存。

假设现在有 100 台服务器，怎么用 Zabbix 去部署监控？触发器怎么设计？

答案：

用自动发现，自动添加服务器，不用手动加，触发器的话，CPU 超过 80%，内存超过 80%，磁盘超过 85%，就告警。

告警

告警是通过什么告警的？

答案：

通过钉钉，把告警发到群里，还有邮件，通知相关的人。

告警分级是具体怎么做的？

答案：

分 P0、P1、P2、P3，不同的等级，不同的通知方式，P0 的话，电话 + 短信 + 群，P1 的话，群 + 邮件，P2 的话，群。

如果要实现告警，实现不同组收到告警信息如何配置？

答案：

在 AlertManager 里，配置路由，根据告警的标签，比如服务的标签，把告警发到不同的组，比如业务的告警，发给业务的组，基础的告警，发给运维的组。

报警抑制怎么做？举一下例子

答案：

比如节点宕机了，会触发很多告警，节点的告警，还有上面的 Pod 的告警，这时候，抑制的话，当节点的告警触发了，就抑制 Pod 的告警，只发节点的，避免告警风暴。

设置报警的时候，怎么进行报警分类？

答案：

根据告警的服务，等级，类型，分类，比如系统的，业务的，中间件的，然后不同的分类，发给不同的人。

告警流程是什么？

答案：

Prometheus 触发告警，发给 AlertManager，AlertManager 分组、抑制，然后发给钉钉、邮件，通知人，然后处理，然后恢复。

监控一般监控啥？

答案：

系统的资源，CPU、内存、磁盘、网络，服务的状态，业务的指标，日志，链路。

系统级别监控包括哪些？

答案：

CPU、内存、磁盘、网络、负载、进程、系统的状态。

应用程序级别监控包括哪些？

答案：

QPS、延迟、错误率、线程数、GC、业务的指标。

ELFK/ELK

ELK 了解吗？怎么用的？

答案：

ELK 是 Elasticsearch、Logstash、Kibana，用来收集日志，存储，分析，现在是 EFK，用 Filebeat 代替 Logstash，Filebeat 采集日志，发给 Elasticsearch，Kibana 展示。

ELFK 架构数据流走向是什么？

答案：

日志 ->Filebeat->Elasticsearch->Kibana，Filebeat 采集节点的日志，发给 ES，ES 存储，Kibana 查询展示。

ELFK 一般用来采集什么日志？

答案：

容器的日志，应用的日志，系统的日志，Nginx 的日志，MySQL 的日志，所有的日志。

Filebeat 怎么推数据？

答案：

Filebeat 采集日志，然后把数据推给 Elasticsearch，或者推给 Kafka，缓冲一下，然后再到 ES。

Elasticsearch 分片和副本是什么？

答案：

分片：把索引分成多个分片，分布到不同的节点，提升性能，扩展。

副本：分片的副本，用来高可用，一个分片挂了，副本可以顶上，同时提升读的性能。

ES 集群写入比较慢了怎么办？

答案：

1. 优化刷新的间隔，增加 refresh_interval，减少刷盘的次数。
2. 用 bulk 批量写入，减少请求的次数。
3. 优化磁盘，用 SSD，提升 IO。
4. 调整副本数，写入的时候，先把副本关掉，写完再开。
5. 扩容节点，提升集群的能力。

ES 集群物理机配置是什么？多少个节点？多少个索引？副本数是多少？分片数怎么形成的？

答案：

3 个节点，CPU16 核，内存 32G，索引的话，按天建索引，副本数 1，分片数 5，根据数据量，比如一天 100G，5 个分片，每个 20G。

关于索引、副本数、分片数大概怎么分的？有没有什么性能瓶颈？官方有没有限制？

答案：

按天建索引，副本 1，分片根据数据量，每个分片 20-30G，不要太大，瓶颈的话，分片太多的话，会有压力，官方建议一个节点的分片不要太多，总共的分片不要超过 1000。

如果分片数量太少有什么影响吗？

答案：

分片太少的话，单个分片太大，查询慢，而且无法扩展，因为分片不能拆分，所以如果数据增长的话，单个分片太大，性能差。

在 Logstash 和 Elasticsearch 之间想到什么办法提升性能？

答案：

加 Kafka，做缓冲，把日志先写到 Kafka，然后 ES 慢慢消费，避免高峰期 ES 压垮。

如果在 Elasticsearch 中已经创建了一个索引，现在需要调整其已有的索引结构，比较好的实践方案是什么？

答案：

创建新的索引，修改结构，然后把旧的索引的数据 reindex 到新的索引，然后把别名指向新的索引，这样不用停机。

影响日志检索的响应时间的因素有哪些？

答案：

数据量，分片的数量，查询的复杂度，节点的性能，缓存。

阿里云 / 云服务

ECS 与基础服务

阿里云 ECS 的底层原理熟悉吗？

答案：

ECS 是阿里云的云服务器，基于 KVM 的虚拟化，把物理机虚拟成多个虚拟机，用户可以自己管理，弹性的，按需创建。

阿里云相关的主要做过什么？使用过什么？

答案：

用过 ECS、SLB、CDN、OSS、RDS、Redis、VPC 这些。

用过哪些公有云？

答案：

阿里云，还有腾讯云，AWS。

你们公司用的云厂商是？一般买多大的云服务器？

答案：

阿里云，一般的业务，用 4 核 8G 的，大的用 8 核 16G 的。

ECS 的 OS 镜像类型你实际用过的有哪些？

答案：

CentOS、Ubuntu、还有 Windows，主要是 CentOS。

SLB 与 CDN

SLB 和 CDN 怎么配置？

之前的，CDN 的源站是 SLB。

SLB 和 Nginx 有什么区别？

之前的，SLB 是云的负载，Nginx 是自己的。

有用过云上的 SLB 吗？搭建流程是什么？

答案：

用过，创建 SLB，然后添加后端的 ECS，配置监听，端口，然后配置健康检查，就可以了。

OSS

OSS 和 NAS、云盘有什么区别？主要存储什么数据？主要用于什么实际场景？

答案：

- 云盘：块存储，挂载到 ECS，随机读写，快，用来存系统盘、数据盘，给 ECS 用。
- NAS：文件存储，NFS/SMB，多个 ECS 可以同时挂载，共享存储，适合多节点共享文件。
- OSS：对象存储，用来存静态文件、备份、日志，海量的非结构化数据，访问通过 HTTP，适合存大文件，静态资源。

WAF

你对 WAF 有没有什么了解？WAF 主要做什么？配置过吗？

答案：

WAF 是 Web 应用防火墙，用来防 Web 攻击，比如 SQL 注入、XSS、CC 攻击，配置过，配置规则，防护哪些攻击，黑白名单。

VPC

阿里云 VPC 网络了解吗？里面子网、路由表和安全组之间都是什么关系？

答案：

VPC 是私有网络，隔离的，里面的子网是 VPC 里的网段，每个子网一个可用区，路由表是子网的路由，控制流量的转发，安全组是实例的防火墙，控制实例的出入流量。

云上的网络策略限制是通过什么做的？

安全组，还有网络 ACL。

云平台架构

云平台的组件架构是什么？

答案：

计算、存储、网络，ECS、SLB、OSS、RDS，这些组件，组成云的架构。

OpenStack 包含哪些组件？

答案：

Nova（计算）、Neutron（网络）、Cinder（块存储）、Glance（镜像）、Keystone（认证）、Horizon（前端）、Swift（对象存储）。

你使用过云服务器干过什么？

答案：

部署业务，部署 K8S 集群，部署监控，部署 CI/CD，各种服务。

除了传统 ECS、OSS、SLB、CDN 之外还接触过什么？

答案：

RDS、Redis、MongoDB、Kubernetes、容器服务、消息队列。

系统优化

内核参数

内核参数优化一般优化哪些？

之前的，TCP 的，内存的，文件的。

net.core.somaxconn 的作用是什么？

之前的，连接队列。

net.ipv4.tcp_fastopen 的作用是什么？

之前的，TCP 快速打开。

net.ipv4.tcp_fin_timeout 的作用是什么？

之前的，TIME_WAIT 的超时。

net.ipv4.tcp_tw_reuse 的作用是什么？

之前的，复用 TIME_WAIT。

vm.swappiness 参数的作用是什么？

之前的，swap 的倾向。

vm.overcommit_memory 参数的作用是什么？

之前的，内存超额分配。

vm.dirty_ratio 和 vm.dirty_background_ratio 的作用是什么？

之前的，脏页。

vm.max_map_count 参数的作用是什么？

之前的，内存映射，给 ES 用的。

文件系统优化挂载选项有哪些？(noatime、nodiratime、barrier=0、data=writeback)

答案：

- noatime：不更新访问时间，减少写。
- nodiratime：不更新目录的访问时间。
- barrier=0：关闭 ext3 的 barrier，提升写性能，但是掉电可能丢数据。
- data=writeback：ext4 的写回模式，提升性能。

SSH

SSH 连接不上 Linux 怎么排查？

之前的。

SSH 优化、禁止 root 登录、创建普通账号怎么实现？

之前的，修改 sshd_config。

SSH 配置文件中 PermitRootLogin 和 PasswordAuthentication 的作用是什么？

之前的。

文件描述符

加大文件描述符怎么配置？

之前的，limits.conf。

worker_rlimit_nofile 的作用是什么？

之前的，Nginx 的文件描述符。

时间同步

时间同步写入定时任务怎么配置？

之前的，crontab，ntpd。

安全相关

防火墙

服务器防火墙会配置吗？

会，iptables，firewalld。

查看防火墙开放了哪些端口？

之前的。

iptables 添加规则

之前的。

iptables 怎么去屏蔽特定网段？

之前的。

如何限制某个 IP 访问服务器某个端口？

之前的。

DDoS 与攻击

当遇到 DDoS 攻击时怎么处理？

答案：

1. 先切高防 IP，把流量引到高防，清洗流量。
2. 封禁攻击的 IP，用防火墙封禁。
3. 扩容，提升带宽，扛住流量。
4. 限流，限制请求的频率，防 CC。

配置过防火墙吗？防 DDoS、防 CC？

配置过，用 WAF，防 CC，限制请求的频率。

漏洞修复

安全这一块了解吗？业务上线做漏洞扫描、攻防检测和代码质量检测

答案：

了解，上线之前，做漏洞扫描，扫系统的漏洞，Web 的漏洞，然后做攻防检测，看有没有攻击，代码质量检测，扫描代码的漏洞。

服务器的安全方面，漏洞的修复怎么做？

答案：

定期扫描漏洞，然后更新软件包，修复漏洞，比如内核的漏洞，更新内核，软件的漏洞，更新软件，然后配置防火墙，关闭不用的端口，禁止 root 登录，提升安全。

挖矿病毒

挖矿病毒怎么处理？

答案：

1. 找到挖矿的进程，杀掉，然后删除对应的文件。
2. 清理定时任务，因为挖矿病毒会加定时任务，下载病毒。
3. 清理 SSH 的密钥，有没有陌生的密钥。
4. 改密码，强密码，避免被暴力破解。
5. 排查入侵的入口，比如是不是弱密码，是不是漏洞，修复。

安全工具

你对 WAF 有没有什么了解？WAF 主要做什么？

之前的。

JVM/Java 相关

GC 排查

JVM 调优怎么做？

答案：

调整堆内存的大小，-Xms 和 -Xmx，然后调整 GC 的算法，比如用 G1，然后调整新生代的比例，Survivor 的比例，然后优化 GC 的停顿，减少 Full GC。

Tomcat CPU 利用率高排查流程是什么？

之前的，看线程，看 GC。

导致频繁 GC 的原因和原理是什么？

答案：

原因是创建了很多的对象，内存不够，导致 GC 频繁，比如大对象，或者内存泄漏，对象无法回收，导致老年代满了，频繁 Full GC。

如果有一个 Java 程序突然占用了大量内存与 CPU, 怎么排查？

答案：

1. top，找到进程。
2. ps -mp PID -o THREAD,tid,time，找到线程。
3. jstack，打印线程栈，看线程在做什么。
4. jstat，看 GC 的情况，是不是 GC 频繁。
5. jmap，dump 内存，分析内存泄漏。

GC 日志分析怎么做？使用什么 JVM 参数？

答案：

加参数，-Xloggc:gc.log -XX:+PrintGCDetails -XX:+PrintGCTimeStamps，打印 GC 日志，然后用工具，比如 GCEasy，分析 GC 日志，看 GC 的次数，停顿。

jstat 怎么监控 GC 情况？

答案：

jstat -gc PID 1000，每秒打印一次 GC 的情况，看 S0、S1、Eden、Old 的使用，YGC、FGC 的次数和时间。

内存分析工具 (MAT、JVisualVM、JProfiler) 怎么用？

答案：

用 jmap dump 内存，然后用 MAT 打开，分析内存的对象，看有没有内存泄漏，哪个对象占用了太多的内存。

用什么 APM 工具做深入分析？(Arthas、SkyWalking)

答案：

用过 Arthas，线上的诊断工具，看线程，看 GC，看方法的调用，不用重启，线上诊断，还有 SkyWalking，分布式链路追踪，看请求的链路，排查性能问题。

具体多线程占用情况怎么看？

jstack，看线程的状态，或者 Arthas 的 thread 命令。

内存溢出

内存溢出常见的原因有哪些？(堆内存溢出、栈内存溢出、永久代内存溢出)

答案：

- 堆内存溢出：创建了太多的对象，堆不够了，比如加载了太多的数据，或者内存泄漏。
- 栈内存溢出：线程的栈太深了，比如递归太深，或者线程太多了。
- 永久代 / 元空间溢出：加载了太多的类，永久代满了，比如动态生成了很多的类。

堆转储分析怎么做？jmap 怎么用？

答案：

jmap -dump:format=b,file=dump.hprof PID，dump 内存，然后用 MAT 分析。

代码层面如何减少频繁 GC？

答案：

1. 避免创建太多的临时对象，重用对象。
2. 使用对象池，重用对象。
3. 优化代码，减少大对象。
4. 调整 JVM 的参数，优化 GC。

JVM 配置参数 (-Xms、-Xmx、-XX:NewRatio、-XX:SurvivorRatio、-XX:+UseG1GC) 怎么调？

答案：

- -Xms：堆的初始大小。
- -Xmx：堆的最大大小，一般设置成一样，避免堆扩展。
- -XX:NewRatio：新生代和老年代的比例，比如 2，就是老年代是新生代的 2 倍。
- -XX:SurvivorRatio：Eden 和 Survivor 的比例，比如 8，就是 Eden:Survivor0:Survivor1=8:1:1。
- -XX:+UseG1GC：使用 G1GC 算法，适合大堆，减少停顿。

Tomcat 优化

Tomcat 线程池怎么配置？(server.xml 中的 Executor)

答案：

在 server.xml 里，配置 Executor：

代码块

```
1 <Executor name="tomcatThreadPool" namePrefix="catalina-exec-"
2     maxThreads="200" minSpareThreads="10"/>
```

然后 Connector 里用这个线程池。

Tomcat 负载高排查流程是什么？

之前的。

Tomcat 假死但进程还在怎么解决？

之前的。

公司里面 Tomcat 和哪些中间件搭配使用，优化过什么方面？

之前的，Nginx，优化线程，JVM。

Ansible

基础使用

Ansible 用过哪些模块？

答案：

copy、file、shell、yum、service、template、lineinfile、user、group 这些。

用过 Ansible 吗？工作方式和模块有哪些？

用过，Ansible 是批量管理工具，基于 SSH，不用装 Agent，模块就是上面的。

Ansible 底层的原理是什么？核心依赖什么？

答案：

底层是 SSH，通过 SSH 连接到节点，然后执行模块，模块是 Python 的脚本，发到节点，执行，然后返回结果，核心依赖 SSH 和 Python。

如何通过 Ansible 批量重启 Nginx 服务？

答案：

```
ansible all -m service -a "name=nginx state=restarted"。
```

给你 10 台机器，用 Ansible 查各个机器里边的磁盘情况，命令是什么？

答案：

```
ansible all -m shell -a "df -h"。
```

我想用 Ansible 给一个文件的权限是 777 怎么改？

答案:

```
ansible all -m file -a "path=/file mode=777"。
```

playbook

Ansible 的 playbook 是用来做什么的?

答案:

playbook 是 Ansible 的配置文件, YAML 格式, 把多个任务写在一起, 批量执行, 比如部署服务, 多个步骤, 写在 playbook 里, 一键执行。

会写 playbook 吗?

会, 比如部署 Nginx 的 playbook, 安装, 配置, 启动。

Ansible 都写了哪些 playbook?

写过部署 Java 服务的, 部署监控的, 初始化系统的, 配置内核的。

加密与变量

在 Ansible 的 playbook 中如何对敏感变量进行加密?

答案:

用 ansible-vault, 加密变量文件, 这样敏感密码, 就不会明文了, 执行的时候输入密码。

条件判断与优化

在 Ansible 中如何进行判断, 条件满足的时候才会去执行?

答案:

用 when, 比如:

代码块

```
1  tasks:
2    - name: install nginx
3      yum: name=nginx state=latest
4      when: ansible_os_family == "RedHat"
```

如何优化 Ansible 的执行速度?

答案:

1. 开启 forks, 并行执行, 默认是 5, 调大, 比如 10。
2. 开启 SSH 的复用, 减少连接的时间。
3. 用 pipelining, 减少传输的次数。

4. 把任务合并，减少 SSH 的连接。

故障排查

Ansible 部署服务出现错误了怎么快速恢复？

答案：

看 Ansible 的日志，看哪个任务失败了，然后手动修复，然后重新执行 playbook。

Ansible 故障排查怎么做？

答案：

加 -v 参数，看详细的日志，看执行的输出，然后看节点的日志，看 SSH 是不是通，模块是不是执行成功。

信创 / 国产化

信创项目你了解哪些？其中有遇到什么问题？

答案：

信创就是信息技术应用创新，国产化，替换国外的产品，用国产的，遇到的问题，比如软件的兼容性，国产的系统，有些软件不兼容，还有硬件的兼容性，还有性能的问题，国产的 CPU，性能比 x86 差点。

你对国产操作系统安装了解吗？

了解，比如麒麟系统，安装和 CentOS 差不多，就是包管理器有点不一样。

你在使用国产 Kylin 系统的时候遇到过哪些坑，相比其他操作系统有哪些不好用的地方？

答案：

比如有些软件的包没有，需要自己编译，还有驱动的问题，有些硬件的驱动不好，还有命令的差异，有些命令不一样，还有桌面的体验，不如 Ubuntu。

国产化操作系统用到过哪些？

麒麟、统信。

说一下你了解的信创产品？

答案：

操作系统：麒麟、统信，数据库：达梦、人大金仓，中间件：东方通，CPU：鲲鹏、飞腾。

Kylin 操作系统部署服务遇到过哪些问题？

答案：

比如 Docker 的版本，有些版本的 Docker 不兼容，还有 JDK 的版本，有些 JDK 在麒麟上有问题，还有依赖的问题，有些依赖的包没有。

用 Kylin 时 Docker 有没有遇到过镜像不兼容？

答案：

遇到过，因为 Kylin 是 ARM 的，x86 的镜像不能用，需要用 ARM 的镜像。

军工单位涉及软件包认证，要打 RPM 包，了解过吗？

答案：

了解，把软件打包成 RPM 包，然后做签名，认证，这样可以安装，符合军工的要求。

公司涉及很多设备都不能联网，部署服务有什么解决办法？

答案：

用离线的源，把所有的包下载下来，做成本地的源，然后离线安装，或者用离线的镜像，把所有的镜像导出，然后导入到离线的机器。

KVM 虚拟化

什么是 KVM? 它的全称是什么？

答案：

KVM 是 Kernel-based Virtual Machine，基于内核的虚拟机，是 Linux 的虚拟化技术，把 Linux 变成一个 hypervisor，运行虚拟机。

KVM 与 Xen、VMware ESXi 在架构上有什么主要区别？

答案：

- KVM：集成到 Linux 内核，轻量，开源，性能好，适合 Linux。
- Xen：半虚拟化，需要修改内核，性能也不错，但是不如 KVM 流行。
- VMware：商业的，闭源，功能全，支持 Windows，性能好，但是收费。

KVM 主要由哪两个组件组成？它们各自的作用是什么？

答案：

KVM 的模块，kvm.ko，是内核模块，管理虚拟机的 CPU 和内存，还有 virtio，是半虚拟化的驱动，提升 IO 的性能。

解释一下 /dev/kvm 这个设备文件的作用

答案：

/dev/kvm 是 KVM 的字符设备，用户空间的 QEMU，通过这个设备，和 KVM 内核模块交互，创建虚拟机，管理虚拟机。

KVM 是如何实现 CPU 虚拟化的？

答案：

通过硬件虚拟化，Intel 的 VT-x，AMD 的 AMD-V，把 CPU 的指令集虚拟化，虚拟机的指令直接交给硬件，不用 QEMU 模拟，所以性能高。

KVM 的内存虚拟化是如何工作的？影子页表和 EPT 是什么？

答案：

内存虚拟化，把主机的物理地址，转换成虚拟机的物理地址，影子页表，是 QEMU 维护的页表，把虚拟机的虚拟地址，映射到主机的物理地址，EPT 是硬件的扩展，硬件来做地址转换，不用影子页表，性能更高。

KVM 的 I/O 虚拟化主要有哪几种方式？比较优缺点

答案：

1. 纯模拟：QEMU 模拟 IO 设备，慢，兼容性好。
2. 半虚拟化：virtio，虚拟机的驱动，直接和主机交互，快，但是需要虚拟机装驱动。
3. 硬件辅助的 IO 虚拟化：VT-d，直接把 PCI 设备透传给虚拟机，性能最好，和物理机一样。

什么是 vhost? 它相对于传统的 virtio 有什么改进？

答案：

vhost 是内核的驱动，把 virtio 的处理放到内核，减少用户态和内核态的切换，提升 IO 的性能，比传统的 virtio 更快。

使用过哪些 KVM 管理工具？比较 libvirt 和 qemu-kvm 命令行

答案：

用过 libvirt，virsh，还有 qemu-kvm，qemu-kvm 是命令行，直接启动虚拟机，libvirt 是管理工具，封装了 qemu，更方便，管理虚拟机，支持远程，API。

如何查看一个宿主机上是否支持并开启了 KVM？

答案：

```
egrep -c '(vmx|svm)' /proc/cpuinfo
```

，如果输出大于 0，说明支持，然后 `lsmod | grep kvm`，看 kvm 模块有没有加载。

如何使用 virsh 命令启动、暂停、关闭和销毁虚拟机？

答案：

- virsh start vm：启动。
- virsh suspend vm：暂停。
- virsh shutdown vm：关闭。
- virsh destroy vm：强制销毁。

虚拟机的配置文件通常存放在哪里？格式是什么？

答案：

/etc/libvirt/qemu/，XML 格式。

如何对 KVM 虚拟机进行 CPU 和内存的过热使用？

答案：

virsh setvcpus，virsh setmem，动态调整，不用重启。

什么是 CPU 亲和性？为什么要设置它？

答案：

把虚拟机的线程，绑定到宿主机的物理 CPU 核心，这样可以避免 CPU 的切换，提升缓存的命中率，提升性能。

常见的 KVM 性能优化技巧有哪些？

答案：

1. 用 virtio 驱动，半虚拟化。
2. CPU 亲和性，绑定 CPU。
3. 内存大页，提升内存的性能。
4. 磁盘用 cache=none，或者 directio，提升 IO。
5. 网络用 vhost，提升网络性能。

什么是 QCOW2 格式？优缺点是什么？

答案：

QCOW2 是 KVM 的磁盘格式，精简置备，用多少占多少，支持快照，支持加密，优点是节省空间，有快照，缺点是比 raw 的性能差点。

如果一个 KVM 虚拟机无法启动，排查思路是什么？

答案：

1. 看 libvirt 的日志，/var/log/libvirt/，看错误。
2. 看虚拟机的配置，是不是正确，磁盘是不是存在。
3. 看资源，CPU、内存是不是够。
4. 看 qemu 的日志，有没有错误。

如何监控 KVM 虚拟机的性能？

答案：

用 libvirt 的指标，或者用 virt-top，看虚拟机的 CPU、内存、IO，或者用 Prometheus 的 libvirt_exporter，采集指标。

你了解哪些虚拟化手段？它们之间有什么区别？

答案：

KVM、Xen、VMware，还有容器，KVM 是硬件虚拟化，完全隔离，容器是轻量的，共享内核，隔离差点。

中间件与其他

消息队列

使用过哪些消息队列中间件？

答案：

用过 RabbitMQ、Kafka、RocketMQ。

如何保证消息不丢失的机制？

答案：

1. 生产者的确认，生产者发送消息，收到 Broker 的 ack。
2. Broker 的持久化，消息写到磁盘，持久化。
3. 消费者的 ack，消费者处理完消息，再 ack，Broker 才删除消息。

Kafka 了解多少？如果想从头开始消费数据应该怎么做？

答案：

Kafka 的消费者，重置 offset，比如 `kafka-consumer-groups.sh`，重置 offset 到最早的，就可以从头消费。

partition 数量是越大越好吗？

答案：

不是，partition 是并发的单位，越大，并发越高，但是太大的话，会有很多的文件，元数据的压力大，所以根据并发，一般每个 partition 的 QPS 几千，就够了。

Maven/Nexus

Maven 是什么？Nexus 私服的作用是什么？

答案：

Maven 是 Java 的项目管理工具，管理依赖，构建项目，Nexus 是私服，用来缓存 Maven 的依赖，自己的私有包，不用每次都去中央仓库下载，提升速度，管理私有包。

Cookie 与 Session

Cookie 与 Session 的区别是什么？

答案：

- Cookie：存在客户端，浏览器，存用户的信息，大小 4K，不安全，用户可以改。
- Session：存在服务端，存用户的信息，大小没限制，安全，通过 SessionID 关联 Cookie。

Jar 与 War 包

Jar 包部署和 War 包部署的区别是什么？

答案：

- Jar 包：内置了 Tomcat，直接 `java -jar` 就可以运行，独立的，不用外部的 Tomcat。
- War 包：需要部署到外部的 Tomcat 里，把 War 包放到 Tomcat 的 webapp 里，Tomcat 运行它。

RPM 打包

RPM 包的编译和打包了解吗？

答案：

了解，把软件打包成 RPM 包，方便安装，管理依赖，用 `rpmbuild`，写 spec 文件，然后打包。

笔试题

选择 / 填空题

在 Linux 系统中，用于查看当前系统进程状态的核心命令是什么？

答案：ps、top。

能强制终止指定 PID 进程的是什么命令？

答案：kill -9。

Docker 中，用于查看本地已下载镜像列表的命令是什么？

答案：docker images。

在 Linux 系统中，要将文件 file1 从 /opt 目录移动到 /home 目录，正确的命令是什么？

答案：mv /opt/file1 /home/。

Docker Compose 的核心配置文件默认名称是什么？

答案：docker-compose.yml。

Linux 系统中，用于查看系统负载情况 (1 分钟、5 分钟、15 分钟) 的命令是什么？

答案: uptime。

能查看 Docker 容器实时日志的是什么命令?

答案: docker logs -f。

Linux 系统中, 要给 test.sh 添加可执行权限, 正确的命令是什么?

答案: chmod +x test.sh。

Docker 中, 用于停止并删除指定容器的命令组合是什么?

答案: docker stop && docker rm。

在 Linux 系统中, 用于查看指定目录下文件详细信息的命令是什么?

答案: ls -l, 或者 stat。

取消挂载磁盘的命令是什么? 强制取消挂载的命令是什么?

答案: umount, umount -f。

ping 使用的是什么协议? 工作在几层?

答案: ICMP, 网络层。

交换机一般工作在几层? 路由器工作在几层?

答案: 交换机数据链路层, 路由器网络层。

系统最大端口号是多少?

答案: 65535。

网段范围 10.10.0.0/19 换算成子网掩码是多少?

答案: 255.255.224.0。

centos 系统下执行 uptime 命令可以查看到的内容都有什么?

答案: 当前时间、运行时间、登录用户数、1/5/15 分钟负载。

last 命令后的 -x 选项代表什么意思?

答案: 显示系统的关机、重启的记录。

chmod 命令是用来做什么? chown 命令是用来做什么的?

答案: chmod 改权限, chown 改属主属组。

操作 / 简答题

请写出 Linux 系统中查询 /var/log 目录下近 24 小时内修改过的以.log 结尾的文件的命令，并说明各参数含义

答案：

```
find /var/log -type f -name "*.log" -mtime -1
```

- -type f: 文件。
- -name: 名字匹配.log 结尾。
- -mtime -1: 修改时间在 1 天内。

请写出 Docker 环境中基于 nginx:latest 镜像创建一个名为 nginx-web 的容器的命令，要求后台运行、端口映射、目录挂载

答案：

代码块

```
1 docker run -d --name nginx-web -p 80:80 -v /host/html:/usr/share/nginx/html
  nginx:latest
```

-d 后台，--name 名字，-p 端口映射，-v 挂载。

Linux 系统中，若发现某台服务器的 sshd 服务无法启动，请写出至少 3 步排查思路及对应的核心命令

答案：

1. 看端口是不是被占用了： `netstat -tulnp | grep 22`。
2. 看配置文件是不是错了： `sshd -t`，测试配置。
3. 看日志： `journalctl -u sshd`，看错误。

请写出 Docker 中批量停止所有运行中容器、批量删除所有已停止容器、批量删除所有未被使用的镜像的命令

答案：

代码块

```
1 docker stop $(docker ps -q)
2 docker container prune
3 docker image prune -a
```

在阿里云 ECS 服务器中，需要定期每天凌晨 2 点执行一个脚本进行数据备份，核心步骤及命令是什么？

答案：

1. 写备份脚本，backup.sh。
2. 加执行权限：chmod +x backup.sh。
3. 配置 crontab： `0 2 * * * /root/backup.sh >> /var/log/backup.log 2>&1`。

写出用 tar 打包压缩 /home/ 目录所有文件并放到 /root/ 目录下名称为 home.tar.gz 的完整命令，并写出解压到 /aaa/bbb/ 目录的完整命令

答案：

打包： `tar zcvf /root/home.tar.gz /home/`

解压： `tar zxvf /root/home.tar.gz -C /aaa/bbb/`

请描述 Linux 系统常用命令以及命令简介

之前的。

请描述常用的 JVM 调优的参数都有哪些

之前的。

请描述 Cookie 与 Session 的区别

之前的。

请描述 Redis 的基本数据类型有哪些

之前的。

请描述使用过的消息队列中间件以及如何保证消息不丢失的机制

之前的。

请描述 MySQL 常用的存储引擎以及区别

之前的。

请描述日常工作中优化 SQL 的方案有哪些？

之前的。

请描述 DevOps 中的 CI 和 CD 分别是做什么用的，优点是什么？

之前的。

请描述 Jar 包部署和 War 包部署的区别

之前的。

请描述常用的反向代理和负载均衡软件以及应用场景

答案：

Nginx：七层，反向代理，负载均衡，Web 服务。

LVS：四层，负载均衡，入口。

HAProxy：四层和七层，负载均衡。

SLB：云的负载均衡。

请描述 MySQL 数据库源码编译工具 (版本 5.6 及以上)

cmake。

请描述 MySQL 数据库源码编译常用参数 (版本 5.6 及以上)

比如 CMAKE_INSTALL_PREFIX, MYSQL_DATADIR, WITH_INNODB_STORAGE_ENGINE 这些。

请描述 MySQL 数据库编译部署后常用初始化命令 (版本 5.6 及以上)

mysqld --initialize。

如何从学生表查询到分数大于等于 80 的所有男性姓名？

答案：

```
select name from student where score >=80 and gender='male';
```

如何将表里的分数在 70 到 80 的女学生信息删除？

答案：

```
delete from student where score between 70 and 80 and gender='female';
```

如何创建一个新表？

答案：

```
create table student (id int, name varchar(20), score int);
```

综合笔试

在 Linux 系统中，磁盘管理相关的命令有哪些？

df、du、fdisk、mkfs、mount、umount。

Docker 容器的数据持久化方案包括哪些？

之前的，volume、bind mount、tmpfs。

可用于查看网络连接状态的命令有哪些？

netstat、ss、tcpdump。

关于 Docker 镜像的说法正确的有哪些？

分层、可共享、UnionFS。

(注：文档部分内容可能由 AI 生成)